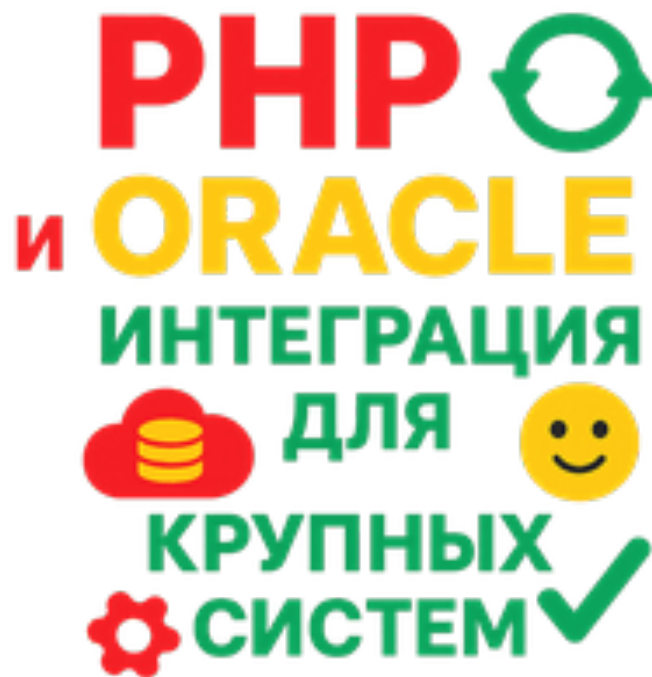




PHP и Oracle Интеграция для крупных систем

Модуль 1 - Введение в интеграцию PHP и Oracle

Урок 1 - Как PHP взаимодействует с Oracle

Шаг 01:

Миссия 1

Анна только что устроилась в крупную IT-компанию. Её первая задача — подключить PHP к корпоративной базе Oracle. Какой PHP-расширение обычно используется для работы с Oracle? Подсказка: Это расширение позволяет работать с OCI (Oracle Call Interface).

Ответ:

- OCI8
- PDO_MySQL
- SQLite3
- mysqli

Шаг 02:

Миссия 2

Сергей пишет скрипт для чтения данных из Oracle. Какой правильный порядок действий при подключении к БД?

Подсказка: Вспомните: сначала соединение, затем запрос, потом выборка результатов.

Ответ:

- **Подключение → Выполнение запроса → Получение данных**
- Получение данных → Подключение → Выполнение запроса
- Выполнение запроса → Подключение → Получение данных
- Подключение → Получение данных → Выполнение запроса

Шаг 03:

Миссия 3

Ирина пробует выполнить первый запрос к Oracle. Какой PHP-функцией выполняется SQL-запрос через OCI8?

Подсказка: Сначала запрос подготавливается, затем выполняется.

Ответ:

- **oci_execute()**
- mysqli_query()
- pdo_query()
- pg_execute()

Шаг 04:

Миссия 4

Михаил подключается к тестовой базе. Ему нужно проверить, удалось ли подключение. Какая функция вернёт ресурс соединения?

Подсказка: Используйте стандартную функцию OCI для подключения.

Ответ:

- **oci_connect()**
- oci_bind_by_name()
- oci_parse()
- oci_free_statement()

Шаг 05:

Миссия 5

Никита заметил, что в его SQL-запросе есть параметры. Какой функцией можно безопасно привязать переменные в OCI8?

Подсказка: Эта функция помогает избежать SQL-инъекций.

Ответ:

- **oci_bind_by_name()**
- oci_execute()
- oci_fetch_array()
- oci_new_descriptor()

Шаг 06:

Миссия 6

Ольга получила ошибку при выполнении запроса. Какой функцией она может получить сообщение об ошибке в OCI8?

Подсказка: Эта функция возвращает массив с кодом и текстом ошибки.

Ответ:

- **oci_error()**
- **oci_fetch_assoc()**
- **oci_close()**
- **oci_define_by_name()**

 **Шаг 07:**

Миссия 7

Виктория заметила, что её запрос занимает много памяти. Как правильно освободить ресурсы после выполнения запроса?

Подсказка: Эта функция освобождает указатель на результат.

Ответ:

- **oci_free_statement()**
- **oci_close()**
- **oci_error()**
- **oci_parse()**

 **Шаг 08:**

Миссия 8

 **Задача:** Сопоставьте функции OCI8 с их назначением.

Подсказка: Подумайте, что делают функции: подключение, подготовка, выполнение, выборка.

Ответ:

- **oci_connect()** — Создает соединение с базой данных Oracle.
- **oci_parse()** — Готовит SQL-запрос к выполнению.
- **oci_execute()** — Запускает подготовленный SQL-запрос.
- **oci_fetch_assoc()** — Получает результат запроса в виде ассоциативного массива.

 **Шаг 09:**

Миссия 9

 **Задача:** Сопоставьте понятия, связанные с безопасностью при работе с Oracle.

Подсказка: Вспомните: защита от SQL-инъекций, хранение паролей, обработка ошибок.

Ответ:

- **oci_bind_by_name()** — Привязывает параметры и защищает от SQL-инъекций.
- **oci_error()** — Возвращает информацию об ошибке.
- **Хэширование паролей** — Используется для безопасного хранения учетных данных.
- **Роли Oracle** — Определяют права доступа пользователя.

 **Шаг 10:**

Миссия 10

 **Задача:** Сопоставьте типы данных Oracle с их назначением.

Подсказка: Вспомните, какой тип для строк, какой для чисел, какой для дат.

Ответ:

- **VARCHAR2** — Строковый тип переменной длины.
- **NUMBER** — Числовой тип данных.

- **DATE** — Хранение даты и времени.
- **CLOB** — Хранение больших текстов.

Шаг 11:

Миссия 11

 Задача: Сопоставьте элементы архитектуры интеграции.

Подсказка: Вспомните: PHP отвечает за бизнес-логику, Oracle — за хранение данных.

Ответ:

- **PHP** — Реализует бизнес-логику приложения.
- **Oracle Database** — Хранит данные и обеспечивает их целостность.
- **OCI8** — Мост между PHP и Oracle.
- **SQL-запрос** — Инструмент для выборки и изменения данных.

Шаг 12:

Миссия 12

 Задача: Сопоставьте команды с их результатами.

Подсказка: Подумайте, какая команда возвращает таблицу, какая изменяет данные.

Ответ:

- **SELECT** — Возвращает данные из таблицы.
- **INSERT** — Добавляет новые записи.
- **UPDATE** — Изменяет существующие записи.
- **DELETE** — Удаляет записи.

Шаг 13:

Миссия 13

Расставьте шаги подключения PHP к Oracle в правильном порядке.

Подсказка: Вспомните: подключение, подготовка, привязка, выполнение, выборка.

Ответ:

- Создать соединение с Oracle (`oci_connect`).
- Подготовить запрос (`oci_parse`).
- Привязать параметры (`oci_bind_by_name`).
- Выполнить запрос (`oci_execute`).
- Выбрать данные (`oci_fetch_assoc`).

Шаг 14:

Миссия 14

Расположите этапы работы программиста при отладке интеграции PHP и Oracle.

Подсказка: Сначала нужно воспроизвести проблему, потом проверить код, затем исправить.

Ответ:

- Воспроизведение ошибки.
- Проверка SQL-запроса.
- Исправление PHP-кода.
- Тестирование результата.

Шаг 15:

Миссия 15

Расположите шаги обработки ошибок при работе с Oracle в PHP.

Подсказка: Сначала попытка выполнить действие, затем проверка ошибок, обработка и логирование.

Ответ:

- Выполнить SQL-запрос.
- Проверить наличие ошибки (oci_error).
- Вывести или обработать сообщение.
- Записать в лог.

Урок 2 - Установка и настройка Oracle Instant Client

Шаг 01:

Миссия 1

Анна готовит свой компьютер для работы с Oracle. Что нужно установить, чтобы PHP мог подключаться к Oracle Database?

Подсказка: Это специальный клиент от Oracle, упрощающий взаимодействие.

Ответ:

- **Oracle Instant Client**
- MySQL Workbench
- PostgreSQL Driver
- SQLite Studio

Шаг 02:

Миссия 2

Сергей скачал архив Instant Client. Какой набор файлов обязательно нужен для базового подключения?

Подсказка: Вспомните: нужны библиотеки для OCI и сетевые файлы.

Ответ:

- **Basic Package (oci.dll/so и зависимые библиотеки)**
- SDK Package (только для компиляции)
- SQL*Plus Utility
- Oracle Enterprise Manager

Шаг 03:

Миссия 3

Ирина работает на Windows. Куда лучше всего распаковать архив Oracle Instant Client?

Подсказка: Важно, чтобы путь был простым, без пробелов.

Ответ:

- **C:\oracle\instantclient**
- C:\Program Files\Oracle Client
- C:\Users\Ирина\Documents
- D:\RandomFolder

Шаг 04:

Миссия 4

Михаил настраивает переменные окружения. Какую переменную нужно указать, чтобы PHP нашёл библиотеки Instant Client?

Подсказка: Это системная переменная, содержащая пути к DLL/SO файлам.

Ответ:

- **PATH**
- TNS_ADMIN
- LD_LIBRARY_PATH_ONLY
- ORACLE_HOME

Шаг 05:

Миссия 5

Никита подключает OCI8 в PHP. Какой файл PHP нужно отредактировать, чтобы загрузить расширение?

Подсказка: Это основной конфигурационный файл PHP.

Ответ:

- **php.ini**
- httpd.conf
- my.cnf
- tnsnames.ora

Шаг 06:

Миссия 6

Ольга включает расширение OCI8. Какая строка должна быть добавлена в php.ini?

Подсказка: В Windows строка начинается с слова extension.

Ответ:

- **extension=oci8**
- load=oracle
- include=oci
- driver=oracle_client

Шаг 07:

Миссия 7

Алексей проверяет, загружено ли расширение OCI8. Какую функцию PHP он может использовать?

Подсказка: Эта функция показывает список всех подключённых модулей.

Ответ:

- **phpinfo()**
- oci_connect()
- extension_loaded()
- get_loaded_extensions()

Шаг 08:

Миссия 8

Виктория работает в Linux. Какую переменную окружения нужно настроить, чтобы система находила библиотеки Instant Client?

Подсказка: Эта переменная указывает пути для динамических библиотек.

Ответ:

- **LD_LIBRARY_PATH**
- HOME
- TMPDIR
- LANG

Шаг 09:

Миссия 9

 Задача: Сопоставьте пакеты Instant Client с их назначением.

Подсказка: Вспомните: один пакет для подключения, другой — для утилит.

Ответ:

- **Basic** — Минимальный набор библиотек для подключения.
- **SQL*Plus** — Консольная утилита для работы с SQL-запросами.
- **SDK** — Набор заголовочных файлов для разработки.
- **ODBC** — Драйвер для доступа к Oracle через ODBC.

Шаг 10:

Миссия 10

 Задача: Сопоставьте операционную систему и способ установки Instant Client.

Подсказка: На Windows чаще используют MSI или ZIP, а на Linux — RPM или ZIP.

Ответ:

- **Windows** — Установка через MSI или распаковка ZIP.
- **Linux** — Установка через RPM или распаковка ZIP.
- **macOS** — Использование DMG или Homebrew.
- **Docker** — Подключение пакета через образ Oracle.

Шаг 11:

Миссия 11

 Задача: Сопоставьте переменные окружения с их назначением.

Подсказка: PATH — для бинарников, TNS_ADMIN — для конфигураций.

Ответ:

- **PATH** — Поиск исполняемых файлов и библиотек.
- **TNS_ADMIN** — Путь к файлам sqlnet.ora и tnsnames.ora.
- **LD_LIBRARY_PATH** — Места поиска динамических библиотек в Linux.
- **ORACLE_HOME** — Корневая папка установки Oracle Database (необязательно для Instant Client).

Шаг 12:

Миссия 12

 **Задача:** Сопоставьте инструмент с его ролью при настройке.

Подсказка: `php.ini` отвечает за PHP, `sqlnet.ora` — за сетевые параметры.

Ответ:

- **php.ini** — Включает расширение OCI8.
- **sqlnet.ora** — Определяет правила подключения.
- **tnsnames.ora** — Содержит алиасы для подключения к базам.
- **listener.ora** — Управляет параметрами слушателя Oracle.

Шаг 13:

Миссия 13

 **Задача:** Сопоставьте проблему и её решение при установке Instant Client.

Подсказка: Если система не находит библиотеки — проверьте `PATH` или `LD_LIBRARY_PATH`.

Ответ:

- **PHP не видит oci8** — Добавить `extension=oci8` в `php.ini`.
- **Система не находит dll/so** — Проверить `PATH` или `LD_LIBRARY_PATH`.
- **Ошибка TNS** — Проверить содержимое `tnsnames.ora`.
- **Нет SQL*Plus** — Установить дополнительный пакет `SQL*Plus`.

Шаг 14:

Миссия 14

Расположите шаги установки Oracle Instant Client на Windows.

Подсказка: Сначала скачайте архив, затем распакуйте, потом настройте `PATH`.

Ответ:

- Скачать ZIP с сайта Oracle.
- Распаковать в `C:\oracle\instantclient`.
- Добавить путь в `PATH`.
- Отредактировать `php.ini` для загрузки OCI8.
- Перезапустить веб-сервер.

Шаг 15:

Миссия 15

Расположите шаги установки Oracle Instant Client на Linux.

Подсказка: Используйте пакетный менеджер или архив, не забудьте `LD_LIBRARY_PATH`.

Ответ:

- Скачать RPM или ZIP.
- Распаковать/установить пакет.
- Настроить `LD_LIBRARY_PATH`.
- Отредактировать `php.ini` для включения OCI8.
- Перезапустить Apache/Nginx.

Шаг 16:

Миссия 16

Расположите шаги проверки корректности установки Instant Client.

Подсказка: После настройки всегда тестируйте `phpinfo` и соединение.

Ответ:

- Создать test.php с phpinfo().
- Открыть страницу и найти OCI8.
- Выполнить oci_connect для тестового подключения.
- Проверить отсутствие ошибок в логах.

 Шаг 17:

Миссия 17

Расположите шаги обновления версии Instant Client.

Подсказка: Всегда сначала удаляйте старую папку, потом указывайте новый путь.

Ответ:

- Скачать новую версию Instant Client.
- Удалить старую папку/архив.
- Распаковать новую версию.
- Обновить PATH или LD_LIBRARY_PATH.
- Перезапустить сервер и проверить phpinfo().

 Урок 3 - Основы работы с PHP OCI8

 Шаг 01:

Миссия 1

Андрей только подключает расширение OCI8 в PHP. Какой шаг он должен сделать первым?

Подсказка: Подумайте, без чего PHP не сможет даже запустить функции OCI.

Ответ:

- **Установить и подключить расширение OCI8 в php.ini**
- Написать SQL-запрос сразу в PHP-коде
- Скачать драйвер MySQL
- Изменить права доступа к Apache

 Шаг 02:

Миссия 2

Марина запускает скрипт и получает ошибку: Call to undefined function oci_connect() . Что это значит?

Подсказка: Ошибка намекает, что PHP не знает про нужную функцию.

Ответ:

- **Расширение OCI8 не установлено или не подключено**
- В базе нет нужной таблицы
- Apache не запущен
- SQL-запрос содержит ошибку

 Шаг 03:

Миссия 3

Иван решил проверить соединение. Какую функцию OCI8 он должен использовать для подключения?

Подсказка: В названии функции есть слово connect.

Ответ:

- **oci_connect()**
- oci_execute()
- oci_parse()
- oci_commit()

 **Шаг 04:**

Миссия 4

Елена подключается к базе с помощью `oci_connect()` . Какие параметры обязательны?
Подсказка: Вспомните, без чего невозможно пройти аутентификацию и указать базу.

Ответ:

- **Имя пользователя**
- **Пароль**
- **Имя подключения (SID или сервис)**
- Номер версии PHP

 **Шаг 05:**

Миссия 5

Сергей подключился к базе, но его запрос не выполняется. Какую функцию он забыл вызвать после `oci_parse()` ?

Подсказка: Функция отвечает за фактическое выполнение подготовленного SQL.

Ответ:

- **oci_execute()**
- oci_commit()
- oci_fetch_array()
- oci_connect()

 **Шаг 06:**

Миссия 6

Дима написал SELECT-запрос и хочет получить строки в виде массива. Какую функцию он применит?

Подсказка: Эта функция умеет возвращать данные в массиве, а не просто ресурс.

Ответ:

- **oci_fetch_array()**
- oci_execute()
- oci_commit()
- oci_connect()

 **Шаг 07:**

Миссия 7

 **Задача:** Сопоставьте функции OCI8 с их назначением. Анна готовит шпаргалку для команды.

Подсказка: Смотрите на ключевые слова в названиях функций.

Ответ:

- **oci_parse()** — Подготавливает SQL-запрос для выполнения

- **oci_execute()** — Выполняет ранее подготовленный SQL-запрос
- **oci_commit()** — Фиксирует изменения в базе
- **oci_fetch_array()** — Получает строку результата в виде массива

🔧 Шаг 08:

Миссия 8

📖 Задача: Максим объясняет стажёрам разницу между типами соединений. Сопоставьте типы и описание.

Подсказка: Слово p в названии функции намекает на persistent (постоянное).

Ответ:

- **oci_connect()** — Создаёт новое соединение каждый раз
- **oci_pconnect()** — Создаёт постоянное соединение, используемое повторно
- **oci_new_connect()** — Явно создаёт новое соединение, минуя кеш
- **oci_close()** — Закрывает соединение

🔧 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте константы с их использованием. Олег проверяет, какие режимы fetch есть.

Подсказка: Каждая константа управляет форматом массива при выборке.

Ответ:

- **OCI_ASSOC** — Возвращает массив с ключами-именами столбцов
- **OCI_NUM** — Возвращает массив с числовыми индексами
- **OCI_BOTH** — Возвращает массив с индексами и именами одновременно
- **OCI_RETURN_NULLS** — Управляет возвращением NULL-значений

🔧 Шаг 10:

Миссия 10

📖 Задача: Лена разбирается с транзакциями. Сопоставьте действия и функции.

Подсказка: commit и rollback — стандартные операции транзакций.

Ответ:

- **oci_commit()** — Подтверждает изменения в базе
- **oci_rollback()** — Отменяет несохранённые изменения
- **oci_execute()** — Может управлять автофиксацией изменений
- **oci_error()** — Возвращает информацию об ошибке

🔧 Шаг 11:

Миссия 11

📖 Задача: Тимур показывает разницу в получении строк. Сопоставьте функции и их описание.

Подсказка: Смотрите, одна возвращает ресурс, другая сразу ассоциативный массив.

Ответ:

- **oci_fetch()** — Получает следующую строку результата, но не возвращает её
- **oci_fetch_assoc()** — Возвращает строку как ассоциативный массив
- **oci_fetch_row()** — Возвращает строку как числовой массив
- **oci_fetch_object()** — Возвращает строку как объект

Шаг 12:

Миссия 12

 Задача: София объясняет разницу в обработке больших данных. Сопоставьте термины и объяснения.

Подсказка: LOB — это работа с большими объектами (Large OBjects).

Ответ:

- **CLOB** — Хранит большие текстовые данные
- **BLOB** — Хранит бинарные данные (картинки, файлы)
- **BFILE** — Хранит ссылку на внешний файл в ФС
- **NCLOB** — Хранит текстовые данные в национальной кодировке

Шаг 13:

Миссия 13

Владимир пишет скрипт для выборки данных. Расположите шаги в правильном порядке.

Подсказка: Подумайте: сначала соединение, потом подготовка, выполнение и получение результата.

Ответ:

- `oci_connect()`
- `oci_parse()`
- `oci_execute()`
- `oci_fetch_array()`

Шаг 14:

Миссия 14

Мария проверяет транзакции. Расположите шаги, чтобы изменения были зафиксированы.

Подсказка: Не забудьте, что `commit` идёт только после `execute`.

Ответ:

- `oci_connect()`
- `oci_parse()`
- `oci_execute()`
- `oci_commit()`

Шаг 15:

Миссия 15

Даниил тренируется на `INSERT`-запросе. Расположите шаги для правильного выполнения с проверкой ошибок.

Подсказка: В конце полезно узнать, всё ли прошло успешно.

Ответ:

- `oci_connect()`
- `oci_parse()`
- `oci_execute()`
- `oci_error()`

🔔 Шаг 16:

Миссия 16

Игорь создаёт скрипт с обновлением таблицы. Расположите шаги, если нужно откатить изменения при ошибке.

Подсказка: Вспомните: rollback используется вместо commit.

Ответ:

- oci_connect()
- oci_parse()
- oci_execute()
- oci_rollback()

📖 Урок 4 - Первые запросы в Oracle через PHP

🔔 Шаг 01:

Миссия 1

Андрей подключает PHP к Oracle и пишет свой первый запрос. Какую функцию он должен использовать для установления соединения?

Подсказка: В PHP OCI8 есть специальные функции для подключения к базе.

Ответ:

- **oci_connect()**
- mysqli_connect()
- pdo_connect()
- oracle_connect_db()

🔔 Шаг 02:

Миссия 2

Марина написала запрос SELECT в Oracle через PHP, но забыла выполнить его. Какая функция выполняет подготовленный запрос?

Подсказка: Сначала мы подготавливаем выражение через oci_parse() , а затем...

Ответ:

- **oci_execute()**
- oci_run()
- oci_start()
- oci_launch()

🔔 Шаг 03:

Миссия 3

Виталий получил ресурс результата после oci_parse() . Какая функция в OCI8 используется для выборки одной строки данных?

Подсказка: Эта функция возвращает массив с данными из запроса.

Ответ:

- **oci_fetch_array()**
- oci_fetch_row()
- oci_result()
- oci_read_line()

Шаг 04:

Миссия 4

Оля пишет код на PHP и хочет убедиться, что соединение с Oracle закрыто после работы. Какая функция используется?

Подсказка: Закрытие соединения в OCI8 похоже по смыслу на `close()` .

Ответ:

- **`oci_close()`**
- `oci_disconnect()`
- `oci_stop()`
- `oci_exit()`

Шаг 05:

Миссия 5

Игорь заметил, что его `SELECT` возвращает несколько строк. Какая функция позволяет получить все строки сразу в массив?

Подсказка: Она собирает сразу целый набор результатов.

Ответ:

- **`oci_fetch_all()`**
- `oci_fetch_array()`
- `oci_get_all()`
- `oci_read_all()`

Шаг 06:

Миссия 6

Сергей тестирует `SELECT`-запрос, но случайно вызвал `INSERT`. Чтобы изменения сохранились в Oracle, какую функцию нужно вызвать?

Подсказка: Вспомни транзакции — они завершаются подтверждением.

Ответ:

- **`oci_commit()`**
- `oci_finish()`
- `oci_save()`
- `oci_end()`

Шаг 07:

Миссия 7

Анна выполняет запрос, но сервер Oracle вернул ошибку. Какой функцией в PHP можно получить текст ошибки из OCI8?

Подсказка: Ошибка обычно возвращается как массив с кодом и сообщением.

Ответ:

- **`oci_error()`**
- `oci_last_error()`
- `oci_get_error()`
- `oci_problem()`

Шаг 08:

Миссия 8

Елена пишет SELECT и хочет получить результат в виде ассоциативного массива. Какой параметр нужно передать в `oci_fetch_array()` ?

Подсказка: Вспомни, в PHP есть режимы: ассоциативный, числовой и оба сразу.

Ответ:

- **OCI_ASSOC**
- OCI_NUM
- OCI_BOTH
- OCI_ROW

Шаг 09:

Миссия 9

Сопоставьте функции OCI8 с их назначением, чтобы помочь Коле разобраться в коде.

Подсказка: Подумайте, какие функции отвечают за соединение, выполнение и выборку.

Ответ:

- **oci_connect()** — Установление соединения с базой Oracle
- **oci_parse()** — Подготовка SQL-запроса для выполнения
- **oci_execute()** — Выполнение подготовленного выражения
- **oci_fetch_array()** — Извлечение строки результата как массива

Шаг 10:

Миссия 10

Дима пытается понять, какие функции нужны для транзакций в OCI8. Сопоставьте функции с их действиями.

Подсказка: Вспомните commit и rollback.

Ответ:

- **oci_commit()** — Сохранение изменений в базе
- **oci_rollback()** — Отмена несохранённых изменений
- **oci_execute()** — Запуск SQL-запроса
- **oci_close()** — Закрытие соединения

Шаг 11:

Миссия 11

Света изучает разные способы выборки данных. Сопоставьте режимы с их описанием.

Подсказка: Ассоциативный массив даёт доступ по именам колонок, числовой — по индексам.

Ответ:

- **OCI_ASSOC** — Массив с ключами по названиям колонок
- **OCI_NUM** — Массив с ключами по числовым индексам
- **OCI_BOTH** — Массив с ключами и по индексам, и по названиям
- **oci_fetch_all()** — Получение всех строк сразу в массив

Шаг 12:

Миссия 12

Алекс хочет научиться обрабатывать ошибки. Сопоставьте функции и то, какую ошибку они могут вернуть.

Подсказка: Ошибки бывают на уровне соединения, выполнения и парсинга.

Ответ:

- **oci_error()** — Возвращает массив с кодом и текстом ошибки
- **oci_connect()** — Может вызвать ошибку при неверных логине или пароле
- **oci_execute()** — Может вернуть ошибку при синтаксической ошибке в SQL
- **oci_parse()** — Вернёт ошибку при некорректном SQL-запросе

🔗 Шаг 13:

Миссия 13

Илья хочет закрепить материал и изучить полный цикл работы с запросом. Сопоставьте шаг с правильным действием.

Подсказка: Подумайте: подключение → парсинг → выполнение → выборка.

Ответ:

- **Подключение** — oci_connect()
- **Подготовка SQL** — oci_parse()
- **Выполнение** — oci_execute()
- **Получение результатов** — oci_fetch_array()

🔗 Шаг 14:

Миссия 14

Поставьте шаги выполнения SELECT в правильном порядке.

Подсказка: Вспомни: подключение → подготовка → выполнение → выборка.

Ответ:

- oci_connect()
- oci_parse()
- oci_execute()
- oci_fetch_array()

🔗 Шаг 15:

Миссия 15

Расставьте шаги работы с транзакцией в правильном порядке.

Подсказка: Сначала изменения, потом commit или rollback, и только потом закрытие соединения.

Ответ:

- Выполнение INSERT/UPDATE
- oci_commit() или oci_rollback()
- oci_close()

🔗 Шаг 16:

Миссия 16

Упорядочите шаги получения всех строк из таблицы.

Подсказка: Здесь вместо fetch_array используем fetch_all.

Ответ:

- `oci_connect()`
- `oci_parse()`
- `oci_execute()`
- `oci_fetch_all()`

 Шаг 17:

Миссия 17

Расположите этапы обработки ошибки при выполнении запроса.

Подсказка: Ошибка возникает после выполнения, и её нужно обработать перед закрытием соединения.

Ответ:

- `oci_execute()`
- `oci_error()`
- Логирование ошибки
- `oci_close()`

 Урок 5 - Подключение и работа с базой данных Oracle

 Шаг 01:

Миссия 1

Разработчик Иван пытается подключиться к базе данных Oracle из PHP. Какой модуль PHP используется для этого чаще всего?

Подсказка: Этот модуль начинается с букв OCI.

Ответ:

- **OCI8**
- PDO_MYSQL
- MSSQL
- SQLite3

 Шаг 02:

Миссия 2

Марина написала скрипт, но при подключении к Oracle ей нужно указать пользователя и пароль. Какой формат вызова функции правильный?

Подсказка: Вспомните функцию `oci_connect()`.

Ответ:

- **`oci_connect('username','password','//localhost/XEPDB1')`**
- `oci_connect('localhost','username','password')`
- `oci_connect('username:password@localhost')`
- `oci_connect()` без параметров

 Шаг 03:

Миссия 3

Сергей хочет проверить, успешно ли подключение к Oracle. Как правильно это сделать в PHP?

Подсказка: Вспомните, что функции могут возвращать `false` при ошибке.

Ответ:

- Проверить `if (!$conn) { echo 'Ошибка'; }`
- Использовать `echo $conn->error`
- Вызвать `mysql_error()`
- Просто проверить наличие файла `tnsnames.ora`

🔗 Шаг 04:

Миссия 4

Алиса пытается выполнить SQL-запрос через PHP и Oracle. Какая функция используется для подготовки SQL?

Подсказка: Она начинается с `oci_parse()`.

Ответ:

- `oci_parse($conn, 'SELECT * FROM users')`
- `oci_prepare($conn, 'SELECT * FROM users')`
- `mysql_query('SELECT * FROM users')`
- `pdo_query($conn, 'SELECT * FROM users')`

🔗 Шаг 05:

Миссия 5

Максим хочет выполнить подготовленный SQL-запрос. Какой функцией это делается?

Подсказка: После `oci_parse` обычно идет эта функция.

Ответ:

- `oci_execute($stid)`
- `oci_run($stid)`
- `pdo_exec($stid)`
- `execute_query($stid)`

🔗 Шаг 06:

Миссия 6

Влад выводит строки из результата запроса. Какой функцией можно получить ассоциативный массив из результата?

Подсказка: В названии функции есть `fetch_assoc`.

Ответ:

- `oci_fetch_assoc($stid)`
- `oci_fetch_row($stid)`
- `oci_fetch_array($stid, OCI_NUM)`
- `oci_get_assoc($stid)`

🔗 Шаг 07:

Миссия 7

Анна заметила утечку ресурсов. Какой функцией закрывается соединение с базой?

Подсказка: Имя функции начинается на `oci_close`.

Ответ:

- `oci_close($conn)`

- `oci_disconnect($conn)`
- `close_connection($conn)`
- `pdo_close($conn)`

🔗 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте функции PHP OCI8 с их назначением.

Подсказка: Подумайте: одни функции подключают, другие выполняют запросы, а третьи извлекают данные.

Ответ:

- **`oci_connect()`** — Устанавливает соединение с базой Oracle
- **`oci_parse()`** — Готовит SQL-запрос для выполнения
- **`oci_execute()`** — Выполняет подготовленный SQL-запрос
- **`oci_fetch_assoc()`** — Возвращает строку результата как ассоциативный массив

🔗 Шаг 09:

Миссия 9

📖 Сопоставьте функции OCI8 с ситуациями их использования.

Подсказка: Представьте, что вы шаг за шагом выполняете запрос и работаете с результатами.

Ответ:

- **`oci_close()`** — Завершение работы и освобождение ресурсов
- **`oci_fetch_row()`** — Получение строки в виде числового массива
- **`oci_fetch_array()`** — Получение строки с возможностью смешанного массива
- **`oci_error()`** — Вывод информации об ошибке соединения или запроса

🔗 Шаг 10:

Миссия 10

📖 Сопоставьте элементы кода с их назначением при работе с Oracle через PHP.

Подсказка: Вспомните, что `$conn` — соединение, `$stid` — подготовленный запрос.

Ответ:

- **`$conn = oci_connect()`** — Создание соединения
- **`$stid = oci_parse()`** — Подготовка SQL-запроса
- **`oci_execute($stid)`** — Запуск выполнения
- **`oci_fetch_assoc($stid)`** — Чтение результата

🔗 Шаг 11:

Миссия 11

📖 Сопоставьте типы выборок с их результатами.

Подсказка: Разные режимы fetch возвращают разные структуры.

Ответ:

- **`OCI_ASSOC`** — Ассоциативный массив с именами колонок
- **`OCI_NUM`** — Массив с числовыми индексами
- **`OCI_BOTH`** — Комбинированный массив
- **`OCI_RETURN_NULLS`** — Включает NULL-значения в выборку

Шаг 12:

Миссия 12

 Сопоставьте ошибки с их возможными причинами.

Подсказка: Подумайте: ошибка может быть в логине, в синтаксисе или в соединении.

Ответ:

- **ORA-01017** — Неправильное имя пользователя или пароль
- **ORA-12541** — Нет слушателя для подключения
- **ORA-00942** — Таблица или представление не существует
- **ORA-00933** — Неправильный SQL-синтаксис

Шаг 13:

Миссия 13

Расставьте шаги подключения к базе Oracle через PHP в правильном порядке.

Подсказка: Сначала подключаемся, затем готовим SQL, выполняем, читаем результат и закрываем соединение.

Ответ:

- `oci_connect()`
- `oci_parse()`
- `oci_execute()`
- `oci_fetch_assoc()`
- `oci_close()`

Шаг 14:

Миссия 14

Упорядочьте этапы обработки ошибок при подключении к Oracle.

Подсказка: Сначала проверка соединения, затем вывод ошибки.

Ответ:

- Попытка подключения через `oci_connect()`
- Проверка: `if (!$conn)`
- Вызов `oci_error()`
- Вывод текста ошибки пользователю

Шаг 15:

Миссия 15

Расставьте действия при подготовке и запуске SQL-запроса в правильном порядке.

Подсказка: Сначала готовим, потом выполняем, затем читаем данные.

Ответ:

- `oci_parse($conn, 'SELECT * FROM users')`
- `oci_execute($stid)`
- `oci_fetch_assoc($stid)`
- `oci_close($conn)`

Шаг 16:

Миссия 16

Расположите шаги извлечения нескольких строк результата.

Подсказка: Обычно используется цикл while.

Ответ:

- Вызов `oci_execute()`
- Запуск while (`oci_fetch_assoc()`)
- Обработка каждой строки
- Закрытие соединения

 **Шаг 17:**

Миссия 17

Расставьте шаги завершения работы приложения с Oracle.

Подсказка: Важно освободить ресурсы в правильном порядке.

Ответ:

- Освободить ресурсы курсора
- Закрыть соединение
- Вывести сообщение об успешном завершении

Модуль 2 - Основы SQL в Oracle

Урок 1 - Синтаксис SQL в Oracle и особенности

 **Шаг 01:**

Миссия 1

Аня только начала работать с Oracle и хочет выбрать правильный оператор для выборки всех строк из таблицы EMPLOYEES. Какой вариант будет корректным в Oracle?

Подсказка: В Oracle синтаксис SQL очень похож на стандартный, но внимательно смотри на ключевые слова.

Ответ:

- **SELECT * FROM EMPLOYEES;**
- GET * FROM EMPLOYEES;
- EXTRACT * EMPLOYEES;
- SHOW EMPLOYEES;

 **Шаг 02:**

Миссия 2

Сергей пишет запрос в Oracle, чтобы выбрать только уникальные должности сотрудников. Какой ключевое слово ему нужно использовать?

Подсказка: В Oracle этот оператор работает так же, как в ANSI SQL.

Ответ:

- **DISTINCT**
- UNIQUE
- DIFFERENT
- ONLY

Шаг 03:

Миссия 3

Катя заметила, что в Oracle можно использовать псевдонимы для столбцов. Как правильно задать псевдоним для столбца FIRST_NAME?

Подсказка: В Oracle можно использовать ключевое слово, но оно не всегда обязательно.

Ответ:

- **SELECT FIRST_NAME AS NAME FROM EMPLOYEES;**
- SELECT FIRST_NAME -> NAME FROM EMPLOYEES;
- SELECT FIRST_NAME = NAME FROM EMPLOYEES;
- SELECT FIRST_NAME ALIAS NAME FROM EMPLOYEES;

Шаг 04:

Миссия 4

Игорь хочет выбрать всех сотрудников, у которых зарплата больше 5000. Какой оператор условия он должен использовать?

Подсказка: Это базовый оператор сравнения в SQL.

Ответ:

- >
- =>
- GRT
- MORE

Шаг 05:

Миссия 5

Лена работает над запросом и хочет отсортировать сотрудников по фамилии. Какой оператор ей нужен?

Подсказка: В Oracle порядок сортировки задается явно.

Ответ:

- **ORDER BY**
- SORT
- GROUP
- ALIGN BY

Шаг 06:

Миссия 6

 Сопоставьте типы данных Oracle с их описанием.

Подсказка: Вспомни, какие типы используют для чисел, текста и дат.

Ответ:

- **NUMBER** — Числовой тип для целых и дробных чисел
- **VARCHAR2** — Переменная длина строк
- **DATE** — Дата и время
- **CLOB** — Хранение больших текстовых данных

Шаг 07:

Миссия 7

 Сопоставьте ключевые слова с их назначением в Oracle SQL.

Подсказка: Каждое ключевое слово выполняет уникальную роль в запросе.

Ответ:

- **WHERE** — Фильтрация строк по условию
- **GROUP BY** — Группировка строк по значениям
- **HAVING** — Условие для групп
- **ORDER BY** — Сортировка результатов

Шаг 08:

Миссия 8

 Сопоставьте функции Oracle с их результатами.

Подсказка: Одни функции работают с числами, другие с текстом или датами.

Ответ:

- **UPPER** — Преобразует строку в верхний регистр
- **SYSDATE** — Возвращает текущую дату и время
- **AVG** — Среднее значение по набору
- **LENGTH** — Длина строки

Шаг 09:

Миссия 9

 Сопоставьте типы соединений в Oracle с их описанием.

Подсказка: Вспомни разницу между INNER и OUTER соединениями.

Ответ:

- **INNER JOIN** — Выбирает строки, у которых есть совпадения в обеих таблицах
- **LEFT JOIN** — Все строки из левой таблицы и совпадения из правой
- **RIGHT JOIN** — Все строки из правой таблицы и совпадения из левой
- **FULL OUTER JOIN** — Все строки из обеих таблиц, даже без совпадений

Шаг 10:

Миссия 10

 Сопоставьте агрегатные функции Oracle с их назначением.

Подсказка: Каждая из этих функций сводит множество значений к одному.

Ответ:

- **COUNT** — Количество строк
- **SUM** — Сумма значений
- **MAX** — Максимальное значение
- **MIN** — Минимальное значение

Шаг 11:

Миссия 11

 Сопоставьте особенности Oracle SQL с их объяснением.

Подсказка: Подумай, чем Oracle отличается от других СУБД.

Ответ:

- **Dual** — Виртуальная таблица для выборок без FROM
- **Sequences** — Механизм генерации уникальных чисел
- **Synonyms** — Альтернативное имя для объекта
- **ROWNUM** — Псевдостолбец для ограничения количества строк

Шаг 12:

Миссия 12

Расположите шаги выполнения SQL-запроса в Oracle в правильном порядке:

Подсказка: Подумайте, что сначала обрабатывается — фильтрация, группировка или сортировка?

Ответ:

- FROM
- WHERE
- GROUP BY
- HAVING
- SELECT
- ORDER BY

Шаг 13:

Миссия 13

Расположите типы SQL-операторов по их сложности от простых к более сложным:

Подсказка: Сначала идут выборки, затем обновления и вставки, а в конце управление транзакциями.

Ответ:

- SELECT
- INSERT
- UPDATE
- DELETE
- COMMIT/ROLLBACK

Шаг 14:

Миссия 14

Расположите операторы ограничения строк по логике их применения:

Подсказка: Подумайте, сначала фильтруются данные, а потом ограничивается количество строк.

Ответ:

- WHERE
- ROWNUM
- FETCH FIRST n ROWS ONLY

Шаг 15:

Миссия 15

Расставьте уровни SQL в Oracle по их назначению:

Подсказка: Сначала идет работа с данными, затем управление транзакциями, а потом управление схемами.

Ответ:

- DQL (запрос данных)
- DML (манипуляция данными)
- TCL (управление транзакциями)
- DDL (описание данных)

Шаг 16:

Миссия 16

Расположите шаги анализа запроса программиста по мере их выполнения:

Подсказка: Обычно сначала пишут запрос, затем проверяют, оптимизируют и тестируют.

Ответ:

- Написание SQL-запроса
- Проверка синтаксиса
- Оптимизация плана выполнения
- Тестирование результата

Шаг 17:

Миссия 17

Расположите уровни изоляции транзакций Oracle от самого слабого к самому строгому:

Подсказка: Начинается с минимального контроля и заканчивается полной изоляцией.

Ответ:

- READ UNCOMMITTED
- READ COMMITTED
- REPEATABLE READ
- SERIALIZABLE

Урок 2 - Индексы, таблицы и схемы в Oracle

Шаг 01:

Миссия 1

Андрей создает первую таблицу в Oracle для хранения данных о сотрудниках. Какой ключевой оператор используется для этого?

Подсказка: Вспомните, что таблица создается явно, а не изменяется.

Ответ:

- **CREATE TABLE**
- ALTER TABLE
- INSERT INTO
- UPDATE TABLE

Шаг 02:

Миссия 2

Марина работает с таблицей PROJECTS. Она хочет удалить её полностью. Какой SQL-запрос она должна использовать?

Подсказка: Удаление всей таблицы отличается от удаления строк.

Ответ:

- **DROP TABLE PROJECTS;**
- DELETE FROM PROJECTS;
- TRUNCATE PROJECTS;
- REMOVE PROJECTS;

 **Шаг 03:**

Миссия 3

Сергей хочет создать уникальный идентификатор для каждой строки таблицы ORDERS. Какой тип ключа ему нужно задать?

Подсказка: Этот ключ гарантирует уникальность каждой записи.

Ответ:

- **PRIMARY KEY**
- FOREIGN KEY
- CHECK CONSTRAINT
- INDEX KEY

 **Шаг 04:**

Миссия 4

Никита замечает, что запросы к таблице CUSTOMERS работают медленно. Что поможет ускорить поиск по столбцу EMAIL?

Подсказка: Это специальная структура, которая строится для оптимизации выборки.

Ответ:

- **Создание индекса**
- Удаление дубликатов
- Добавление FOREIGN KEY
- Использование TRUNCATE

 **Шаг 05:**

Миссия 5

Вика хочет создать индекс для таблицы ORDERS по колонке ORDER_DATE. Какой оператор используется?

Подсказка: Индекс создаётся похожим образом, как и таблица, но с другим ключевым словом.

Ответ:

- **CREATE INDEX**
- ALTER INDEX
- INSERT INDEX
- UPDATE INDEX

 **Шаг 06:**

Миссия 6

Компания решила создать несколько схем для разных отделов. Что в Oracle означает схема ?

Подсказка: Она принадлежит пользователю и хранит его объекты.

Ответ:

- **Набор объектов базы данных, принадлежащих пользователю**
- Файл настроек сервера
- Отдельная база данных
- Список индексов

🔮 **Шаг 07:**

Миссия 7

Сопоставьте объекты Oracle с их описанием.

Подсказка: Подумайте, что хранит таблица, что делает индекс, и зачем нужна схема.

Ответ:

- **Таблица** — Хранит данные в строках и столбцах
- **Индекс** — Ускоряет поиск по таблице
- **Схема** — Объединяет объекты пользователя
- **Представление** — Сохраняет запрос как виртуальную таблицу

🔮 **Шаг 08:**

Миссия 8

Сопоставьте типы индексов Oracle с их характеристиками.

Подсказка: Вспомните, что уникальные индексы запрещают дубли, а обычные просто ускоряют поиск.

Ответ:

- **B-tree индекс** — Используется по умолчанию для большинства запросов
- **Уникальный индекс** — Гарантирует уникальность значений
- **Bitmap индекс** — Эффективен для колонок с небольшим числом уникальных значений
- **Функциональный индекс** — Создается на основе выражений или функций

🔮 **Шаг 09:**

Миссия 9

Сопоставьте команды SQL с их назначением.

Подсказка: DROP удаляет объекты, ALTER изменяет, CREATE создаёт.

Ответ:

- **CREATE TABLE** — Создает новую таблицу
- **ALTER TABLE** — Изменяет структуру таблицы
- **DROP TABLE** — Удаляет таблицу
- **TRUNCATE TABLE** — Очищает таблицу, сохраняя её структуру

🔮 **Шаг 10:**

Миссия 10

Сопоставьте элементы схемы с примерами использования.

Подсказка: Пользователь создаёт объекты в своей схеме.

Ответ:

- **Схема SALES** — Содержит таблицы заказов и клиентов
- **Схема HR** — Содержит данные о сотрудниках

- **Схема DEV** — Используется разработчиками для тестирования
- **Схема FINANCE** — Хранит бухгалтерские таблицы

Шаг 11:

Миссия 11

Сопоставьте ограничения таблиц Oracle с их функцией.

Подсказка: Ограничения помогают поддерживать целостность данных.

Ответ:

- **PRIMARY KEY** — Гарантирует уникальность и NOT NULL
- **FOREIGN KEY** — Связывает таблицы через ключи
- **CHECK** — Ограничивает значения по условию
- **UNIQUE** — Запрещает дубликаты

Шаг 12:

Миссия 12

Сопоставьте действия администратора с командами SQL.

Подсказка: Каждое действие связано с конкретной DDL-командой.

Ответ:

- **Создать новую схему** — CREATE USER
- **Удалить таблицу** — DROP TABLE
- **Изменить колонку** — ALTER TABLE
- **Очистить таблицу** — TRUNCATE TABLE

Шаг 13:

Миссия 13

Расставьте шаги создания новой таблицы в правильном порядке.

Подсказка: Сначала проектируем, затем создаём, потом добавляем данные.

Ответ:

- Определить структуру таблицы
- Выполнить команду CREATE TABLE
- Добавить ограничения (PRIMARY KEY, FOREIGN KEY)
- Вставить данные через INSERT INTO

Шаг 14:

Миссия 14

Расставьте шаги при создании индекса для оптимизации поиска.

Подсказка: Сначала выбирается колонка, потом создается индекс, после чего тестируются запросы.

Ответ:

- Выбрать колонку для индекса
- Выполнить CREATE INDEX
- Проверить производительность запросов
- При необходимости удалить или изменить индекс

Шаг 15:

Миссия 15

Определите правильную последовательность действий при изменении структуры таблицы.

Подсказка: Всегда начинаем с анализа, потом применяем ALTER, и только затем проверяем.

Ответ:

- Проанализировать необходимость изменений
- Выполнить ALTER TABLE
- Добавить или изменить ограничения
- Протестировать изменения

Шаг 16:

Миссия 16

Расставьте шаги при очистке таблицы с сохранением структуры.

Подсказка: Это делается быстрее, чем удаление строк через DELETE.

Ответ:

- Определить таблицу для очистки
- Выполнить TRUNCATE TABLE
- Проверить, что структура осталась
- Вставить новые данные при необходимости

Шаг 17:

Миссия 17

Расположите шаги при создании новой схемы в Oracle.

Подсказка: Пользователь создается до того, как он сможет владеть объектами.

Ответ:

- Создать пользователя
- Назначить права пользователю
- Создать таблицы и индексы в его схеме
- Наполнить таблицы данными

Урок 3 - Основы работы с PLSQL и хранимыми процедурами

Шаг 01:

Миссия 1

Молодой программист Саша пишет первую хранимую процедуру в Oracle. С чего она должна начинаться?

Подсказка: Вспомни ключевое слово, с которого стартует любой блок PL/SQL.

Ответ:

- **Ключевое слово CREATE OR REPLACE PROCEDURE**
- Ключевое слово BEGIN
- Ключевое слово DECLARE
- Ключевое слово FUNCTION

Шаг 02:

Миссия 2

Инженер Оля создала процедуру, но при запуске получила ошибку. Какой оператор завершает тело процедуры?

Подсказка: Процедура должна быть явно закрыта специальным словом, чтобы Oracle понял её границы.

Ответ:

- **END;**
- CLOSE;
- FINISH;
- STOP;

Шаг 03:

Миссия 3

Разработчик Игорь хочет передать в процедуру параметр с именем user_id. Какая форма записи верна?

Подсказка: В PL/SQL всегда указывается тип параметра.

Ответ:

- **user_id IN NUMBER**
- user_id STRING
- IN user_id CHAR
- NUMBER AS user_id

Шаг 04:

Миссия 4

Аркадий отлаживает код и хочет добавить вывод отладочной информации в процедуру. Какой пакет для этого чаще всего используется?

Подсказка: Вспомни стандартный пакет, который выводит строки в буфер.

Ответ:

- **DBMS_OUTPUT**
- DBMS_DEBUG
- UTL_FILE
- PLS_LOG

Шаг 05:

Миссия 5

Лена создаёт процедуру для обновления зарплаты сотрудника. Где должен находиться оператор UPDATE?

Подсказка: Обновление — это часть исполняемого кода.

Ответ:

- **В блоке BEGIN ... END**
- В разделе DECLARE
- В заголовке процедуры
- После ключевого слова CREATE

Шаг 06:

Миссия 6

Сопоставьте элементы структуры процедуры с их назначением.

Подсказка: Подумайте, где описываются переменные, а где выполняется код.

Ответ:

- **DECLARE** — Раздел объявления переменных
- **BEGIN** — Начало исполняемого блока
- **EXCEPTION** — Обработка ошибок
- **END** — Завершение процедуры

Шаг 07:

Миссия 7

Сопоставьте типы параметров процедуры с их смыслом.

Подсказка: Подумайте, когда данные передаются только внутрь, а когда наружу.

Ответ:

- **IN** — Передача значения в процедуру
- **OUT** — Возврат значения из процедуры
- **IN OUT** — Передача значения в обе стороны
- **DEFAULT** — Установка значения по умолчанию

Шаг 08:

Миссия 8

Сопоставьте объекты Oracle с возможностями их использования в процедурах.

Подсказка: Вспомните, что можно менять, а что только читать.

Ответ:

- **Таблица** — Можно читать и изменять данные
- **Представление** — Чаще используется для чтения, чем для записи
- **Секвенция** — Генерация последовательных чисел
- **Синоним** — Упрощённое имя для объекта

Шаг 09:

Миссия 9

Сопоставьте виды ошибок с их примерами.

Подсказка: Обратите внимание: одни ошибки ловятся на этапе компиляции, другие — во время выполнения.

Ответ:

- **Синтаксическая ошибка** — Пропущено ключевое слово END
- **Логическая ошибка** — Неверное условие в операторе IF
- **Рантайм ошибка** — Деление на ноль
- **Исключение NO_DATA_FOUND** — Запрос не вернул строк

Шаг 10:

Миссия 10

Сопоставьте пакеты Oracle с их назначением.

Подсказка: Эти пакеты часто встречаются при написании процедур.

Ответ:

- **DBMS_OUTPUT** — Вывод строк в буфер
- **UTL_FILE** — Работа с файлами
- **DBMS_SCHEDULER** — Планирование заданий
- **DBMS_SQL** — Динамическое выполнение SQL

Шаг 11:

Миссия 11

Сопоставьте элементы процедуры с их функцией при обработке ошибок.

Подсказка: Каждое исключение требует особого подхода.

Ответ:

- **WHEN OTHERS** — Обработка любых ошибок
- **NO_DATA_FOUND** — Запрос не вернул строк
- **TOO_MANY_ROWS** — Запрос вернул больше одной строки
- **ZERO_DIVIDE** — Попытка деления на ноль

Шаг 12:

Миссия 12

Сопоставьте типы PL/SQL блоков с их характеристикой.

Подсказка: Некоторые блоки можно вложить, а некоторые являются отдельными объектами.

Ответ:

- **Анонимный блок** — Не имеет имени и не хранится в БД
- **Процедура** — Выполняет действия, не возвращает значение напрямую
- **Функция** — Возвращает одно значение
- **Пакет** — Содержит группу связанных процедур и функций

Шаг 13:

Миссия 13

Расположите шаги создания хранимой процедуры в правильном порядке.

Подсказка: Подумай: сначала идёт объявление, потом тело, а затем завершение.

Ответ:

- CREATE OR REPLACE PROCEDURE
- Раздел DECLARE (при необходимости)
- BEGIN блок с командами
- END;

Шаг 14:

Миссия 14

Упорядочьте этапы выполнения процедуры с параметрами.

Подсказка: Сначала определяется интерфейс, затем передаются данные, потом идёт исполнение и возврат.

Ответ:

- Определение параметров

- Передача значений
- Выполнение операторов внутри BEGIN ... END
- Возврат OUT параметров

Шаг 15:

Миссия 15

Расположите шаги обработки исключений в процедуре.

Подсказка: Ошибки ловятся в EXCEPTION блоке после выполнения основной логики.

Ответ:

- Выполнение основного кода
- Переход в блок EXCEPTION
- Определение конкретных ошибок
- Выполнение действий по обработке

Шаг 16:

Миссия 16

Разместите шаги отладки процедуры в правильном порядке.

Подсказка: Обычно начинают с воспроизведения ошибки, а завершают тестированием исправлений.

Ответ:

- Запуск процедуры и воспроизведение ошибки
- Анализ сообщения об ошибке
- Внесение исправлений в код
- Повторный запуск и проверка результата

Шаг 17:

Миссия 17

Определи последовательность действий при создании пакета с процедурами.

Подсказка: Сначала описывается интерфейс, потом реализуется тело, а затем идёт тестирование.

Ответ:

- CREATE OR REPLACE PACKAGE specification
- CREATE OR REPLACE PACKAGE BODY
- Тестирование вызова процедур из пакета
- Использование пакета в приложении

Урок 4 - Оптимизация SQL-запросов для Oracle

Шаг 01:

Миссия 1

Программист Иван написал запрос без индексов, и он выполняется очень медленно. Что в первую очередь стоит проверить?

Подсказка: В Oracle ключ к скорости часто скрывается в структуре таблицы.

Ответ:

- Наличие индексов на используемых в WHERE столбцах

- Количество комментариев в SQL-коде
- Форматирование запроса пробелами и табуляцией
- Наличие подзаголовков в SELECT

🔗 Шаг 02:

Миссия 2

Мария оптимизирует отчёт и думает, что быстрее — SELECT * или выбор конкретных столбцов. Что лучше для производительности?

Подсказка: Избыточные данные = лишняя нагрузка.

Ответ:

- **Выбор только необходимых столбцов**
- SELECT * всегда быстрее
- Лучше загружать все поля заранее
- SELECT * экономит память

🔗 Шаг 03:

Миссия 3

Алексей заметил, что запрос с JOIN работает слишком медленно. Какой первый шаг стоит предпринять?

Подсказка: Смотрите в сторону индексов и условий соединения.

Ответ:

- **Проверить индексацию соединяемых ключей**
- Добавить больше таблиц в запрос
- Удалить WHERE для упрощения
- Заменить JOIN на подзапрос всегда

🔗 Шаг 04:

Миссия 4

Разработчик Костя использует подзапросы в SELECT. Иногда они тормозят систему. Что может помочь?

Подсказка: Oracle умеет объединять логику через специальные конструкции.

Ответ:

- **Заменить подзапросы на JOIN там, где возможно**
- Увеличить количество пробелов в SQL
- Дублировать подзапрос для надёжности
- Писать все ключевые слова строчными буквами

🔗 Шаг 05:

Миссия 5

Светлана анализирует план выполнения (EXPLAIN PLAN). Что является ключевым для понимания, где «узкое место»?

Подсказка: В Oracle дерево выполнения выдаёт ответ.

Ответ:

- **Количество операций полного сканирования (FULL TABLE SCAN)**
- Количество символов в SQL-запросе

- Наличие комментариев в коде
- Использование заглавных букв

🔗 Шаг 06:

Миссия 6

Разработчик Дима заметил, что при использовании функции в WHERE запрос стал медленнее. Почему?

Подсказка: Подумайте, как Oracle применяет индексы.

Ответ:

- **Функция делает индекс неиспользуемым**
- Oracle запрещает функции в SQL
- Функция уменьшает читаемость
- Функции всегда быстрее индексов

🔗 Шаг 07:

Миссия 7

Юля анализирует запросы, которые часто обращаются к одним и тем же данным. Что можно использовать для ускорения?

Подсказка: Oracle умеет хранить результаты для повторного доступа.

Ответ:

- **Материализованные представления (Materialized Views)**
- Комментарии внутри SQL
- Дублирование таблиц
- Хранение данных в текстовых файлах

🔗 Шаг 08:

Миссия 8

Сопоставьте элементы SQL-оптимизации с их назначением.

Подсказка: Каждое средство Oracle имеет своё применение: индексы, представления, партиционирование.

Ответ:

- **Индекс** — Ускоряет поиск строк по условию WHERE
- **Материализованное представление** — Хранит заранее рассчитанные данные
- **Партиционирование** — Разделяет таблицу на логические сегменты
- **EXPLAIN PLAN** — Показывает план выполнения запроса

🔗 Шаг 09:

Миссия 9

Разработчик Саша изучает разные типы индексов. Сопоставьте тип индекса с его описанием.

Подсказка: В Oracle индексы бывают разных видов и применяются в разных случаях.

Ответ:

- **B-Tree индекс** — Подходит для выборок по равенству и диапазону
- **Bitmap индекс** — Эффективен для столбцов с малым количеством уникальных значений
- **Function-Based индекс** — Создан на основе выражения или функции
- **Unique индекс** — Гарантирует уникальность значений

🔮 Шаг 10:

Миссия 10

Аналитик Петя оптимизирует запросы с агрегациями. Сопоставьте технику с её преимуществом.

Подсказка: Подумайте, где группировка и индексы помогают экономить ресурсы.

Ответ:

- **GROUP BY с индексами** — Ускоряет агрегацию
- **HAVING вместо WHERE** — Фильтрация после агрегации
- **Предварительная агрегация** — Снижает нагрузку при повторных запросах
- **ROLLUP / CUBE** — Упрощают многомерные отчёты

🔮 Шаг 11:

Миссия 11

Сопоставьте ситуации с рекомендуемым приёмом оптимизации.

Подсказка: В Oracle разные сценарии требуют разных решений.

Ответ:

- **Большая таблица логов** — Использовать партиционирование
- **Частые повторяющиеся запросы** — Создать материализованное представление
- **Фильтрация по дате** — Индекс на столбец даты
- **Медленный сложный JOIN** — Проверить наличие индексов на ключах соединения

🔮 Шаг 12:

Миссия 12

База данных перегружена запросами. Сопоставьте инструмент анализа с его назначением.

Подсказка: Эти средства помогают DBA выявлять «узкие места».

Ответ:

- **AWR Report** — Даёт исторический обзор производительности
- **ASH Report** — Анализ активности сессий в реальном времени
- **SQL Trace** — Подробный разбор выполнения конкретных запросов
- **TKPROF** — Форматирует результаты трассировки

🔮 Шаг 13:

Миссия 13

Расположите шаги анализа медленного запроса в правильном порядке.

Подсказка: Сначала нужно зафиксировать проблему, затем изучить план, после чего оптимизировать.

Ответ:

- Запустить EXPLAIN PLAN
- Определить медленный запрос
- Внести оптимизации (индексы, переписывание)
- Проверить результат повторным запуском

🔮 Шаг 14:

Миссия 14

Упорядочите шаги создания индексируемого представления для ускорения отчётов.

Подсказка: Сначала формируем запрос, потом создаём объект, затем используем.

Ответ:

- Написать SELECT-запрос для отчёта
- Создать материализованное представление
- Добавить индекс на представление
- Использовать в приложении

 **Шаг 15:**

Миссия 15

Расставьте этапы оптимизации сложного JOIN-запроса в правильном порядке.

Подсказка: Нужно двигаться от анализа структуры к проверке результата.

Ответ:

- Проверить ключи соединения
- Построить EXPLAIN PLAN
- Добавить недостающие индексы
- Запустить оптимизированный запрос

 **Шаг 16:**

Миссия 16

Упорядочьте шаги диагностики чрезмерной нагрузки на базу данных.

Подсказка: Сначала анализируем метрики, затем точно изучаем SQL.

Ответ:

- Собрать AWR-отчёт
- Определить проблемные SQL-запросы
- Проанализировать план выполнения
- Внести оптимизации

Урок 5 - Работа с транзакциями и блокировками в Oracle

 **Шаг 01:**

Миссия 1

Андрей тестирует новую систему бронирования билетов. Он открыл транзакцию и сделал UPDATE, но результат пока не сохранился. Какое действие нужно выполнить, чтобы изменения стали видны другим пользователям?

Подсказка: Вспомните, что транзакция должна быть завершена.

Ответ:

- **COMMIT**
- ROLLBACK
- SAVEPOINT
- GRANT

 **Шаг 02:**

Миссия 2

Марина запустила транзакцию на удаление записей, но поняла, что удалила не те строки. Какой оператор позволит ей отменить изменения?

Подсказка: Подумайте, какой оператор полностью откатывает изменения в транзакции.

Ответ:

- **ROLLBACK**
- COMMIT
- SAVEPOINT
- LOCK TABLE

Шаг 03:

Миссия 3

Разработчик заметил, что таблица блокируется, и другие пользователи не могут читать данные. Какой тип блокировки может вызывать такие трудности?

Подсказка: Вспомните, что блокировки бывают разные: одни только читают, другие мешают всем.

Ответ:

- **Эксклюзивная блокировка**
- Шаринг-блокировка (Shared Lock)
- READ ONLY режим
- NOLOCK

Шаг 04:

Миссия 4

Иван создал SAVEPOINT в транзакции. Что он сможет сделать с его помощью?

Подсказка: Это похоже на контрольную точку в игре.

Ответ:

- **Откатиться к определённом состоянию в транзакции**
- Сохранить изменения навсегда
- Завершить сессию
- Заблокировать таблицу

Шаг 05:

Миссия 5

У Юлии две транзакции одновременно изменяют одну строку. В первой транзакции сделан UPDATE, а вторая ждёт. Что произошло?

Подсказка: Вспомните о механизме конкурентного доступа.

Ответ:

- **Вторая транзакция ждёт снятия блокировки первой**
- Обе транзакции выполняются параллельно
- Обе транзакции сразу откатились
- Oracle автоматически сделал COMMIT

Шаг 06:

Миссия 6

Стас выполняет транзакцию, но внезапно отключился от базы. Что сделает Oracle с его незавершённой транзакцией?

Подсказка: Вспомните принцип надёжности ACID.

Ответ:

- **Oracle выполнит ROLLBACK этой транзакции**
- Oracle сделает COMMIT автоматически
- Oracle переведёт изменения в SAVEPOINT
- Oracle ничего не сделает

Шаг 07:

Миссия 7

 Сопоставьте ключевые элементы транзакций с их назначением.

Подсказка: Подумайте: одни команды фиксируют изменения, другие — откатывают.

Ответ:

- **COMMIT** — Фиксирует изменения навсегда
- **ROLLBACK** — Отменяет изменения
- **SAVEPOINT** — Создает точку возврата
- **LOCK TABLE** — Явная блокировка таблицы

Шаг 08:

Миссия 8

Программисты спорят, чем отличаются уровни изоляции транзакций. Соотнесите уровень и его описание.

Подсказка: Чем выше уровень, тем меньше проблем при одновременных запросах.

Ответ:

- **READ UNCOMMITTED** — Может читать неподтверждённые изменения
- **READ COMMITTED** — Читает только подтверждённые данные
- **REPEATABLE READ** — Гарантирует одинаковые результаты при повторных чтениях
- **SERIALIZABLE** — Полностью исключает фантомные строки

Шаг 09:

Миссия 9

Сопоставьте проблему конкуренции с её определением.

Подсказка: Эти термины описывают разные эффекты параллельного доступа.

Ответ:

- **Dirty Read** — Чтение неподтверждённых данных
- **Non-Repeatable Read** — Разные результаты при повторных запросах
- **Phantom Read** — Появление новых строк при повторных запросах
- **Deadlock** — Две транзакции ждут друг друга бесконечно

Шаг 10:

Миссия 10

 В команде обсуждают ACID. Соотнесите букву и её характеристику.

Подсказка: Вспомните классическую расшифровку ACID.

Ответ:

- **Atomicity** — Все операции либо выполняются, либо нет
- **Consistency** — База остаётся в согласованном состоянии

- **Isolation** — Результаты не зависят от других транзакций
- **Durability** — Подтверждённые изменения сохраняются даже после сбоя

🧙 Шаг 11:

Миссия 11

🧙 Разработчики спорят о блокировках. Соотнесите тип блокировки с его определением.

Подсказка: Одна блокировка разрешает совместное чтение, другая — нет.

Ответ:

- **Shared Lock** — Разрешает нескольким транзакциям читать
- **Exclusive Lock** — Разрешает изменять только одной транзакции
- **Row-Level Lock** — Блокирует только одну строку
- **Table-Level Lock** — Блокирует всю таблицу

🧙 Шаг 12:

Миссия 12

🧙💻 Транзакции создают разные эффекты. Сопоставьте сценарий с его названием.

Подсказка: Обратите внимание на повторные запросы и неожиданные изменения.

Ответ:

- **Чтение данных, которые потом откатились** — Dirty Read
- **При повторном запросе значения изменились** — Non-Repeatable Read
- **При повторном запросе появились новые строки** — Phantom Read
- **Две транзакции заблокировали друг друга** — Deadlock

🧙 Шаг 13:

Миссия 13

Расположите шаги работы с транзакцией в правильном порядке:

1. Фиксация изменений
2. Внесение изменений
3. Начало транзакции
4. Возможный откат при ошибке

Подсказка: Вспомните: сначала старт, потом изменения, затем завершение.

Ответ:

- Начало транзакции
- Внесение изменений
- Фиксация изменений
- Возможный откат при ошибке

🧙 Шаг 14:

Миссия 14

Определи порядок действий Oracle при обнаружении deadlock:

1. Выбор транзакции-жертвы
2. Фиксация изменений обеих транзакций
3. Откат транзакции-жертвы
4. Освобождение ресурсов

Подсказка: Oracle не ждёт бесконечно, он разрывает цикл.

Ответ:

- Выбор транзакции-жертвы
- Откат транзакции-жертвы
- Освобождение ресурсов
- Фиксация изменений обеих транзакций

 **Шаг 15:**

Миссия 15

Расставьте шаги при использовании SAVEPOINT:

1. Создание транзакции
2. Создание SAVEPOINT
3. Внесение изменений
4. Откат к SAVEPOINT

Подсказка: SAVEPOINT обычно создаётся до рискованных изменений.

Ответ:

- Создание транзакции
- Создание SAVEPOINT
- Внесение изменений
- Откат к SAVEPOINT

Модуль 3 - Оптимизация работы с базой данных

Урок 1 - Профилирование запросов с Oracle SQL Developer

 **Шаг 01:**

Миссия 1

Аня только начала работать с Oracle SQL Developer и хочет понять, какой инструмент поможет измерять время выполнения SQL-запросов.

Подсказка: Посмотрите на встроенные функции анализа выполнения запросов.

Ответ:

- **SQL Trace и Autotrace**
- PL/SQL Profiler
- Data Pump Export

 **Шаг 02:**

Миссия 2

Иван обнаружил, что один его запрос к большой таблице работает слишком долго. Какой тип индекса поможет ускорить выборку по уникальному ключу?

Подсказка: Подумайте, какой индекс оптимален для поиска по уникальному полю.

Ответ:

- **B-Tree индекс**
- Bitmap индекс
- Clustered индекс

Шаг 03:

Миссия 3

Сергей хочет посмотреть план выполнения SQL-запроса в Oracle SQL Developer. Какой инструмент ему нужен?

Подсказка: План выполнения показывает, какие операции выполняются базой и в каком порядке.

Ответ:

- **Explain Plan**
- Data Pump Import
- SQL Worksheet

Шаг 04:

Миссия 4

 Задача: Сопоставление инструментов Oracle с их функциями.

Формулировка: Сопоставьте инструмент с его основной функцией.

Ответ:

- SQL Trace
- Autotrace
- Explain Plan
- PL/SQL Profiler

1. Показывает статистику выполнения запросов и использование ресурсов.
2. Анализирует время выполнения запросов и количество операций.
3. Демонстрирует пошаговый план выполнения запроса.
4. Профилирует выполнение PL/SQL блоков и процедур.

Подсказка: Подумайте, что измеряет каждый инструмент: ресурсы, порядок операций или отдельные блоки кода.

- **SQL Trace** — Анализирует время выполнения запросов и количество операций.
- **Autotrace** — Показывает статистику выполнения запросов и использование ресурсов.
- **Explain Plan** — Демонстрирует пошаговый план выполнения запроса.
- **PL/SQL Profiler** — Профилирует выполнение PL/SQL блоков и процедур.

Шаг 05:

Миссия 5

 Задача: Сопоставление типов индексов с их применением.

Ответ:

- B-Tree индекс
- Bitmap индекс
- Clustered индекс
- Function-based индекс

1. Идеален для поиска по уникальному ключу.
2. Эффективен для колонок с небольшим числом уникальных значений.
3. Физическое упорядочение данных для ускорения выборки.
4. Позволяет индексировать выражения или функции.

Подсказка: Обратите внимание на уникальность и частоту повторения значений в столбцах.

- **B-Tree индекс** — Идеален для поиска по уникальному ключу.
- **Bitmap индекс** — Эффективен для колонок с небольшим числом уникальных значений.
- **Clustered индекс** — Физическое упорядочение данных для ускорения выборки.
- **Function-based индекс** — Позволяет индексировать выражения или функции.

Шаг 06:

Миссия 6

 Задача: Сопоставление сценариев с методами оптимизации SQL.

Ответ:

- Запрос работает слишком долго из-за большого объема данных
 - Много соединений между таблицами
 - Повторяющиеся вычисления в запросе
 - Фильтрация по функции над столбцом
1. Добавление индекса на столбец
 2. Переписать запрос с использованием JOIN вместо подзапросов
 3. Использование WITH или временных таблиц для повторного вычисления
 4. Создать function-based индекс

Подсказка: Подумайте, какой подход уменьшает нагрузку на базу данных и ускоряет вычисления.

- **Запрос работает слишком долго из-за большого объема данных** — Добавление индекса на столбец
- **Много соединений между таблицами** — Переписать запрос с использованием JOIN вместо подзапросов
- **Повторяющиеся вычисления в запросе** — Использование WITH или временных таблиц для повторного вычисления
- **Фильтрация по функции над столбцом** — Создать function-based индекс

Шаг 07:

Миссия 7

Расположите шаги профилирования SQL-запроса в правильном порядке:

1. Включение SQL Trace
2. Выполнение запроса
3. Сбор статистики Autotrace
4. Анализ Explain Plan

Подсказка: Сначала включаем инструменты, потом выполняем запрос и только потом анализируем результат.

Ответ:

- Включение SQL Trace
- Выполнение запроса
- Сбор статистики Autotrace
- Анализ Explain Plan

Шаг 08:

Миссия 8

Расположите действия DBA для оптимизации медленного запроса:

1. Идентификация проблемных таблиц и столбцов
2. Создание индексов
3. Переписывание запроса для уменьшения JOIN
4. Мониторинг производительности после изменений

Подсказка: Оптимизация начинается с анализа, затем внедрение изменений и проверка результата.

Ответ:

- Идентификация проблемных таблиц и столбцов
- Создание индексов
- Переписывание запроса для уменьшения JOIN
- Мониторинг производительности после изменений

Шаг 09:

Миссия 9

Расположите этапы работы с PL/SQL Profiler:

1. Запуск профилирования PL/SQL кода
2. Выполнение процедуры
3. Сбор статистики времени выполнения каждой строки
4. Анализ результатов и оптимизация кода

Подсказка: Профилирование начинается с включения инструмента, далее выполнение и сбор данных.

Ответ:

- Запуск профилирования PL/SQL кода
- Выполнение процедуры
- Сбор статистики времени выполнения каждой строки
- Анализ результатов и оптимизация кода

Шаг 10:

Миссия 10

Олег хочет уменьшить время выполнения запроса с агрегатными функциями. Какой подход будет наиболее эффективным?

Подсказка: Подумайте, что уменьшает объем данных, которые обрабатываются функцией.

Ответ:

- **Использовать индексируемые колонки для фильтрации**
- Удалить WHERE условия
- Использовать только подзапросы

Шаг 11:

Миссия 11

Марина хочет понять, почему её запрос возвращает много строк медленно. Что ей стоит проверить в первую очередь?

Подсказка: Иногда причина скрыта в фильтрах и условиях соединения таблиц.

Ответ:

- **Использование индексов и условий WHERE**
- Добавление лишних JOIN
- Использование функций в SELECT без фильтрации

Шаг 12:

Миссия 12

 Задача: Сопоставление причин медленного запроса с методами оптимизации.

Ответ:

- Большая таблица без индексов
 - Использование функций в WHERE
 - Сложные подзапросы
 - Высокая конкуренция за ресурсы
1. Создать индексы на нужные столбцы
 2. Использовать function-based индекс или переписать условия
 3. Разбить запрос на несколько более простых
 4. Оптимизировать параллельность и план выполнения

Подсказка: Сначала смотрим на структуру данных, затем на сам запрос и условия.

- **Большая таблица без индексов** — Создать индексы на нужные столбцы
- **Использование функций в WHERE** — Использовать function-based индекс или переписать условия
- **Сложные подзапросы** — Разбить запрос на несколько более простых
- **Высокая конкуренция за ресурсы** — Оптимизировать параллельность и план выполнения

Шаг 13:

Миссия 13

 Сопоставьте типы статистики Autotrace с тем, что они показывают.

Ответ:

- Logical Reads
 - Physical Reads
 - CPU Time
 - Elapsed Time
1. Количество чтений из памяти
 2. Количество чтений с диска
 3. Время процессора, затраченное на выполнение
 4. Общее время выполнения запроса

Подсказка: Подумайте, какая метрика показывает нагрузку на диск, а какая на процессор.

- **Logical Reads** — Количество чтений из памяти
- **Physical Reads** — Количество чтений с диска
- **CPU Time** — Время процессора, затраченное на выполнение
- **Elapsed Time** — Общее время выполнения запроса

Шаг 14:

Миссия 14

Расположите шаги профилирования сложного SQL-запроса для выявления узких мест:

1. Включение SQL Trace
2. Выполнение запроса
3. Сбор статистики Autotrace
4. Анализ Explain Plan
5. Оптимизация запроса или структуры таблиц

Подсказка: Сначала включаем инструменты для профилирования, потом выполняем и анализируем результаты, только после этого оптимизируем.

Ответ:

- Включение SQL Trace
- Выполнение запроса
- Сбор статистики Autotrace
- Анализ Explain Plan
- Оптимизация запроса или структуры таблиц

Шаг 15:

Миссия 15

Расположите этапы работы с PL/SQL Profiler для улучшения производительности процедуры:

1. Запуск профилирования PL/SQL кода
2. Выполнение процедуры с тестовыми данными
3. Сбор статистики времени выполнения каждой строки
4. Анализ «узких мест»
5. Внесение оптимизаций и повторное тестирование

Подсказка: Помните: сначала собираем данные о производительности, затем анализируем и оптимизируем.

Ответ:

- Запуск профилирования PL/SQL кода
- Выполнение процедуры с тестовыми данными
- Сбор статистики времени выполнения каждой строки
- Анализ «узких мест»
- Внесение оптимизаций и повторное тестирование

Шаг 16:

Миссия 16

Расположите шаги комплексной оптимизации запросов в большом проекте:

1. Сбор статистики по медленным запросам
2. Профилирование запросов с Autotrace и Explain Plan
3. Определение индексов и пересмотр структуры таблиц
4. Рефакторинг запросов и PL/SQL блоков
5. Мониторинг производительности после внедрения изменений

Подсказка: Оптимизация в крупных системах — это последовательная работа: сначала анализ, затем изменения, потом проверка результата.

Ответ:

- Сбор статистики по медленным запросам
- Профилирование запросов с Autotrace и Explain Plan
- Определение индексов и пересмотр структуры таблиц
- Рефакторинг запросов и PL/SQL блоков
- Мониторинг производительности после внедрения изменений

Урок 2 - Использование кэширования для ускорения запросов

Шаг 01:

Миссия 1

Анна работает с PHP-скриптом, который часто делает один и тот же SELECT на Oracle. Что она может сделать, чтобы снизить нагрузку на БД?

Подсказка: Кэширование — это хранение ранее полученного результата для повторного использования.

Ответ:

- **Сохранить результат запроса в кэше (memory / Redis)**
- Всегда использовать чистый SELECT без изменений
- Отключить кэширование на стороне PHP

Шаг 02:

Миссия 2

Борис хочет кэшировать результаты запросов, но опасается, что данные в кэше станут устаревшими. Какой подход лучше всего для обновления кэша?

Подсказка: Подумайте о TTL или инвалидации.

Ответ:

- **Установить TTL (время жизни) для кэша**
- Никогда не очищать кэш
- Каждый запрос пересоздавать кэш

Шаг 03:

Миссия 3

Ирина делает PHP-скрипт, который запрашивает список пользователей с атрибутами. Если она кэширует весь список навсегда, что может случиться?

Подсказка: Подумайте о свежести и изменениях в данных.

Ответ:

- **Кэш будет содержать устаревшие данные**
- Кэш автоматически синхронизируется с БД
- БД станет быстрее обслуживать запросы

Шаг 04:

Миссия 4

Сергей интегрировал Redis для кэширования результатов сложного запроса. Когда разумнее всего проверять кэш прежде, чем идти в базу?

Подсказка: Сначала — проверка кэша, потом — запрос, потом — сохранение.

Ответ:

- **Сначала попытаться взять из кэша, если есть** — вернуть, иначе запрос в Oracle и сохранить в кэш
- Всегда делать запрос к Oracle и потом переписывать кэш
- Никогда не проверять кэш, только база

 Шаг 05:

Миссия 5

Юлия хочет кэшировать результат, который зависит от параметра (например, фильтр по дате). Что нужно делать при формировании ключа кэша?

Подсказка: Ключ должен отражать все входные параметры запроса.

Ответ:

- **Включать в ключ кэша параметры запроса (например, дату)**
- Использовать один фиксированный ключ для всех вариаций
- Не использовать ключи, только объекты

 Шаг 06:

Миссия 6

Вадим хочет комбинировать кэширование и прямые запросы: сначала кэш, затем БД, затем снова кэш. Какой паттерн это описывает?

Подсказка: Этот шаблон часто называют read-through (чтение через кэш).

Ответ:

- **Паттерн read-through (чтение через кэш)**
- Паттерн write-behind (запись в кэш позже)
- Паттерн cache-aside (промежуточный кэш)

 Шаг 07:

Миссия 7

 **Задача:** Сопоставьте паттерны кэширования с их описаниями.

Паттерны:

Ответ:

- cache-aside
- read-through
- write-behind
- write-through

Описание:

1. Запись в кэш и БД одновременно при записи.
2. **Когда чтение идёт сначала через кэш, при отсутствии** — запрашивается база и сохраняется в кэш.
3. **Чтение напрямую из кэша, запись сначала в кэш, а потом асинхронно** — в БД.
4. **Приложение самостоятельно проверяет кэш, при промахе** — обращается к БД и сохраняет в кэш.

Подсказка: Обратите внимание: cache-aside — это самостоятельная логика, read-through — посредник, write-behind — асинхронная запись.

- **cache-aside** — Приложение самостоятельно проверяет кэш, при промахе — обращается к БД и сохраняет в кэш.
- **read-through** — Когда чтение идёт сначала через кэш, при отсутствии — запрашивается база и сохраняется в кэш.
- **write-behind** — Чтение напрямую из кэша, запись сначала в кэш, а потом асинхронно — в БД.
- **write-through** — Запись в кэш и БД одновременно при записи.

💡 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте тип кеша с примером использования.

Типы кеша:

Ответ:

- локальный (in-memory внутри PHP)
- распределённый кеш (Redis, Memcached)
- кеш страниц (full page cache)
- кеш фрагментов (fragment cache)

Примеры:

1. Сохранять на уровне кластерного Redis результаты сложного JOIN-запроса.
2. Кешировать целую HTML-страницу для неавторизованных пользователей.
3. Держать в массиве PHP результаты часто запрашиваемого справочника после первого вызова.
4. Кешировать часть шаблона (список новостей) в веб-странице.

Подсказка: Локальный кеш — внутри процесса, распределённый — общий между серверами, фрагмент — часть страницы.

- **локальный (in-memory внутри PHP)** — Держать в массиве PHP результаты часто запрашиваемого справочника после первого вызова.
- **распределённый кеш (Redis, Memcached)** — Сохранять на уровне кластерного Redis результаты сложного JOIN-запроса.
- **кеш страниц (full page cache)** — Кешировать целую HTML-страницу для неавторизованных пользователей.
- **кеш фрагментов (fragment cache)** — Кешировать часть шаблона (список новостей) в веб-странице.

💡 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте термин и преимущество кеширования.

Термины:

Ответ:

- hit rate
- cache miss
- TTL
- инвалидация

Преимущества / определения:

1. Когда данные в кэше теряют актуальность и должны быть очищены.

2. Когда запрошенного элемента нет в кэше и приходится обращаться к БД.
3. Процент запросов, обслуженных из кэша.
4. Максимальное время жизни элемента в кэше.

Подсказка: Акцентируйте внимание на hit / miss и времени жизни элементов.

- **hit rate** — Процент запросов, обслуженных из кэша.
- **cache miss** — Когда запрошенного элемента нет в кэше и приходится обращаться к БД.
- **TTL** — Максимальное время жизни элемента в кэше.
- **инвалидация** — Когда данные в кэше теряют актуальность и должны быть очищены.

Шаг 10:

Миссия 10

 Задача: Сопоставьте подход к инвалидации кэша с примером события.

Подходы:

Ответ:

- инвалидация по времени (TTL)
- инвалидация по событию
- инвалидация по ключу
- инвалидация целого сегмента

События:

1. Удаление записи из справочника, нужно очистить соответствующий ключ.
2. Прошло N секунд, ключ автоматически устарел.
3. Обновление нескольких записей, нужно сбросить целый сегмент кэша.
4. Каждый элемент хранится отдельно, можно сбросить только один ключ.

Подсказка: TTL — по времени; событие — по действию; ключ / сегмент — по объему.

- **инвалидация по времени (TTL)** — Прошло N секунд, ключ автоматически устарел.
- **инвалидация по событию** — Удаление записи из справочника, нужно очистить соответствующий ключ.
- **инвалидация по ключу** — Каждый элемент хранится отдельно, можно сбросить только один ключ.
- **инвалидация целого сегмента** — Обновление нескольких записей, нужно сбросить целый сегмент кэша.

Шаг 11:

Миссия 11

 Задача: Сопоставьте тип данных кэша и его характерные свойства.

Типы данных:

Ответ:

- строка
- хэш / словарь
- список / очередь
- набор (set)

Свойства / примеры:

1. Подходит для хранения одного объекта (JSON)

2. Можно хранить пары ключ→значение (словарь полей)
3. Используется для очередей задач или списков результатов
4. Набор уникальных элементов (без порядка)

Подсказка: Redis поддерживает типы данных: string, hash, list, set.

- **строка** — Подходит для хранения одного объекта (JSON)
- **хэш / словарь** — Можно хранить пары ключ→значение (словарь полей)
- **список / очередь** — Используется для очередей задач или списков результатов
- **набор (set)** — Набор уникальных элементов (без порядка)

🔮 Шаг 12:

Миссия 12

Расположите шаги кэширования на уровне cache-aside:

1. Обновление кэша после записи в БД
2. Проверка кэша при чтении
3. Возврат закэшированного результата, если найден
4. Запрос к БД при отсутствии в кэше

Подсказка: Чтение: проверка → хватаем или идём в БД → записываем кэш при успехе.

Ответ:

- Проверка кэша при чтении
- Возврат закэшированного результат, если найден
- Запрос к БД при отсутствии в кэше
- Обновление кэша после записи в БД

🔮 Шаг 13:

Миссия 13

Расположите шаги read-through паттерна при операции чтения:

1. Если в кэше нет, читать из БД и записывать в кэш
2. Обращение через прокси-кеш (middleware)
3. Возврат результата клиенту из кэша или базы
4. Проверка кэша посредником

Подсказка: Посредник cache-layer сам проверяет кэш и при промахе обращается к базе.

Ответ:

- Обращение через прокси-кеш (middleware)
- Проверка кэша посредником
- Если в кэше нет, читать из БД и записывать в кэш
- Возврат результата клиенту из кэша или базы

🔮 Шаг 14:

Миссия 14

Расположите шаги write-through при записи:

1. Запись в БД
2. Запись одновременно в кэш
3. Ответ клиенту подтверждением успеха

4. При ошибке откат записей

Подсказка: write-through гарантирует синхронность записи в оба хранилища.

Ответ:

- Запись в БД
- Запись одновременно в кэш
- Ответ клиенту подтверждением успеха
- При ошибке откат записей

Шаг 15:

Миссия 15

Расположите этапы гибридного варианта: cache aside + write-behind:

1. Приложение записывает сначала в кэш
2. Асинхронно обновляет БД
3. Во время чтения проверяет кэш прежде, чем идти в БД
4. **При промахе в кэше** — запрос к БД и запись результата в кэш

Подсказка: Комбинация read + асинхронной записи.

Ответ:

- Приложение записывает сначала в кэш
- Асинхронно обновляет БД
- Во время чтения проверяет кэш прежде, чем идти в БД
- **При промахе в кэше** — запрос к БД и запись результата в кэш

Шаг 16:

Миссия 16

Расположите шаги для сброса сегментного кэша:

1. Идентификация сегмента (например, префикса ключей)
2. Удаление всех ключей, начинающихся с префикса
3. Обработка ошибок удаления ключей
4. Логирование события сброса для мониторинга

Подсказка: Сегмент — группа связанных ключей.

Ответ:

- Идентификация сегмента (например, префикса ключей)
- Удаление всех ключей, начинающихся с префикса
- Обработка ошибок удаления ключей
- Логирование события сброса для мониторинга

Шаг 17:

Миссия 17

Расположите этапы реализации гибкой стратегии TTL на уровне запросов:

1. Определение времени жизни в зависимости от типа данных (частые / редкие)
2. При записи устанавливать соответствующий TTL
3. При чтении проверка кэша и возврат данных, если не истёк TTL
4. **При промахе в кэше** — запрос к БД и установка нового TTL

Подсказка: TTL можно настраивать индивидуально под каждую сущность.

Ответ:

- Определение времени жизни в зависимости от типа данных (частые / редкие)
- При записи устанавливать соответствующий TTL
- При чтении проверка кэша и возврат данных, если не истёк TTL
- **При промахе в кэше** — запрос к БД и установка нового TTL

Урок 3 - Индексация таблиц для ускорения выборки данных

Шаг 01:

Миссия 7

 Задача: Сопоставьте тип индекса Oracle с его особенностью.

Типы индекса:

Ответ:

- B-tree
- bitmap
- комбинированный (composite)
- функциональный (function-based)

Особенности / применения:

1. Хорош для выборок по диапазону и точечных запросов.
2. Эффективен при низкой кардинальности, используется в DWH.
3. Покрывает сразу несколько колонок (например, (a, b)).
4. Индекс по выражению или функции, не по исходной колонке.

Подсказка: Подумайте, когда нужен функциональный индекс, а когда — composite.

- **B-tree** — Хорош для выборок по диапазону и точечных запросов.
- **bitmap** — Эффективен при низкой кардинальности, используется в DWH.
- **комбинированный (composite)** — Покрывает сразу несколько колонок (например, (a, b)).
- **функциональный (function-based)** — Индекс по выражению или функции, не по исходной колонке.

Шаг 02:

Миссия 8

 Задача: Сопоставьте ситуацию с лучшим решением индексирования.

Ситуации:

Ответ:

- Фильтрация по LOWER(username)
- Запрос по двум колонкам (city, state)
- Низкий выбор возможных значений (true/false)
- Поиск по диапазону дат

Решения:

1. Функциональный индекс на LOWER(username)
2. Bitmap-индекс
3. Комбинированный индекс (city, state)
4. B-tree индекс на колонку даты

Подсказка: Используйте functional, composite, bitmap, B-tree в подходящих случаях.

- **Фильтрация по LOWER(username)** — Функциональный индекс на LOWER(username)
- **Запрос по двум колонкам (city, state)** — Комбинированный индекс (city, state)
- **Низкий выбор возможных значений (true/false)** — Bitmap-индекс
- **Поиск по диапазону дат** — B-tree индекс на колонку даты

🔮 Шаг 03:

Миссия 9

📖 Задача: Сопоставьте термин с пояснением в контексте индексов.

Термины:

Ответ:

- покрывающий индекс (covering index)
- индексный промах (index miss)
- фрагментированность индекса
- rebuild index

Пояснения:

1. Когда индекс не содержит все нужные столбцы для запроса и требует «визита» к таблице.
2. Индекс содержит все поля SELECT и может обслуживать запрос целиком.
3. **Индекс разбросан, содержит пустые места** — снижает эффективность.
4. Перестроение индекса для дефрагментации.

Подсказка: Разница между покрывающим и промахом — есть ли все поля в индексе.

- **покрывающий индекс (covering index)** — Индекс содержит все поля SELECT и может обслуживать запрос целиком.
- **индексный промах (index miss)** — Когда индекс не содержит все нужные столбцы для запроса и требует «визита» к таблице.
- **фрагментированность индекса** — Индекс разбросан, содержит пустые места — снижает эффективность.
- **rebuild index** — Перестроение индекса для дефрагментации.

🔮 Шаг 04:

Миссия 10

📖 Задача: Сопоставьте влияние индекса с эффектом на запросы.

Влияния:

Ответ:

- ускорение SELECT
- замедление INSERT/UPDATE/DELETE
- увеличение использования дискового пространства
- потенциальная блокировка при перестроении

Описания:

1. SELECT-запросы ищут быстрее, особенно по WHERE.
2. Каждое изменение данных обновляет индекс, добавляя накладные расходы.
3. Индекс хранит структуру и данные, требует места.
4. Во время rebuild или создания индекса возможны блокировки таблицы.

Подсказка: Индексы — компромисс между чтением и записью, пространством и блокировками.

- **ускорение SELECT** — SELECT-запросы ищут быстрее, особенно по WHERE.
- **замедление INSERT/UPDATE/DELETE** — Каждое изменение данных обновляет индекс, добавляя накладные расходы.
- **увеличение использования дискового пространства** — Индекс хранит структуру и данные, требует места.
- **потенциальная блокировка при перестроении** — Во время rebuild или создания индекса возможны блокировки таблицы.

Шаг 05:

Миссия 11

Расположите шаги создания комбинированного индекса на (city, state):

1. Проанализировать, по каким столбцам чаще фильтруют
2. Создать индекс (city, state)
3. Проверить, что запросы используют этот индекс (EXPLAIN PLAN)
4. Мониторить производительность после внедрения

Подсказка: Не спешите создавать — сначала анализ, потом проверка.

Ответ:

- Проанализировать, по каким столбцам чаще фильтруют
- Создать индекс (city, state)
- Проверить, что запросы используют этот индекс (EXPLAIN PLAN)
- Мониторить производительность после внедрения

Шаг 06:

Миссия 13

Расположите шаги выбора между одиночным и комбинированным индексом:

1. Смотреть статистику запросов, как часто используются оба поля вместе
2. Проверить, есть ли случаи использования только одного поля
3. Создать комбинированный индекс, если совместное использование часто
4. Создать одиночные индексы, если поля используются независимо чаще

Подсказка: Анализ статистики запросов — ключевой шаг.

Ответ:

- Смотреть статистику запросов, как часто используются оба поля вместе
- Проверить, есть ли случаи использования только одного поля
- Создать комбинированный индекс, если совместное использование часто
- Создать одиночные индексы, если поля используются независимо чаще

Шаг 07:

Миссия 14

Расположите шаги при обновлении большого количества строк, чтобы не перегрузить индексы:

1. Отключить индекс (если возможно)
2. Выполнить массовое обновление
3. Пересоздать индекс / rebuild после обновлений
4. Собрать статистику снова

Подсказка: Иногда проще временно убрать нагрузку, затем восстановить индексы.

Ответ:

- Отключить индекс (если возможно)
- Выполнить массовое обновление
- Пересоздать индекс / rebuild после обновлений
- Собрать статистику снова

 Шаг 08:

Миссия 15

Расположите шаги внедрения функционального индекса для LOWER(email):

1. Определить выражение (LOWER(email)) как часть запроса
2. Создать индекс на выражение LOWER(email)
3. Обновить запросы, чтобы они использовали это выражение в WHERE
4. Проверить через EXPLAIN, что индекс применяется

Подсказка: Функциональные индексы работают только если запрос совпадает с выражением.

Ответ:

- Определить выражение (LOWER(email)) как часть запроса
- Создать индекс на выражение LOWER(email)
- Обновить запросы, чтобы они использовали это выражение в WHERE
- Проверить через EXPLAIN, что индекс применяется

 Шаг 09:

Миссия 16

Расположите шаги для мониторинга использования индексов:

1. Включить статистику исполнения запросов
2. Собирать метрики (использование индекса, время запросов)
3. Анализировать, какие индексы не используются
4. Удалять или перестраивать редко используемые индексы

Подсказка: Мониторинг — непрерывный шаг в жизни крупной системы.

Ответ:

- Включить статистику исполнения запросов
- Собирать метрики (использование индекса, время запросов)
- Анализировать, какие индексы не используются
- Удалять или перестраивать редко используемые индексы

 Шаг 10:

Миссия 17

Расположите стратегию внедрения индексов на большой существующей базе:

1. Выбрать диагональные запросы (горячие) для анализа
2. Создать индексы на тестовой среде и измерить влияние
3. Поэтапно внедрить индексы в продакшн ночью или в окно обслуживания
4. Мониторить производительность и откатить, если ухудшение

Подсказка: Внедрение индексов на живую систему требует осторожности.

Ответ:

- Выбрать диагональные запросы (горячие) для анализа
- Создать индексы на тестовой среде и измерить влияние
- Поэтапно внедрить индексы в продакшн ночью или в окно обслуживания
- Мониторить производительность и откатить, если ухудшение

 Шаг 11:

Миссия 1

Вы разработчик PHP, и вам нужно ускорить выборку по таблице users, где много строк. Что вы сделаете в первую очередь?

Подсказка: Подумайте, что чаще всего тормозит поиск по столбцу без вспомогательных структур.

Ответ:

- **Создам индекс на поле, по которому ищут**
- Добавлю больше PHP-логики для фильтрации
- Увеличу память Oracle
- Перенесу часть данных в файл

 Шаг 12:

Миссия 2

Ваш коллега предлагает создать индекс на каждый столбец, «чтобы всё работало быстрее». Вы сомневаетесь. Какой аргумент вы приведёте против такого подхода?

Подсказка: Индексы тоже требуют ресурсов для обновления.

Ответ:

- **Каждый индекс замедляет вставку и обновление**
- Индексы делают SELECT медленнее
- Индексы требуют PHP-кода
- Индексы занимают меньше места, чем данные

 Шаг 13:

Миссия 4

Вы замечаете в запросах конструкции WHERE LOWER(username) = 'ivan'. Как можно ускорить такие запросы?

Подсказка: Подумайте про функциональные / вычисляемые индексы.

Ответ:

- **Создать индекс на функцию LOWER(username)**
- Переводить username в нижний регистр в PHP
- Не делать индекс, поскольку это вычисление
- Избавиться от LOWER и хранить username в нижнем регистре

 Шаг 14:

Миссия 6

В таблице logs хранится миллиард записей. Часто выполняют запросы по уровню log_level = 'ERROR'. Что вы сделаете?

Подсказка: Подумайте, как ускорить фильтрацию по часто встречающемуся значению.

Ответ:

- **Создать индекс на log_level**
- Кешировать результаты в PHP
- Удалить все записи с другим уровнем
- Создать индекс только на текст сообщения

Шаг 15:

Миссия 7

Сопоставьте тип индекса Oracle с его описанием.

Типы индексов:

Ответ:

- B-tree индекс
- Bitmap индекс
- Функциональный индекс
- Составной индекс

Описание:

1. Индекс по выражению или функции
2. Индекс, включающий два или более столбцов
3. Индекс для низкой кардинальности, хранит биты
4. Стандартный сбалансированный деревом поиск

Подсказка: Подумайте, когда используется bitmap и когда — дерево.

- **B-tree индекс** — Стандартный сбалансированный деревом поиск
- **Bitmap индекс** — Индекс для низкой кардинальности, хранит биты
- **Функциональный индекс** — Индекс по выражению или функции
- **Составной индекс** — Индекс, включающий два или более столбцов

Шаг 16:

Миссия 8

Сопоставьте проблему с тем, на какой тип индекса она влияет лучше всего.

Проблемы:

Ответ:

- Медленный поиск по функции LOWER(col)
- Запрос по дате range
- Фильтрация по двум критериям вместе (col1, col2)
- Многие одинаковые значения одного столбца

Тип индекса:

1. Функциональный индекс
2. Индекс по дате / простого типа
3. Составной индекс
4. Bitmap индекс

Подсказка: Подумайте, что делать, если множество строк имеют одно и то же значение.

- **Медленный поиск по функции LOWER(col)** — Функциональный индекс

- **Запрос по дате range** — Индекс по дате / простого типа
- **Фильтрация по двум критериям вместе (col1, col2)** — Составной индекс
- **Многие одинаковые значения одного столбца** — Bitmap индекс

🔗 Шаг 17:

Миссия 9

Сопоставьте SQL-конструкцию с её влиянием на использование индекса.

Конструкция:

Ответ:

- WHERE col = :v
- WHERE col LIKE 'abc%'
- WHERE col LIKE '%abc'
- WHERE col + 1 = :v

Влияние:

1. Индекс может быть использован полностью
2. Индекс используется частично (префикс)
3. Индекс не используется
4. Индекс не используется, из-за вычисления

Подсказка: Подумайте, что нарушает прямое попадание в индекс.

- **WHERE col = :v** — Индекс может быть использован полностью
- **WHERE col LIKE 'abc%'** — Индекс используется частично (префикс)
- **WHERE col LIKE '%abc'** — Индекс не используется
- **WHERE col + 1 = :v** — Индекс не используется, из-за вычисления

🔗 Шаг 18:

Миссия 10

Сопоставьте задачу DBA с приемом по работе с индексами.

Задача:

Ответ:

- Сократить время вставки
- Снижение дискового пространства
- Улучшить выборку по трём колонкам вместе
- Ускорить фильтрацию по выражению

Прием:

1. Удаление избыточных индексов
2. Индекс-сжатие или коалесценция
3. Составной индекс
4. Функциональный индекс

Подсказка: При отрицательных эффектах индексы могут мешать.

- **Сократить время вставки** — Удаление избыточных индексов
- **Снижение дискового пространства** — Индекс-сжатие или коалесценция
- **Улучшить выборку по трём колонкам вместе** — Составной индекс
- **Ускорить фильтрацию по выражению** — Функциональный индекс

Шаг 19:

Миссия 11

Расположите этапы оптимизации индексации:

1. Проверка использования индексов в планах выполнения
2. Добавление индекса и анализ нагрузки
3. Удаление или изменение неэффективных индексов
4. Сбор статистики и обновление статистики после изменений

Подсказка: Сначала нужно увидеть, что реально используется, а не гадать.

Ответ:

- Проверка использования индексов в планах выполнения
- Добавление индекса и анализ нагрузки
- Удаление или изменение неэффективных индексов
- Сбор статистики и обновление статистики после изменений

Урок 4 - Использование материалов и представлений в Oracle

Шаг 01:

Миссия 1

Инженер Мария создает материал в Oracle для ускорения выборки часто запрашиваемых данных. Какой тип материала чаще всего используют?

Подсказка: Он хранит результаты как таблицу и периодически обновляется.

Ответ:

- **Материализованное представление**
- Виртуальное представление
- Синоним
- Последовательность

Шаг 02:

Миссия 2

Юлия рассматривает вариант использования обычного представления вместо материала. Что из нижеперечисленного является явным недостатком обычного представления в условиях высокой нагрузки?

Подсказка: Обычное представление не хранит данные — оно вычисляет на лету.

Ответ:

- **Высокая нагрузка на вычисления при каждом запросе**
- Невозможность использования индексов
- Нельзя объявить зависимости
- Нельзя фильтровать данные

Шаг 03:

Миссия 3

Команда обсуждает, можно ли индексировать материализованное представление. Какой ответ наиболее корректен?

Подсказка: Материал действует как таблица с физическим хранением.

Ответ:

- **Да, можно создавать индексы на материализованном представлении**
- Нет, нельзя индексировать
- Индексы создаются автоматически
- Индексы работают только для обычного представления

Шаг 04:

Миссия 4

При выполнении сложного запроса с объединением нескольких таблиц Иван замечает, что материал работает быстрее, но не знает, как выбрать механику обновления. Что влияет на выбор механизма обновления?

Подсказка: Ответ зависит от частоты обновлений и требований к свежести данных.

Ответ:

- **Как часто данные изменяются в базовой таблице**
- **Допустимая задержка (staleness)**
- Имя материала
- **Объем данных и стоимость перерасчета**

Шаг 05:

Миссия 5

Лена хочет запретить изменения в базовых таблицах, чтобы материал всегда был валидным. Что она может использовать?

Подсказка: Есть ограничения целостности, триггеры и правила блокировки.

Ответ:

- **Применить триггеры BEFORE INSERT/UPDATE с откатом**
- Запретить доступ через синоним
- Удалить базовые таблицы
- Использовать стандартный представление

Шаг 06:

Миссия 6

 Сопоставьте термины с их описаниями (материалы, обычные представления и т.п.).

Понятия:

Ответ:

- Материализованное представление
- Обычное представление (VIEW)
- ON COMMIT
- REFRESH FAST

Описания:

1. Обновление происходит при фиксации транзакции
2. Представление без физического хранения данных
3. Обновляется только измененные строки
4. Физический результат запроса хранится в диске

Подсказка: Вспомните, что VIEW не содержит данных, а материал — содержит.

- **Материализованное представление** — Физический результат запроса хранится в диске
- **Обычное представление (VIEW)** — Представление без физического хранения данных
- **ON COMMIT** — Обновление происходит при фиксации транзакции
- **REFRESH FAST** — Обновляется только измененные строки

🔗 Шаг 07:

Миссия 7

📖 Сопоставьте режимы обновления материала и их особенности.

Режимы:

Ответ:

- COMPLETE
- FAST
- FORCE
- ON DEMAND

Характеристики:

1. Полный пересчет независимо от изменений
2. Пытается использовать инкрементальные изменения
3. Автоматически выбирает между COMPLETE и FAST
4. Обновление запускается вручную

Подсказка: FORCE — гибридный режим, способен переключаться.

- **COMPLETE** — Полный пересчет независимо от изменений
- **FAST** — Пытается использовать инкрементальные изменения
- **FORCE** — Автоматически выбирает между COMPLETE и FAST
- **ON DEMAND** — Обновление запускается вручную

🔗 Шаг 08:

Миссия 8

📖 Сопоставьте преимущества использования материалов и представлений.

Варианты:

Ответ:

- **Материализованное представление** — преимущество
- **Обычное представление** — преимущество
- **Материализованное представление** — недостаток
- **Обычное представление** — недостаток

Описания:

1. Более свежие данные, без задержки
2. Минимальное хранилище и динамическая адаптация
3. Необходимость периодического обновления
4. Высокая нагрузка на вычисления при каждом запросе

Подсказка: Сравните, что важнее — быстродействие или свежесть данных.

- **Материализованное представление** — преимущество — Более свежие данные, без задержки

- **Обычное представление** — преимущество — Минимальное хранилище и динамическая адаптация
- **Материализованное представление** — недостаток — Необходимость периодического обновления
- **Обычное представление** — недостаток — Высокая нагрузка на вычисления при каждом запросе

💡 Шаг 09:

Миссия 9

📖 Сопоставьте термины с SQL-конструкциями:

Термины:

Ответ:

- REFRESH COMPLETE
- REFRESH FAST WITH PRIMARY KEYS
- BUILD IMMEDIATE
- USING INDEX

Описание:

1. Указывает пересоздавать полностью
2. Использует только измененные строки с ключами
3. Создает материал сразу при определении
4. Применяет индекс к созданному материалу

Подсказка: Обратите внимание на слова COMPLETE, FAST, BUILD, INDEX.

- **REFRESH COMPLETE** — Указывает пересоздавать полностью
- **REFRESH FAST WITH PRIMARY KEYS** — Использует только измененные строки с ключами
- **BUILD IMMEDIATE** — Создает материал сразу при определении
- **USING INDEX** — Применяет индекс к созданному материалу

💡 Шаг 10:

Миссия 10

📖 Сопоставьте команды SQL с их эффектом в контексте материалов и представлений.

Команды:

Ответ:

- CREATE MATERIALIZED VIEW
- CREATE VIEW
- ALTER MATERIALIZED VIEW ... REFRESH
- DROP VIEW

Эффекты:

1. Удаляет объект представления
2. Определяет представление как виртуальный (без хранения данных)
3. Запускает обновление для материала
4. Создает материал как таблицу на основе запроса

Подсказка: Обратите внимание на глаголы CREATE, DROP и ключевые слова VIEW/MATERIALIZED.

- **CREATE MATERIALIZED VIEW** — Создает материал как таблицу на основе запроса

- **CREATE VIEW** — Определяет представление как виртуальный (без хранения данных)
- **ALTER MATERIALIZED VIEW ... REFRESH** — Запускает обновление для материала
- **DROP VIEW** — Удаляет объект представления

Шаг 11:

Миссия 11

Расположите шаги создания и использования материализованного представления в правильном порядке:

1. Запрос SELECT с использованием представления
2. CREATE MATERIALIZED VIEW ... WITH PRIMARY KEY
3. Обновление REFRESH
4. Определение индекса на материал

Подсказка: Сначала создаём, затем индексируем, затем обновляем и используем.

Ответ:

- CREATE MATERIALIZED VIEW ... WITH PRIMARY KEY
- Определение индекса на материал
- Обновление REFRESH
- Запрос SELECT с использованием представления

Шаг 12:

Миссия 12

Расположите этапы отладки медленного запроса, при использовании материалов:

1. Анализ плана выполнения (EXPLAIN PLAN)
2. Создание материала или представления
3. Сбор статистики на базовые таблицы
4. Добавление индекса к материалу
5. Обновить материал

Подсказка: Важен цикл: статистика → план → индекс → обновление.

Ответ:

- Сбор статистики на базовые таблицы
- Анализ плана выполнения (EXPLAIN PLAN)
- Создание материала или представления
- Добавление индекса к материалу
- Обновить материал

Шаг 13:

Миссия 13

Расположите действия для обеспечения консистентности материала:

1. Отключение DML для базовых таблиц
2. Запуск REFRESH
3. Разрешение DML изменений после обновления
4. Проверка целостности данных

Подсказка: Нарушения согласованности чаще всего возникают при одновременном изменении данных.

Ответ:

- Отключение DML для базовых таблиц
- Запуск REFRESH
- Проверка целостности данных
- Разрешение DML изменений после обновления

 Шаг 14:

Миссия 14

Расположите шаги внедрения материала в web-приложение с PHP и Oracle:

1. Использовать PDO/OCI для обращения к материалу
2. Обработать результат SELECT
3. Создать материал в Oracle
4. Настроить расписание обновления

Подсказка: Разработку ведут от базы к приложению.

Ответ:

- Создать материал в Oracle
- Настроить расписание обновления
- Использовать PDO/OCI для обращения к материалу
- Обработать результат SELECT

 Шаг 15:

Миссия 15

Расположите шаги для обкатки миграции с материалами на тестовой среде:

1. Создать материалы на тестовой БД
2. Запустить нагрузочное тестирование
3. Проверить производительность и корректность
4. Прокатить изменения на продакшн

Подсказка: Не переходите на продакшн раньше уверенности в тестах.

Ответ:

- Создать материалы на тестовой БД
- Запустить нагрузочное тестирование
- Проверить производительность и корректность
- Прокатить изменения на продакшн

 Шаг 16:

Миссия 16

Расположите действия по плану отката при проблемах с материалом:

1. Откатить транзакции, затронувшие материал
2. Удалить проблемное материализованное представление
3. Вернуть старую версию (резервную копию)
4. Проверить корректность после отката

Подсказка: При откате важно сохранять целостность данных.

Ответ:

- Откатить транзакции, затронувшие материал
- Удалить проблемное материализованное представление
- Вернуть старую версию (резервную копию)
- Проверить корректность после отката

Урок 5 - Распараллеливание запросов и улучшение производительности

Шаг 01:

Миссия 1

Вы работаете над системой аналитики: одна тяжелая агрегация в Oracle занимает 10 секунд. Вы хотите распараллелить задачи на уровне RHP. Какое решение подходит в первую очередь?

Подсказка: Подумайте, что можно делать параллельно вне одной монолитной функции.

Ответ:

- **Запустить несколько параллельных RHP-процессов, каждый с частью данных**
- Изменить весь запрос на PL/SQL функцию
- Уменьшить выборку, отсекая часть данных
- Добавить больше индексов на все столбцы

Шаг 02:

Миссия 2

Вы разделили задачу на три параллельных запроса из RHP, но неожиданно нагрузка выросла. Что из нижеперечисленного — основной риск?

Подсказка: Подумайте об одновременном доступе к нагрузке и блокировках.

Ответ:

- **Блокировки и межтранзакционные конкуренции в Oracle**
- RHP-скрипты упадут из-за переполнения памяти
- Запросы выполняются слишком быстро
- RHP не поддерживает многопоточность

Шаг 03:

Миссия 3

Вы заметили, что соединения с Oracle часто создаются и закрываются. Что лучше использовать для улучшения производительности?

Подсказка: Подумайте про переиспользование соединений.

Ответ:

- **Пул соединений (connection pool) в RHP**
- Закрывать соединение на каждый запрос
- Использовать отдельный клиентский драйвер
- Записывать результаты в файлы, а не в базу

Шаг 04:

Миссия 4

После подключения пула вы видите, что очереди запросов к Oracle всё равно растут. Что может помочь снизить задержки?

Подсказка: Подумайте об очередях задач и отложенных вызовах.

Ответ:

- **Использовать очередь задач (job queue), асинхронный режим**
- Увеличить таймауты PHP
- Отправлять запросы пакетами по 1 строке
- Отключить логирование

Шаг 05:

Миссия 5

Вы хотите внедрить шардирование таблиц по дате, и каждый шард обрабатывается параллельно. Что ключевое условие для корректной агрегации?

Подсказка: Подумайте, как объединять частичные результаты.

Ответ:

- **Суммировать, а не делать агрегацию по всему набору сразу**
- Использовать идентичные индексы на каждом шарде
- Использовать глобальный курсор Oracle
- Сжать данные перед отправкой

Шаг 06:

Миссия 6

Вы решили распараллелить выполнение сложного отчёта: один PHP запускает несколько потоков через ``proc_open``. Что важно учесть?

Подсказка: Подумайте про ошибки и сбор результатов.

Ответ:

- **Обработку ошибок каждого процесса и сбор всех ответов**
- Запуск как можно большего числа процессов сразу
- Использовать ``exec()`` вместо ``proc_open``
- Игнорировать таймауты

Шаг 07:

Миссия 7

 Задача: Сопоставьте термин параллелизации с его описанием.

Термины:

Ответ:

- MapReduce
- Шардирование
- Batch-обработка
- Поточковый API (streaming)

Определения:

1. Деление данных по ключам и распределение вычислений по узлам.
2. Обработка данных порциями, с накоплением и отправкой пакетами.
3. Парадигма: разбивать задачу на map и reduce шаги.
4. Непрерывная обработка событий по мере поступления.

Подсказка: Вспомните Hadoop / Большие данные, но применимо к запросам в Oracle.

- **MapReduce** — Парадигма: разбивать задачу на map и reduce шаги.
- **Шардирование** — Деление данных по ключам и распределение вычислений по узлам.
- **Batch-обработка** — Обработка данных порциями, с накоплением и отправкой пакетами.
- **Потоковый API (streaming)** — Непрерывная обработка событий по мере поступления.

🔮 Шаг 08:

Миссия 8

📖 Задача: Сопоставьте тип параллелизма в PHP-Oracle интеграции и его ограничение.

Типы параллелизма:

Ответ:

- Параллельные PHP-процессы
- Пакетные PL/SQL-процедуры
- DBMS_PARALLEL_EXECUTE
- Materialized view rewrite

Ограничения:

1. Не контролируется из PHP, накладные на пересоздание MV.
2. Не гарантирует распределение нагрузки по узлам.
3. Нужно чтобы задачи разделялись по ключу, иначе дублирование.
4. Нужны внешние процессы и сбор результата.

Подсказка: Подумайте, как управляется параллелизм из разных слоёв.

- **Параллельные PHP-процессы** — Нужны внешние процессы и сбор результата.
- **Пакетные PL/SQL-процедуры** — Не гарантирует распределение нагрузки по узлам.
- **DBMS_PARALLEL_EXECUTE** — Нужно чтобы задачи разделялись по ключу, иначе дублирование.
- **Materialized view rewrite** — Не контролируется из PHP, накладные на пересоздание MV.

🔮 Шаг 09:

Миссия 9

📖 Задача: Сопоставьте тип данных / конструкции Oracle с их ролью в параллельных запросах.

Конструкции:

Ответ:

- PARALLEL hint
- Parallel DML
- RESULT CACHE
- Collection / PIPELINED функции

Роли / эффекты:

1. Кеширует результат запроса для повторного использования.
2. Позволяет выполнить DML параллельно.
3. Подсказка, чтобы Oracle распараллелил оператор.
4. Позволяет потоковую генерацию строк при параллелизме.

Подсказка: Подумайте, что ваш SQL может предложить Oracle, а что — ваша логика.

- **PARALLEL hint** — Подсказка, чтобы Oracle распараллелил оператор.

- **Parallel DML** — Позволяет выполнить DML параллельно.
- **RESULT CACHE** — Кеширует результат запроса для повторного использования.
- **Collection / PIPELINED функции** — Позволяет потоковую генерацию строк при параллелизме.

🔮 Шаг 10:

Миссия 10

📖 Задача: Сопоставьте примеры из жизни команды с типом параллелизма.

Сценарии:

Ответ:

- Каждый PHP-воркер обрабатывает диапазон ID
- В Oracle вызывается DBMS_PARALLEL_EXECUTE над запросом
- Данные агрегации предвычисляются в MV
- PHP ставит задачи в очередь и воркеры берут части

Типы:

1. Пул воркеров с очередью задач
2. Параллельный движок Oracle
3. Материализованный подход (пред-агрегации)
4. Гранулированное деление диапазонов

Подсказка: Вспомните предыдущие миссии.

- **Каждый PHP-воркер обрабатывает диапазон ID** — Гранулированное деление диапазонов
- **В Oracle вызывается DBMS_PARALLEL_EXECUTE над запросом** — Параллельный движок Oracle
- **Данные агрегации предвычисляются в MV** — Материализованный подход (пред-агрегации)
- **PHP ставит задачи в очередь и воркеры берут части** — Пул воркеров с очередью задач

🔮 Шаг 11:

Миссия 11

📖 Задача: Сопоставьте этапы жизни параллельного запроса с тем, что на нём может пойти не так.

Этапы:

Ответ:

- Разбиение на части (splitting)
- Выполнение частей
- Агрегация результатов
- Обработка ошибок / повтор

Риски:

1. Один фрагмент падает, остальные успешны.
2. Неправильное объединение даёт дубли.
3. Задачи тяжёлые и злоупотребляют БД.
4. Сложности с выбором границ частей.

Подсказка: Подумайте о контроле и устойчивости на каждом шаге.

- **Разбиение на части (splitting)** — Сложности с выбором границ частей.
- **Выполнение частей** — Задачи тяжёлые и злоупотребляют БД.
- **Агрегация результатов** — Неправильное объединение даёт дубли.
- **Обработка ошибок / повтор** — Один фрагмент падает, остальные успешны.

🔮 Шаг 12:

Миссия 12

Расположите шаги при внедрении параллельного исполнения отчёта в порядке:

1. Запуск параллельных процессов или задач
2. Сбор частичных результатов и объединение
3. Мониторинг и логирование выполнения
4. Разбиение задачи на сегменты
5. Обработка ошибок и перезапуск части

Подсказка: Представьте, как выстроить конвейер от сегментации до итогового результата.

Ответ:

- Разбиение задачи на сегменты
- Запуск параллельных процессов или задач
- Обработка ошибок и перезапуск части
- Сбор частичных результатов и объединение
- Мониторинг и логирование выполнения

🔮 Шаг 13:

Миссия 13

Расположите по порядку: как PHP-скрипт управляет параллельной задачей:

1. Запрос статуса задач
2. Формирование сегментов запросов
3. Запуск подпроцессов / задач
4. Сбор и объединение ответа
5. Обработка ошибок и повтор

Подсказка: Подумайте, как выстроить цикл запуска и контроля.

Ответ:

- Формирование сегментов запросов
- Запуск подпроцессов / задач
- Запрос статуса задач
- Обработка ошибок и повтор
- Сбор и объединение ответа

🔮 Шаг 14:

Миссия 14

Расположите шаги для безопасного выполнения параллельного DML-оператора:

1. Отключение ограничений / триггеров
2. Разделение данных на диапазоны
3. Запуск параллельных DML

4. Проверка целостности данных
5. Включение ограничений / триггеров

Подсказка: Подумайте, как минимизировать конфликт между ветвями.

Ответ:

- Разделение данных на диапазоны
- Отключение ограничений / триггеров
- Запуск параллельных DML
- Проверка целостности данных
- Включение ограничений / триггеров

Шаг 15:

Миссия 15

Расположите этапы применения DBMS_PARALLEL_EXECUTE с контролем ошибок:

1. Создание задания через DBMS_PARALLEL_EXECUTE.CREATE_TASK
2. Добавление сегментов через DBMS_PARALLEL_EXECUTE.RUN_TASK
3. Мониторинг выполнения сегментов
4. Обработка сегментов с ошибками (RETRY / SKIP)
5. Удаление задачи после завершения

Подсказка: Вспомните документацию Oracle по DBMS_PARALLEL_EXECUTE.

Ответ:

- Создание задания через DBMS_PARALLEL_EXECUTE.CREATE_TASK
- Добавление сегментов через DBMS_PARALLEL_EXECUTE.RUN_TASK
- Мониторинг выполнения сегментов
- Обработка сегментов с ошибками (RETRY / SKIP)
- Удаление задачи после завершения

Шаг 16:

Миссия 16

Расположите стратегические этапы внедрения параллелизма в крупной системе (от подготовки до промышленной эксплуатации):

1. Пилотный запуск на малом наборе данных
2. Нагрузочное тестирование и симуляция отказов
3. Оптимизация планов запросов и индексов</ >
4. Полномасштабный перевод в продакшн
5. Мониторинг, логирование и откат на случай ошибок

Подсказка: Подумайте, как постепенно снижать риск внедрения.

Ответ:

- Оптимизация планов запросов и индексов
- Пилотный запуск на малом наборе данных
- Нагрузочное тестирование и симуляция отказов
- Полномасштабный перевод в продакшн
- Мониторинг, логирование и откат на случай ошибок

Модуль 4 - Работа с большими объемами данных

Урок 1 - Оптимизация работы с большими таблицами

Шаг 01:

Миссия 1

Анна — backend-разработчик, она видит, что таблица «events» растёт до миллиарда строк, и некоторые простые SELECT начинают тормозить. Что стоит попробовать сначала?

Подсказка: Начните с минимальных изменений перед масштабной переделкой.

Ответ:

- **Добавление подходящего индекса на часто фильтруемые столбцы**
- Разбиение таблицы вручную на части
- Переход на NoSQL
- Удаление всех старых строк без архивации

Шаг 02:

Миссия 2

После добавления индекса запрос всё ещё медленный. Коллега предлагает использовать партиционирование. Что важно учесть перед партиционированием?

Подсказка: Партиционирование имеет смысл, если часто фильтруют по какому-то полю.

Ответ:

- **Наличие предсказуемого ключа разделения (например, дата или диапазон)**
- Партиционирование по всем столбцам одновременно
- Сжатие данных перед партиционированием
- Перенос всех данных на отдельный физический диск

Шаг 03:

Миссия 3

Вы — PHP-разработчик крупной системы, которая взаимодействует с базой данных Oracle. Обнаружили, что запросы к большим таблицам слишком медленные. Что поможет улучшить производительность на первом этапе?

Подсказка: Начните с простых и эффективных методов оптимизации, не меняя архитектуру.

Ответ:

- **Добавить индексы на часто используемые поля**
- Разбить таблицу на части вручную
- Использовать несколько подключений к базе параллельно
- Полностью переписать SQL-запросы на PL/SQL

Шаг 04:

Миссия 4

Вы заметили, что SELECT-запрос к большой таблице часто вызывает блокировки, замедляя систему. Какой подход поможет уменьшить блокировки при чтении данных?

Подсказка: Подумайте, как отделить операции чтения от записи для повышения параллелизма.

Ответ:

- **Использовать чтение с уровнем изоляции READ ONLY**

- Увеличить таймауты на блокировки
- Уменьшить количество подключений к базе
- Удалить индексы, чтобы ускорить вставку

Шаг 05:

Миссия 5

Ваша задача — ускорить выборку из огромной таблицы с миллионами строк, используя Oracle. Что из ниже перечисленного окажется самым эффективным?

Подсказка: Вспомните, как физически разбивают большие таблицы для повышения скорости доступа.

Ответ:

- **Настроить партиционирование таблицы**
- Увеличить размер буфера Oracle
- Использовать только плоские индексы
- Часто перезапускать сервер базы данных

Шаг 06:

Миссия 6

Вы обнаружили, что некоторые запросы в Oracle, которые соединяют большие таблицы, работают очень долго. Что может помочь ускорить такие запросы?

Подсказка: Подумайте о том, как уменьшить объём обрабатываемых данных и ускорить соединение таблиц.

Ответ:

- **Использовать индексы по колонкам, участвующим в JOIN**
- Удалить все индексы
- Уменьшить размер кэш-памяти Oracle
- Использовать вложенные циклы вместо хеш-соединений

Шаг 07:

Миссия 7

 Задача: Сопоставьте термины оптимизации работы с большими таблицами Oracle с их описаниями.

Подсказка: Обратите внимание, как разные методы помогают работать с данными быстрее и эффективнее.

Термины:

Ответ:

- Партиционирование
- Индексы
- Буферизация
- Фрагментация
- Оптимизатор запросов

Определения:

1. Разбиение таблицы на логические части по определённому признаку.
2. Структуры для быстрого поиска данных по ключам.
3. Механизм хранения часто используемых данных в памяти для ускорения доступа.

4. Накопление и распределение данных, ухудшающее производительность.

5. Компонент Oracle, выбирающий лучший план выполнения запроса.

- **Партиционирование** — Разбиение таблицы на логические части по определённому признаку.
- **Индексы** — Структуры для быстрого поиска данных по ключам.
- **Буферизация** — Механизм хранения часто используемых данных в памяти для ускорения доступа.
- **Фрагментация** — Накопление и распределение данных, ухудшающее производительность.
- **Оптимизатор запросов** — Компонент Oracle, выбирающий лучший план выполнения запроса.

Шаг 08:

Миссия 8

 Задача: Сопоставьте признаки плохой производительности запросов с возможными причинами.

Подсказка: Подумайте, как архитектура базы влияет на время выполнения.

Признаки:

Ответ:

- Длительное выполнение запроса
- Частые блокировки
- Высокая нагрузка на CPU
- Большой объем данных на диск
- Повторяющиеся сканирования таблиц

Причины:

1. Отсутствие индексов по ключевым полям
2. Неоптимальные транзакции и уровни изоляции
3. Сложные запросы с множеством соединений
4. Недостаток памяти для буферизации
5. Использование полных сканирований вместо выборочных

- **Длительное выполнение запроса** — Сложные запросы с множеством соединений
- **Частые блокировки** — Неоптимальные транзакции и уровни изоляции
- **Высокая нагрузка на CPU** — Отсутствие индексов по ключевым полям
- **Большой объем данных на диск** — Недостаток памяти для буферизации
- **Повторяющиеся сканирования таблиц** — Использование полных сканирований вместо выборочных

Шаг 09:

Миссия 9

 Задача: Сопоставьте виды индексов Oracle с их особенностями.

Подсказка: Индексы бывают разными — вспомните, какие используются для уникальности, быстрого доступа и хранения.

Виды индексов:

Ответ:

- B-tree индекс
- Bitmap индекс

- Кластерный индекс
- Индекс на основе хеширования
- Индекс с обратным порядком (Reverse key)

Особенности:

1. Оптимален для поиска по диапазонам и сортировки
 2. Эффективен для столбцов с малым числом уникальных значений
 3. Данные физически организованы по ключу индекса
 4. Хорош для равенства, но плохо с диапазонами
 5. Защищает от «горячих точек» при последовательных вставках
- **B-tree индекс** — Оптимален для поиска по диапазонам и сортировки
 - **Bitmap индекс** — Эффективен для столбцов с малым числом уникальных значений
 - **Кластерный индекс** — Данные физически организованы по ключу индекса
 - **Индекс на основе хеширования** — Хорош для равенства, но плохо с диапазонами
 - **Индекс с обратным порядком (Reverse key)** — Защищает от «горячих точек» при последовательных вставках

Шаг 10:

Миссия 10

 Задача: Сопоставьте методы оптимизации запросов с их эффектами.

Подсказка: Помните, что разные методы влияют на время выполнения и нагрузку по-разному.

Методы оптимизации:

Ответ:

- Использование сжатия данных (compression)
- Параллельное выполнение запросов
- Использование материализованных представлений
- Реорганизация таблиц (rebuild)
- Обновление статистики таблиц

Эффекты:

1. Уменьшение размера таблиц и ускорение сканирования
 2. Ускорение вычислений за счёт распараллеливания
 3. Кеширование результата часто используемых запросов
 4. Удаление фрагментации и повышение производительности вставок
 5. Обеспечение правильных планов оптимизатора
- **Использование сжатия данных (compression)** — Уменьшение размера таблиц и ускорение сканирования
 - **Параллельное выполнение запросов** — Ускорение вычислений за счёт распараллеливания
 - **Использование материализованных представлений** — Кеширование результата часто используемых запросов
 - **Реорганизация таблиц (rebuild)** — Удаление фрагментации и повышение производительности вставок
 - **Обновление статистики таблиц** — Обеспечение правильных планов оптимизатора

Шаг 11:

Миссия 11

Расположите шаги диагностики медленных запросов в Oracle в правильном порядке:

1. Запустить план выполнения (EXPLAIN PLAN) для запроса
2. Проанализировать статистику индексов и таблиц
3. Определить узкие места и возможные причины замедления
4. Применить корректирующие меры (индексы, партиционирование и т.д.)
5. Повторно протестировать запрос на производительность

🔮 **Подсказка:**

Диагностика — это итеративный процесс от анализа к улучшению.

Ответ:

- Запустить план выполнения (EXPLAIN PLAN) для запроса
- Проанализировать статистику индексов и таблиц
- Определить узкие места и возможные причины замедления
- Применить корректирующие меры (индексы, партиционирование и т.д.)
- Повторно протестировать запрос на производительность

🔮 **Шаг 12:**

Миссия 12

Упорядочите этапы оптимизации большого SELECT-запроса в PHP, который обращается к Oracle:

1. Провести профилирование запроса (SQL Trace, TKPROF)
2. Проверить правильность индексов и статистики
3. Переписать запрос с использованием партиционирования
4. Обновить метаданные и пересобрать индексы при необходимости
5. Тестировать производительность после каждого изменения

🔮 **Подсказка:**

Работа должна идти циклично: анализ — исправление — проверка.

Ответ:

- Провести профилирование запроса (SQL Trace, TKPROF)
- Проверить правильность индексов и статистики
- Переписать запрос с использованием партиционирования
- Обновить метаданные и пересобрать индексы при необходимости
- Тестировать производительность после каждого изменения

🔮 **Шаг 13:**

Миссия 13

Андрей, PHP-разработчик, заметил, что запросы к большим таблицам в Oracle выполняются слишком долго. Какое из решений поможет уменьшить время выполнения простого SELECT по большому объему данных?

Подсказка: Подумайте, как можно уменьшить количество данных для обработки.

Ответ:

- **Добавить индекс по часто используемому полю**
- Использовать SELECT * для получения всех столбцов
- Увеличить размер блока чтения без анализа запросов
- Уменьшить время отклика клиента

🔗 Шаг 14:

Миссия 14

Ольга разрабатывает API на PHP для большого хранилища Oracle. Она хочет оптимизировать работу с данными при пагинации. Что из перечисленного поможет ускорить выборку страниц?

Подсказка: Подумайте, как минимизировать обработку ненужных строк.

Ответ:

- **Использовать индексируемые колонки в WHERE и OFFSET**
- Всегда делать полное сканирование таблицы
- Увеличить размер кэша PHP
- Использовать несколько отдельных запросов для каждого поля

🔗 Шаг 15:

Миссия 15

Павел заметил, что частые вставки большого количества строк в Oracle тормозят систему. Какой способ оптимизации поможет ускорить массовую вставку данных через PHP?

Подсказка: Сократите количество отдельных операций вставки.

Ответ:

- **Использовать пакетные вставки (batch insert)**
- Вставлять по одной строке за раз
- Добавлять индекс перед вставкой
- Выключить все триггеры

🔗 Шаг 16:

Миссия 16

Марина хочет ускорить выборку из большой таблицы. Что из перечисленного поможет уменьшить нагрузку на сервер Oracle?

Подсказка: Используйте стратегии для ограничения объема данных.

Ответ:

- **Создать материализованный вид с агрегатами**
- Увеличить количество параллельных запросов
- Запросить все строки без условий
- Использовать внешние соединения для всех таблиц

🔗 Шаг 17:

Миссия 17

Сергей заметил, что запросы с JOIN на больших таблицах выполняются медленно. Что поможет повысить производительность?

Подсказка: Обратите внимание на ключи и индексы.

Ответ:

- **Добавить индексы по колонкам для соединения**
- Использовать CROSS JOIN всегда
- Увеличить время ожидания таймаута
- Удалить все ограничения таблиц

Шаг 18:

Миссия 18

Сопоставьте понятия оптимизации работы с большими таблицами и их определения:

Понятия:

Ответ:

- Индекс
- Партирование (partitioning)
- Материализованный вид
- Пакетная вставка

Определения:

1. Разделение таблицы на логические части для ускорения запросов
2. Объект базы данных, который хранит предвычисленные результаты запроса
3. Структура данных для быстрого поиска по таблице
4. Вставка нескольких строк за одну операцию

Подсказка: Вспомните, как ускоряют доступ и уменьшают нагрузку при работе с большими объемами.

- **Индекс** — Структура данных для быстрого поиска по таблице
- **Партирование (partitioning)** — Разделение таблицы на логические части для ускорения запросов
- **Материализованный вид** — Объект базы данных, который хранит предвычисленные результаты запроса
- **Пакетная вставка** — Вставка нескольких строк за одну операцию

Шаг 19:

Миссия 19

Сопоставьте причины медленной работы запросов и их возможные решения:

Причины:

Ответ:

- Отсутствие индексов
- Большое количество соединений JOIN
- Использование SELECT *
- Частые мелкие вставки

Решения:

1. Создать необходимые индексы по ключевым колонкам
2. Оптимизировать структуру запросов, уменьшить количество JOIN
3. Выбирать только нужные колонки
4. Использовать пакетную вставку данных

Подсказка: Подумайте, какие операции создают нагрузку и как их снизить.

- **Отсутствие индексов** — Создать необходимые индексы по ключевым колонкам
- **Большое количество соединений JOIN** — Оптимизировать структуру запросов, уменьшить количество JOIN
- **Использование SELECT *** — Выбирать только нужные колонки
- **Частые мелкие вставки** — Использовать пакетную вставку данных

Урок 2 - Использование partitioning для разделения данных

Шаг 01:

Миссия 1

Иван — разработчик крупной системы, и у него есть таблица логов с миллиардами строк. Он слышал про partitioning в Oracle. Что из нижеперечисленного — основное преимущество partitioning?

Подсказка: Подумайте, как разделение данных влияет на план запросов и управление данными.

Ответ:

- **Позволяет обращаться к части данных, избегая сканирования всей таблицы**
- Устраняет необходимость в индексах вовсе
- Автоматически распределяет нагрузку на разные БД-серверы
- Уменьшает объём данных на диске вдвое без сжатия

Шаг 02:

Миссия 2

 Задача: Сопоставьте тип partitioning с его описанием.

Типы partitioning:

Ответ:

- Range Partitioning
- List Partitioning
- Hash Partitioning
- Composite Partitioning

Описание:

1. Разделение по диапазонам значений (например, даты).
2. Разделение по значению из конечного списка (например, категорий).
3. Разделение по хеш-функции, для равномерного распределения.
4. Комбинация двух методов (например, сначала по range, потом hash).

Подсказка: Часто используют гибридные схемы для сложных данных.

- **Range Partitioning** — Разделение по диапазонам значений (например, даты).
- **List Partitioning** — Разделение по значению из конечного списка (например, категорий).
- **Hash Partitioning** — Разделение по хеш-функции, для равномерного распределения.
- **Composite Partitioning** — Комбинация двух методов (например, сначала по range, потом hash).

Шаг 03:

Миссия 3

 Задача: Сопоставьте задачу, с которой столкнулась команда, с подходящим решением partitioning.

Проблемы:

Ответ:

- Сканирование всей таблицы при запросах по дате
- Нечёткое распределение по категориям
- Неравномерная нагрузка на некоторые partition'ы

- Очень большие партиции мешают обслуживанию

Решения:

1. Использовать range partition по дате
2. Использовать list partitioning по категории
3. Использовать hash или composite-дополнение
4. Разбить слишком крупные partition'ы на поддиапазоны

Подсказка: В сложных случаях комбинируют разные подходы.

- **Сканирование всей таблицы при запросах по дате** — Использовать range partition по дате
- **Нечёткое распределение по категории** — Использовать list partitioning по категории
- **Неравномерная нагрузка на некоторые partition'ы** — Использовать hash или composite-дополнение
- **Очень большие партиции мешают обслуживанию** — Разбить слишком крупные partition'ы на поддиапазоны

Шаг 04:

Миссия 4

 Задача: Сопоставьте термин partition-схемы с её преимуществом.

Схемы / термины:

Ответ:

- Interval Partitioning
- Subpartitioning
- Virtual Column Partitioning
- Reference Partitioning

Преимущества:

1. Автоматическое создание новых partition'ов по мере роста данных.
2. Дополнительный уровень деления внутри partition'ов.
3. Разделение по выражению (виртуальному столбцу).
4. Partitioning по ссылке на родительскую таблицу (foreign key).

Подсказка: Такие схемы помогают уменьшить ручное управление partition'ами.

- **Interval Partitioning** — Автоматическое создание новых partition'ов по мере роста данных.
- **Subpartitioning** — Дополнительный уровень деления внутри partition'ов.
- **Virtual Column Partitioning** — Разделение по выражению (виртуальному столбцу).
- **Reference Partitioning** — Partitioning по ссылке на родительскую таблицу (foreign key).

Шаг 05:

Миссия 5

 Задача: Сопоставьте ситуацию и возможную проблему при partitioning.

Ситуации:

Ответ:

- Запрос не использует ключ partition'а
- Много мелких partition'ов

- Частое объединение между partition'ами
- Изменение ключа partition'ing'a

Проблемы:

1. Partition pruning не работает, будет full scan
2. Управление накладных становится большим
3. Сложность выполнения JOIN / перераспределения
4. Нельзя просто изменить ключ после создания partition'ов

Подсказка: Подумайте о поведении Oracle при разных сценариях.

- **Запрос не использует ключ partition'a** — Partition pruning не работает, будет full scan
- **Много мелких partition'ов** — Управление накладных становится большим
- **Частое объединение между partition'ами** — Сложность выполнения JOIN / перераспределения
- **Изменение ключа partition'ing'a** — Нельзя просто изменить ключ после создания partition'ов

Шаг 06:

Миссия 6

 Задача: Сопоставьте этапы работы с партициями и что нужно делать.

Этапы:

Ответ:

- Создание новых partition'ов
- Объединение мелких partition'ов (merge)
- Разделение слишком больших partition'ов (split)
- Удаление устаревших partition'ов (drop)

Действия:

1. alter table ... split partition ...
2. alter table ... add partition ...
3. alter table ... drop partition ...
4. alter table ... merge partition ...

Подсказка: В Oracle есть конкретные DDL-операции для каждого вида работы с partition.

- **Создание новых partition'ов** — alter table ... add partition ...
- **Объединение мелких partition'ов (merge)** — alter table ... merge partition ...
- **Разделение слишком больших partition'ов (split)** — alter table ... split partition ...
- **Удаление устаревших partition'ов (drop)** — alter table ... drop partition ...

Шаг 07:

Миссия 7

Расположите шаги при миграции существующей монолитной таблицы в partitioned таблицу:

1. Создать новую таблицу с partitioning
2. Копировать данные порциями в новую таблицу
3. Синхронизировать изменения (дельты) между старой и новой
4. Переименовать таблицы и переключить пользователей
5. Удалить старую таблицу и очистить дельты

Подсказка: Важен момент переключения без простоя.

Ответ:

- Создать новую таблицу с partitioning
- Копировать данные порциями в новую таблицу
- Синхронизировать изменения (дельты) между старой и новой
- Переименовать таблицы и переключить пользователей
- Удалить старую таблицу и очистить дельты

 Шаг 08:

Миссия 8

Расположите действия при обслуживании partition'ов (reorganize / rebuild):

1. Выбрать проблемные партии
2. Выполнить REBUILD / MODIFY PARTITION
3. Синхронизировать индексы
4. Проверить статистику
5. Сделать COMMIT / проверить состояние

Подсказка: Обслуживание партиций включает перерасчёт и синхронизацию.

Ответ:

- Выбрать проблемные партии
- Выполнить REBUILD / MODIFY PARTITION
- Синхронизировать индексы
- Проверить статистику
- Сделать COMMIT / проверить состояние

 Шаг 09:

Миссия 9

Расположите этапы работы при interval-partitioning (с авто-созданием партиций):

1. Создать таблицу с interval partitioning
2. Положить данные в таблицу (Oracle сам создаёт партиции)
3. Контроль за ростом партиций
4. Обслуживание старых партиций
5. Удаление или архивирование устаревших партиций

Подсказка: Interval partitioning упрощает ручное управление partition'ами.

Ответ:

- Создать таблицу с interval partitioning
- Положить данные в таблицу (Oracle сам создаёт партиции)
- Контроль за ростом партиций
- Обслуживание старых партиций
- Удаление или архивирование устаревших партиций

 Шаг 10:

Миссия 10

Расположите операции при ребалансировке данных между partition'ами / шарами:

1. Проанализировать нагрузку и размер partition'ов

2. Выбрать partition'ы для сдвига или перераспределения
3. Выполнить split / merge / move
4. Обновить маршрутизацию / метаданные приложения
5. Мониторинг после операций

Подсказка: Ребалансировка часто нужна после роста данных.

Ответ:

- Проанализировать нагрузку и размер partition'ов
- Выбрать partition'ы для сдвига или перераспределения
- Выполнить split / merge / move
- Обновить маршрутизацию / метаданные приложения
- Мониторинг после операций

Шаг 11:

Миссия 11

Расположите этапы внедрения composite partitioning (range + hash):

1. Определить первичный range ключ (например, дата)
2. Определить вторичный hash ключ (например, user_id)
3. Создать таблицу composite partitioning
4. Мигрировать данные в новую структуру
5. Проверить распределение и производительность

Подсказка: Composite позволяет комбинировать преимущества разных методов.

Ответ:

- Определить первичный range ключ (например, дата)
- Определить вторичный hash ключ (например, user_id)
- Создать таблицу composite partitioning
- Мигрировать данные в новую структуру
- Проверить распределение и производительность

Шаг 12:

Миссия 12

Расположите порядок действий при замене существующей partitioned таблицы без простоев:

1. Создать новую partitioned структуру параллельно
2. Мигрировать исторические данные частями
3. Синхронизация последних изменений (trigger / лог)
4. Коммутирование доступа (alias / rename)
5. Удаление старой структуры и очистка триггеров

Подсказка: Задача — минимальный downtime и консистентность.

Ответ:

- Создать новую partitioned структуру параллельно
- Мигрировать исторические данные частями
- Синхронизация последних изменений (trigger / лог)
- Коммутирование доступа (alias / rename)
- Удаление старой структуры и очистка триггеров

🔮 Шаг 13:

Миссия 13

Расположите уровни сложности внедрения partitioning (от простейшего к наиболее сложному):

1. Range partitioning
2. Interval partitioning
3. Subpartitioning
4. Composite partitioning
5. Reference / hybrid схемы с шардированием

Подсказка: Более сложные схемы дают гибкость, но требуют больше усилий.

Ответ:

- Range partitioning
- Interval partitioning
- Subpartitioning
- Composite partitioning
- Reference / hybrid схемы с шардированием

🔮 Шаг 14:

Миссия 14

Аня, разработчик крупной банковской системы, хочет ускорить запросы к огромной таблице транзакций. Какой подход Oracle для разделения данных стоит применить в первую очередь?

Подсказка: Подумайте, как можно логически разделить данные для повышения производительности.

Ответ:

- **Создать partition по дате транзакций**
- Использовать индекс по всем полям
- Создать view для данных
- Оптимизировать PL/SQL код

🔮 Шаг 15:

Миссия 15

Павел столкнулся с медленной загрузкой отчетов по региональным продажам. Какой тип partitioning поможет ему оптимизировать выборки по региону?

Подсказка: Подумайте, как можно логически распределить данные по географии.

Ответ:

- **Range partition по регионам**
- Hash partition по идентификаторам клиентов
- List partition по типам продуктов

🔮 Шаг 16:

Миссия 16

Инна хочет, чтобы таблица логов автоматически создавалась с новым партиционированием каждый месяц. Что ей нужно использовать?

Подсказка: Автоматическое создание партиций связано с динамическими интервалами времени.

Ответ:

- **Interval partitioning**
- List partitioning
- Hash partitioning
- Composite partitioning

 **Шаг 17:**

Миссия 17

Для увеличения производительности выборки Иван хочет использовать комбинированное партиционирование по дате и типу клиента. Как называется такой тип partitioning?

Подсказка: Вспомните, что можно комбинировать несколько методов partitioning.

Ответ:

- **Composite partitioning**
- Hash partitioning
- Range partitioning

 **Шаг 18:**

Миссия 18

Сергей замечает, что некоторые партии сильно загружены, а другие почти пусты. Что он должен проверить в первую очередь?

Подсказка: Это может быть связано с неравномерным распределением данных.

Ответ:

- **Правильность выбранного ключа партиционирования**
- Настройку кеша Oracle
- Версию PHP
- Параметры сетевого подключения

Урок 3 - Загрузка и выгрузка данных с помощью SQLLoader

 **Шаг 01:**

Миссия 1

Алексей только что получил задание загрузить большой CSV-файл в базу Oracle с помощью SQL*Loader. Что нужно сделать в первую очередь?

Подсказка: Перед началом загрузки важно подготовить правильный файл управления.

Ответ:

- **Создать control-файл с описанием формата данных и таблицы**
- Запустить sqlplus и выполнить insert-запрос
- Просто скопировать CSV в директорию базы данных
- Перезапустить базу данных

 **Шаг 02:**

Миссия 2

Мария хочет импортировать данные, которые содержат даты в формате "DD-MM-YYYY". Как ей указать правильный формат даты в control-файле SQL*Loader?

Подсказка: Формат даты должен совпадать с форматом в данных и быть указан в разделе fields.

Ответ:

- **DATE "DD-MM-YYYY"**
- DATE "YYYY-MM-DD"
- DATE "MM-DD-YYYY"
- DATE "YY/MM/DD"

 **Шаг 03:**

Миссия 3

Иван загружает данные с помощью SQL*Loader и хочет пропустить первые 2 строки файла с заголовками. Как он это укажет в control-файле?

Подсказка: В control-файле можно использовать директиву, которая игнорирует начальные строки.

Ответ:

- **SKIP 2**
- IGNORE 2
- START 2

 **Шаг 04:**

Миссия 4

Алена заметила, что в данных могут быть ошибки и хочет, чтобы SQL*Loader продолжил загрузку, даже если встретит ошибочные строки. Что ей нужно добавить в команду загрузки?

Подсказка: Обработка ошибок позволяет загружать максимально возможные данные.

Ответ:

- **ERRORS=50**
- STOPONERROR=TRUE
- CONTINUE=FALSE
- SKIP_ERRORS

 **Шаг 05:**

Миссия 5

Даниил хочет загрузить данные в конкретную таблицу, но боится, что существующие данные будут перезаписаны. Какой режим загрузки нужно выбрать в SQL*Loader?

Подсказка: Различные режимы управления загрузкой позволяют добавлять или заменять данные.

Ответ:

- **APPEND**
- REPLACE
- TRUNCATE

 **Шаг 06:**

Миссия 6

 **Задача:** Сопоставьте параметры SQL*Loader с их описаниями.

Параметры:

Ответ:

- LOAD DATA

- INFILE
- BADFILE
- DISCARDFILE
- CONTROL

Определения:

1. Файл, который содержит правила загрузки и формат данных.
2. Основной файл с данными для загрузки.
3. Файл для записи строк с ошибками загрузки.
4. Файл для записи строк, не удовлетворяющих условиям загрузки.
5. Команда начала определения загрузки данных.

Подсказка: Подумайте, какую роль играет каждый файл в процессе загрузки.

- **LOAD DATA** — Команда начала определения загрузки данных.
- **INFILE** — Основной файл с данными для загрузки.
- **BADFILE** — Файл для записи строк с ошибками загрузки.
- **DISCARDFILE** — Файл для записи строк, не удовлетворяющих условиям загрузки.
- **CONTROL** — Файл, который содержит правила загрузки и формат данных.

Шаг 07:

Миссия 7

 Задача: Сопоставьте режимы загрузки с их описаниями.

Режимы:

Ответ:

- INSERT
- APPEND
- REPLACE
- TRUNCATE

Определения:

1. Заменяет все данные в таблице перед загрузкой новых.
2. Добавляет данные к существующим в таблице.
3. Вставляет данные, ожидая, что таблица пуста.
4. Очищает таблицу и загружает новые данные, более эффективно чем REPLACE.

Подсказка: Режимы управления загрузкой влияют на существующие данные в таблице.

- **REPLACE** — Заменяет все данные в таблице перед загрузкой новых.
- **APPEND** — Добавляет данные к существующим в таблице.
- **INSERT** — Вставляет данные, ожидая, что таблица пуста.
- **TRUNCATE** — Очищает таблицу и загружает новые данные, более эффективно чем REPLACE.

Шаг 08:

Миссия 8

 Задача: Сопоставьте типы данных SQL*Loader с примерами из control-файла.

Типы:

Ответ:

- CHAR
- DATE
- INTEGER EXTERNAL
- FLOAT EXTERNAL

Примеры:

1. Поле с датой в формате "YYYY-MM-DD".
2. Текстовое поле фиксированной длины.
3. Целое число в текстовом формате.
4. Число с плавающей точкой в текстовом формате.

Подсказка: В control-файле типы данных описывают, как парсить входные данные.

- **CHAR** — Текстовое поле фиксированной длины.
- **DATE** — Поле с датой в формате "YYYY-MM-DD".
- **INTEGER EXTERNAL** — Целое число в текстовом формате.
- **FLOAT EXTERNAL** — Число с плавающей точкой в текстовом формате.

 Шаг 09:

Миссия 9

 Задача: Сопоставьте ключевые параметры SQL*Loader с их назначением.

Параметры:

Ответ:

- ROWS
- BINDSIZE
- READSIZE
- DIRECT
- PARALLEL

Назначения:

1. Определяет число строк, загружаемых за одну операцию.
2. Устанавливает размер буфера для связывания данных с базой.
3. Размер блока чтения данных из файла.
4. Использование прямой загрузки, минуя буфер SQL.
5. Позволяет выполнять параллельную загрузку для ускорения.

Подсказка: Эти параметры влияют на производительность загрузки.

- **ROWS** — Определяет число строк, загружаемых за одну операцию.
- **BINDSIZE** — Устанавливает размер буфера для связывания данных с базой.
- **READSIZE** — Размер блока чтения данных из файла.
- **DIRECT** — Использование прямой загрузки, минуя буфер SQL.
- **PARALLEL** — Позволяет выполнять параллельную загрузку для ускорения.

 Шаг 10:

Миссия 10

Расположите шаги подготовки control-файла для загрузки данных с помощью SQL*Loader в правильном порядке:

1. Определить таблицу и поля для загрузки
2. Указать формат входных данных и разделители
3. Написать заголовок LOAD DATA и определить имя infile
4. Сохранить control-файл и выполнить загрузку командой sqlldr

Подсказка: Контрольный файл должен содержать четкое описание формата и данных перед запуском.

Ответ:

- Написать заголовок LOAD DATA и определить имя infile
- Определить таблицу и поля для загрузки
- Указать формат входных данных и разделители
- Сохранить control-файл и выполнить загрузку командой sqlldr

Шаг 11:

Миссия 11

Расположите этапы обработки ошибок при загрузке данных с помощью SQL*Loader в правильном порядке:

1. Проверка BADFILE для анализа ошибочных записей
2. Настройка параметра ERRORS для допуска числа ошибок
3. Запуск загрузки с контролем ошибок
4. Исправление данных и повторная загрузка

Подсказка: Обработка ошибок помогает сохранить целостность данных и корректность загрузки.

Ответ:

- Настройка параметра ERRORS для допуска числа ошибок
- Запуск загрузки с контролем ошибок
- Проверка BADFILE для анализа ошибочных записей
- Исправление данных и повторная загрузка

Шаг 12:

Миссия 12

Расположите шаги настройки загрузки данных с несколькими файлами (INFILE) в control-файле:

1. Перечислить все файлы с данными через ключевое слово INFILE
2. Указать таблицу и поля для загрузки
3. Определить правила обработки для каждого файла (если отличаются)
4. Запустить sqlldr с подготовленным control-файлом

Подсказка: Можно загружать сразу несколько файлов, указав их в control-файле.

Ответ:

- Перечислить все файлы с данными через ключевое слово INFILE
- Указать таблицу и поля для загрузки
- Определить правила обработки для каждого файла (если отличаются)
- Запустить sqlldr с подготовленным control-файлом

Шаг 13:

Миссия 13

Анна, разработчица, столкнулась с проблемой: данные из CSV-файла неправильно загружаются в базу. Какой параметр SQL*Loader поможет ей определить корректный формат даты при загрузке?

Подсказка: Формат даты должен точно соответствовать шаблону в control-файле.

Ответ:

- **DATE "YYYY-MM-DD"**
- CHAR
- INTEGER
- FLOAT

Шаг 14:

Миссия 14

Максим отвечает за выгрузку больших объемов данных в Oracle. Он хочет повысить скорость загрузки. Какой режим загрузки SQL*Loader следует использовать?

Подсказка: Прямая загрузка минуется SQL буфер и ускоряет процесс.

Ответ:

- **DIRECT=TRUE**
- ROWS=1000
- BINDSIZE=1048576
- PARALLEL=FALSE

Шаг 15:

Миссия 15

 Задача: Сопоставьте параметры SQL*Loader с их значениями и описаниями.

Параметры:

Ответ:

- BADFILE
- DISCARDFILE
- ERRORS
- SKIP

Назначения:

1. Файл для записи строк, которые не удалось загрузить
2. Максимальное число ошибок перед остановкой загрузки
3. Файл для записи отфильтрованных строк, не загружаемых в таблицу
4. Пропускает указанное количество строк в начале файла

Подсказка: Эти параметры помогают контролировать обработку ошибок и исключений при загрузке.

- **BADFILE** — Файл для записи строк, которые не удалось загрузить
- **ERRORS** — Максимальное число ошибок перед остановкой загрузки
- **DISCARDFILE** — Файл для записи отфильтрованных строк, не загружаемых в таблицу
- **SKIP** — Пропускает указанное количество строк в начале файла

Шаг 16:

Миссия 16

 Задача: Сопоставьте типы ошибок SQL*Loader с возможными причинами.

Типы ошибок:

Ответ:

- ORACLE error
- Data error
- File I/O error
- Control file error

Возможные причины:

1. Ошибки в SQL запросах или триггерах
2. Неверный формат или тип данных в исходном файле
3. Проблемы с доступом к файлам или правами
4. Ошибки в синтаксисе или структуре control-файла

Подсказка: Ошибки могут возникать на разных этапах загрузки и по разным причинам.

- **ORACLE error** — Ошибки в SQL запросах или триггерах
- **Data error** — Неверный формат или тип данных в исходном файле
- **File I/O error** — Проблемы с доступом к файлам или правами
- **Control file error** — Ошибки в синтаксисе или структуре control-файла

Шаг 17:

Миссия 17

Расположите этапы выполнения загрузки данных с помощью SQL*Loader в правильном порядке:

1. Создание control-файла с описанием формата данных
2. Запуск команды sqlldr с параметрами
3. Проверка логов и файлов ошибок после загрузки
4. Подготовка исходного файла с данными

Подсказка: Все шаги важны для успешной загрузки и отладки.

Ответ:

- Подготовка исходного файла с данными
- Создание control-файла с описанием формата данных
- Запуск команды sqlldr с параметрами
- Проверка логов и файлов ошибок после загрузки

Шаг 18:

Миссия 18

Расположите шаги настройки параллельной загрузки данных в SQL*Loader:

1. Подготовка control-файла с параметром PARALLEL
2. Запуск нескольких процессов sqlldr с разными параметрами
3. Установка DIRECT=TRUE для повышения скорости
4. Настройка таблицы для поддержки параллельных операций

Подсказка: Параллельная загрузка требует согласованной подготовки нескольких компонентов.

Ответ:

- Подготовка control-файла с параметром PARALLEL
- Настройка таблицы для поддержки параллельных операций
- Установка DIRECT=TRUE для повышения скорости
- Запуск нескольких процессов sqlldr с разными параметрами

 **Шаг 19:**

Миссия 19

Расположите шаги обработки ошибок при загрузке данных с SQL*Loader в правильном порядке:

1. Анализ BADFILE и DISCARDFILE для выявления проблемных записей
2. Исправление ошибок в исходном файле или control-файле
3. Повторный запуск загрузки с исправленными файлами
4. Просмотр логов для подтверждения успешной загрузки

Подсказка: Работа с ошибками — важный этап для корректной загрузки данных.

Ответ:

- Анализ BADFILE и DISCARDFILE для выявления проблемных записей
- Исправление ошибок в исходном файле или control-файле
- Повторный запуск загрузки с исправленными файлами
- Просмотр логов для подтверждения успешной загрузки

Урок 4 - Обработка больших данных в реальном времени

 **Шаг 01:**

Миссия 1

Алексей, разработчик крупной финансовой системы, получает первый запрос на обработку потока данных из Oracle в PHP в режиме реального времени. Какое решение будет самым простым и эффективным для старта интеграции?

Подсказка: Начинайте с проверенных подходов, минимизируя сложность.

Ответ:

- **Использовать OCI8 для подключения к Oracle и запросы с fetch в цикле**
- Писать собственный парсер бинарных данных из Oracle
- Заменить PHP на Java для работы с данными в реальном времени
- Записывать данные во временные файлы и считывать их в PHP

 **Шаг 02:**

Миссия 2

Марина работает с большим объемом финансовых транзакций, поступающих из Oracle. Какой подход поможет снизить нагрузку на сервер при обработке в PHP?

Подсказка: Думайте о распределении нагрузки и уменьшении повторной работы.

Ответ:

- **Использовать кэширование результатов запросов и батчевую обработку данных**
- Запрашивать все данные сразу и обрабатывать их в одном PHP-скрипте
- Каждый запрос обрабатывать в отдельном PHP-процессе
- Использовать один большой SQL-запрос без разбивки

Шаг 03:

Миссия 3

Игорь пишет PHP-сервис, который обрабатывает потоковые данные из Oracle. Какие индексы в базе помогут ускорить выборку по временным меткам?

Подсказка: Индексы помогают быстро находить записи по определённым колонкам.

Ответ:

- **Индексы по колонкам с датой и временем события**
- Индексы по текстовым полям с описаниями
- Индексы по случайным числам в таблице
- Индексы по булевым флагам

Шаг 04:

Миссия 4

Светлана разрабатывает PHP-приложение, которое должно реагировать на изменения данных в Oracle в режиме реального времени. Что из следующего может помочь реализовать такую реакцию?

Подсказка: Обратите внимание на возможности Oracle для уведомления клиентов.

Ответ:

- **Использовать Oracle Continuous Query Notification (CQN)**
- Постоянно опрашивать базу запросами SELECT
- Настроить триггеры, которые запускают PHP-скрипт
- Сохранять данные в файл и опрашивать его PHP

Шаг 05:

Миссия 5

Алена хочет обрабатывать потоковые данные из Oracle в PHP, минимизируя задержки. Что из следующего будет наилучшим способом обработки?

Подсказка: Подумайте о том, чтобы избежать блокировок и долго выполняющихся запросов.

Ответ:

- **Использовать асинхронные запросы и обработку в нескольких потоках**
- Обрабатывать все данные последовательно в одном процессе
- Использовать блокирующие транзакции для синхронизации
- Обрабатывать данные через один большой SQL-запрос

Шаг 06:

Миссия 6

 Задача: Сопоставьте типы кеширования с их описаниями для оптимизации обработки данных из Oracle в PHP.

Типы кеширования:

Ответ:

- Кеширование на уровне запроса
- Кеширование на уровне сессии
- Кеширование в памяти (in-memory)
- Кеширование на стороне клиента

Описание:

1. Хранение данных между запросами для одного пользователя
2. Использование памяти сервера для быстрого доступа к часто используемым данным
3. Сохранение результатов конкретного SQL-запроса для повторного использования
4. Хранение данных в браузере пользователя для уменьшения обращений к серверу

Подсказка:

Подумайте, где именно и как долго должны храниться данные для каждой стратегии кеширования.

- **Кеширование на уровне запроса** — Сохранение результатов конкретного SQL-запроса для повторного использования
- **Кеширование на уровне сессии** — Хранение данных между запросами для одного пользователя
- **Кеширование в памяти (in-memory)** — Использование памяти сервера для быстрого доступа к часто используемым данным
- **Кеширование на стороне клиента** — Хранение данных в браузере пользователя для уменьшения обращений к серверу

Шаг 07:

Миссия 7

 Задача: Сопоставьте методы оптимизации обработки больших данных из Oracle в PHP с их описаниями.

Методы оптимизации:

Ответ:

- Использование индексированных запросов
- Батчевая обработка данных
- Параллельная обработка потоков
- Минимизация объема передаваемых данных

Описание:

1. Обработка данных частями, чтобы избежать перегрузки памяти
2. Использование нескольких процессов для ускорения вычислений
3. Запрос только необходимых колонок и строк из базы
4. Использование специально подготовленных структур данных для быстрого поиска

Подсказка:

Подумайте, как каждая стратегия помогает снизить нагрузку или ускорить обработку.

- **Использование индексированных запросов** — Использование специально подготовленных структур данных для быстрого поиска
- **Батчевая обработка данных** — Обработка данных частями, чтобы избежать перегрузки памяти
- **Параллельная обработка потоков** — Использование нескольких процессов для ускорения вычислений
- **Минимизация объема передаваемых данных** — Запрос только необходимых колонок и строк из базы

Шаг 08:

Миссия 8

 **Задача:** Сопоставьте этапы обработки больших потоков данных в PHP и Oracle с их описаниями.

Этапы:

Ответ:

- Получение данных из Oracle
- Преобразование данных в PHP
- Кеширование результатов
- Отправка результата клиенту

Описание:

1. Использование OCI8 для выборки и обработки курсоров
2. Форматирование данных и выполнение логики бизнес-процессов
3. Сохранение часто запрашиваемых данных для ускорения ответов
4. Формирование HTTP-ответа с готовыми данными

 **Подсказка:**

Подумайте, в каком порядке должны происходить эти этапы для эффективной работы.

- **Получение данных из Oracle** — Использование OCI8 для выборки и обработки курсоров
- **Преобразование данных в PHP** — Форматирование данных и выполнение логики бизнес-процессов
- **Кеширование результатов** — Сохранение часто запрашиваемых данных для ускорения ответов
- **Отправка результата клиенту** — Формирование HTTP-ответа с готовыми данными

 **Шаг 09:**

Миссия 9

 **Задача:** Сопоставьте термины и их определения, связанные с обработкой данных в реальном времени из Oracle в PHP.

Термины:

Ответ:

- Латентность
- Пропускная способность
- Буферизация
- Транзакция

Определения:

1. Время задержки от поступления данных до их обработки
2. Объем данных, который система может обработать за единицу времени
3. Временное хранение данных перед обработкой
4. Набор операций, выполняемых атомарно

 **Подсказка:**

Разберитесь, какой термин описывает скорость, какой — задержку, а что связано с целостностью данных.

- **Латентность** — Время задержки от поступления данных до их обработки
- **Пропускная способность** — Объем данных, который система может обработать за единицу времени
- **Буферизация** — Временное хранение данных перед обработкой

- **Транзакция** — Набор операций, выполняемых атомарно

Шаг 10:

Миссия 10

Расположите этапы настройки Oracle Continuous Query Notification (CQN) для PHP-приложения в правильном порядке:

1. Регистрация слушателя уведомлений в PHP
2. Настройка подписки на события в Oracle
3. Обработка уведомлений о изменениях данных в PHP
4. Создание необходимой таблицы и индексов в Oracle

Подсказка:

Ответ:

- Начните с подготовки базы данных.
- Далее настройте подписку на события.
- Зарегистрируйте слушателя в PHP.
- Обработывайте уведомления при их поступлении.
- Создание необходимой таблицы и индексов в Oracle
- Настройка подписки на события в Oracle
- Регистрация слушателя уведомлений в PHP
- Обработка уведомлений о изменениях данных в PHP

Шаг 11:

Миссия 11

Упорядочите шаги по оптимизации PHP-кода для обработки больших данных из Oracle:

1. Профилирование и выявление узких мест
2. Использование подготовленных запросов и параметризации
3. Внедрение кеширования и асинхронной обработки
4. Рефакторинг и удаление дублирующегося кода

Подсказка:

Ответ:

- Сначала выясните, что тормозит работу.
- Оптимизируйте работу с базой данных.
- Добавьте ускорители и распараллеливание.
- Улучшите структуру кода для поддержки.
- Профилирование и выявление узких мест
- Использование подготовленных запросов и параметризации
- Внедрение кеширования и асинхронной обработки
- Рефакторинг и удаление дублирующегося кода

Шаг 12:

Миссия 12

Андрей разрабатывает систему мониторинга, которая получает данные из Oracle в реальном времени. Какой подход поможет уменьшить задержки при обработке больших объемов данных?

Подсказка: Важно минимизировать время ожидания между отправкой и получением данных.

Ответ:

- **Использование потоковой передачи данных (streaming) вместо пакетной загрузки**
- Загрузка всех данных целиком раз в час
- Ручной экспорт данных из Oracle в CSV
- Обработка данных только после полной загрузки

🔮 **Шаг 13:**

Миссия 13

Екатерина хочет повысить производительность PHP-скрипта, который обрабатывает миллионы записей из Oracle. Что ей стоит использовать?

Подсказка: Иногда помогает хранить результаты промежуточных вычислений.

Ответ:

- **Кэширование результатов запросов к Oracle**
- Повторный запуск всех запросов с нуля
- Использование PHP в режиме отладки
- Отключение индексов в Oracle

🔮 **Шаг 14:**

Миссия 14

Сопоставьте технологии и их роль в обработке больших данных в реальном времени.

Подсказка: Разные инструменты решают разные задачи: хранение, обработка, передача.

Ответ:

- Apache Kafka
 - Oracle Advanced Queuing
 - PHP OCI8
 - Redis
1. Клиентская библиотека PHP для работы с Oracle
 2. Система очередей сообщений для интеграции данных
 3. Платформа для потоковой передачи и обработки сообщений
 4. Высокоскоростной кэш для быстрого доступа к данным
- **Apache Kafka** — Платформа для потоковой передачи и обработки сообщений
 - **Oracle Advanced Queuing** — Система очередей сообщений для интеграции данных
 - **PHP OCI8** — Клиентская библиотека PHP для работы с Oracle
 - **Redis** — Высокоскоростной кэш для быстрого доступа к данным

🔮 **Шаг 15:**

Миссия 15

Сопоставьте методы оптимизации работы с большими данными в PHP и Oracle.

Подсказка: Подумайте о том, что помогает уменьшить нагрузку и ускорить обработку.

Ответ:

- Использование индексов в Oracle
- Разбиение больших запросов на части
- Асинхронная обработка в PHP

- Кэширование на уровне приложения

1. Ускорение поиска и выборки данных
2. Управление памятью и нагрузкой
3. Обработка без блокировки интерфейса
4. Сокращение числа обращений к базе

- **Использование индексов в Oracle** — Ускорение поиска и выборки данных
- **Разбиение больших запросов на части** — Управление памятью и нагрузкой
- **Асинхронная обработка в PHP** — Обработка без блокировки интерфейса
- **Кэширование на уровне приложения** — Сокращение числа обращений к базе

Шаг 16:

Миссия 16

Расположите этапы обработки потоковых данных из Oracle в PHP в правильном порядке:

1. Получение данных из очереди Oracle Advanced Queuing
2. Предварительная фильтрация и преобразование данных в PHP
3. Сохранение результатов обработки в базу данных
4. Настройка и инициализация потоковой обработки

Подсказка: Начните с подготовки, потом получение, обработку и запись результатов.

Ответ:

- Настройка и инициализация потоковой обработки
- Получение данных из очереди Oracle Advanced Queuing
- Предварительная фильтрация и преобразование данных в PHP
- Сохранение результатов обработки в базу данных

Шаг 17:

Миссия 17

Упорядочите шаги масштабирования системы обработки потоковых данных из Oracle в PHP:

1. Добавление очередей сообщений (например, RabbitMQ)
2. Разделение потоков данных по темам
3. Оптимизация SQL-запросов
4. Вертикальное и горизонтальное масштабирование серверов

Подсказка: Масштабирование начинается с оптимизации и расширения архитектуры.

Ответ:

- Оптимизация SQL-запросов
- Добавление очередей сообщений (например, RabbitMQ)
- Разделение потоков данных по темам
- Вертикальное и горизонтальное масштабирование серверов

Шаг 18:

Миссия 18

Расположите этапы устранения узких мест при работе с большими потоковыми данными:

1. Анализ производительности и выявление проблем
2. Оптимизация кода и запросов

3. Тестирование изменений под нагрузкой
4. Внедрение масштабируемых решений

Подсказка: Сначала выявляем проблемы, потом их исправляем и проверяем.

Ответ:

- Анализ производительности и выявление проблем
- Оптимизация кода и запросов
- Тестирование изменений под нагрузкой
- Внедрение масштабируемых решений

Урок 5 - Репликация данных в Oracle для повышения доступности

Шаг 01:

Миссия 1

Вы работаете над системой учёта с PHP и Oracle. Какой подход репликации наиболее прост для начала, чтобы повысить доступность чтения?

Подсказка: Начните с чего-то менее сложного, не сразу с кластеров.

Ответ:

- **Master → Read-only slave (односторонняя репликация)**
- Multi-master репликация
- Синхронный кластер на двух мастерах

Шаг 02:

Миссия 2

После выбора Master → Read-only slave, что критично удостовериться перед запуском репликации?

Подсказка: Подумайте о согласованности данных и нагрузке на сеть.

Ответ:

- **Снимок данных (флешбек или экспорт) и сетевой канал низкой задержки**
- Настроить все приложения писать только на slave
- Отключить архивирование redo логов
- Использовать несжатые сетевые пакеты по UDP

Шаг 03:

Миссия 3

Разработчик PHP спрашивает: “А что насчёт задержки между master и slave?” Что вы ответите?

Подсказка: Небольшая задержка (lag) — нормальное явление при асинхронной репликации.

Ответ:

- **Небольшой лаг возможен, но slave обычно отстаёт лишь на миллисекунды или секунды**
- Позже slave станет master автоматически без вмешательства
- Lag нельзя избежать, поэтому репликацию не стоит применять

Шаг 04:

Миссия 4

Вы решаете настроить Data Guard в Oracle. Какой режим для начала выбрать, чтобы не рисковать производительностью?

Подсказка: Синхронный режим может влиять на задержки записи.

Ответ:

- **Maximum Performance (асинхронный режим)**
- Maximum Protection (синхронный режим)
- Maximum Availability (полусинхронный режим)

Шаг 05:

Миссия 5

Вы выбираете между физической и логической репликацией Oracle. Что характерно для логической репликации?

Подсказка: Логическая — на уровне данных и DML.

Ответ:

- **Позволяет фильтровать таблицы и колонки**
- Копирует файлы данных блок за блоком
- Требуется идентичную файловую структуру хранения

Шаг 06:

Миссия 6

 Сопоставьте режим Data Guard с его свойствами.

Режимы:

Ответ:

- Maximum Performance
- Maximum Availability
- Maximum Protection

Свойства:

1. Синхронная передача, самый строгий контроль риска потери данных
2. Асинхронный режим, максимальная пропускная способность
3. Полусинхронный режим, компромисс между безопасностью и производительностью

Подсказка: Один режим жертвует задержками ради безопасности, другой — скоростью.

- **Maximum Performance** — Асинхронный режим, максимальная пропускная способность
- **Maximum Availability** — Полусинхронный режим, компромисс между безопасностью и производительностью
- **Maximum Protection** — Синхронная передача, самый строгий контроль риска потери данных

Шаг 07:

Миссия 7

 Сопоставьте понятия, связанные с репликацией, и их определения.

Понятия:

Ответ:

- Lag (задержка)
- Switchover

- Failover
- Snapshot Standby

Определения:

1. Автоматический переход на standby при сбое мастера
2. Синхронизация копии в течение определённого времени, с возможностью записи
3. Официальная плановая смена ролей master ↔ standby
4. Разница по времени или по числу применённых изменений между master и slave

Подсказка: Внимательно читайте определения: switchover — плановая смена, failover — аварийный.

- **Lag (задержка)** — Разница по времени или по числу применённых изменений между master и slave
- **Switchover** — Официальная плановая смена ролей master ↔ standby
- **Failover** — Автоматический переход на standby при сбое мастера
- **Snapshot Standby** — Синхронизация копии в течение определённого времени, с возможностью записи

Шаг 08:

Миссия 8

 Сопоставьте виды репликации Oracle с их характеристиками.

Виды:

Ответ:

- Physical standby
- Logical standby
- Active Data Guard

Характеристики:

1. Поддерживает запросы только на чтение, репликация на уровне файлов
2. Поддерживает запросы и DML на некоторых объектах, репликация через SQL
- <
3. Разрешает читать данные на standby параллельно с применением изменений

Подсказка: Active Data Guard — дополнение к physical standby.

- **Physical standby** — Поддерживает запросы только на чтение, репликация на уровне файлов
- **Logical standby** — Поддерживает запросы и DML на некоторых объектов, репликация через SQL
- **Active Data Guard** — Разрешает читать данные на standby параллельно с применением изменений

Шаг 09:

Миссия 9

Расположите шаги настройки физической репликации Oracle (Data Guard):

1. Подготовка standby и создание control file
2. Настройка сетевых связей (tnsnames, listener) между master и standby
3. Запуск логической передачи архивных логов на standby
4. Запуск служебной процедуры to start managed recovery

Подсказка: Сначала готовим окружение, затем запускаем передачу, затем recovery.

Ответ:

- Подготовка standby и создание control file
- Настройка сетевых связей (tnsnames, listener) между master и standby
- Запуск логической передачи архивных логов на standby
- Запуск служебной процедуры to start managed recovery

 Шаг 10:

Миссия 10

Расположите шаги переключения roles: switchover от master к standby:

1. Остановить службы на primary
2. Перевести standby в режим primary
3. Отправить команду switchover на primary
4. Запустить former primary как standby

Подсказка: Switchover — плановая смена ролей без потери данных.

Ответ:

- Остановить службы на primary
- Перевести standby в режим primary
- Отправить команду switchover на primary
- Запустить former primary как standby

 Шаг 11:

Миссия 11

Расположите шаги аварийного переключения (failover):

1. Обнаружение, что primary сломался
2. Активировать standby как новый primary
3. Проверить, что клиентские подключения переподключились
4. Отключить старый primary (если он вернётся, сделать его standby)

Подсказка: Failover — срочное переключение при неполадках.

Ответ:

- Обнаружение, что primary сломался
- Активировать standby как новый primary
- Проверить, что клиентские подключения переподключились
- Отключить старый primary (если он вернётся, сделать его standby)

 Шаг 12:

Миссия 12

Расположите шаги развёртывания Active Data Guard:

1. Настроить физический standby
2. Включить режим чтения на standby (open read-only)
3. Включить Apply режим фонового применения redo
4. Активировать возможность чтения во время apply

Подсказка: Добавляем чтение на standby после базовой настройки.

Ответ:

- Настроить физический standby
- Включить режим чтения на standby (open read-only)
- Включить Apply режим фонового применения redo
- Активировать возможность чтения во время apply

 Шаг 13:

Миссия 13

Расположите шаги при возврате после аварии (recovery и возврат роли):

1. Синхронизировать данные на бывшем primary
2. Сделать прежний primary standby
3. Выполнить switchover обратно, если требуется
4. Проверить консистентность и целостность данных

Подсказка: Восстановление должно идти аккуратно, с проверками.

Ответ:

- Синхронизировать данные на бывшем primary
- Сделать прежний primary standby
- Выполнить switchover обратно, если требуется
- Проверить консистентность и целостность данных

 Шаг 14:

Миссия 14

Расположите фазы тестирования отказоустойчивости после внедрения репликации:

1. Имитировать отказ primary (принудительное выключение)
2. Проверить, что приложение переключилось на standby
3. Вернуть primary и синхронизировать
4. Проверить корректность данных и журналов

Подсказка: Тестируйте сначала отказ, затем восстановление.

Ответ:

- Имитировать отказ primary (принудительное выключение)
- Проверить, что приложение переключилось на standby
- Вернуть primary и синхронизировать
- Проверить корректность данных и журналов

 Шаг 15:

Миссия 15

Распорядите шаги эволюции архитектуры от одножинной репликации к мульти-региональной:

1. Добавить второй slave в том же датацентре
2. Добавить реплику в удалённый регион
3. Настроить географическое распределение нагрузки чтения
4. Внедрить мониторинг репликации и метрики lag

Подсказка: Сначала масштаб внутри региона, затем гео.

Ответ:

- Добавить второй slave в том же датацентре
- Добавить реплику в удалённый регион
- Настроить географическое распределение нагрузки чтения
- Внедрить мониторинг репликации и метрики lag

 **Шаг 16:**

Миссия 16

Расположите этапы реагирования на рост lag выше порога в PHP-приложении:

1. Обнаружить, что lag превышен (метрики)
2. Перенаправить чтение на master временно
3. Оповестить инженеров и логгировать событие
4. **Когда lag спал** — вернуть чтения на slave

Подсказка: Важна быстрая реакция, затем возврат в норму.

Ответ:

- Обнаружить, что lag превышен (метрики)
- Перенаправить чтение на master временно
- Оповестить инженеров и логгировать событие
- **Когда lag спал** — вернуть чтения на slave

Модуль 5 - Интеграция и взаимодействие с другими сервисами

Урок 1 - Интеграция с внешними API через Oracle

 **Шаг 01:**

Миссия 1

Вы — разработчик, получивший задачу — интегрировать в Oracle внешний REST API погоды, чтобы сохранять данные прогноза в таблицу. Что вы выберете первым шагом?

Подсказка: Сначала важно удостовериться, что внешний API доступен и возвращает ожидаемые данные.

Ответ:

- **Сделать тестовый HTTP-запрос к API и посмотреть ответ**
- Сразу начать писать PL/SQL обёртку
- Создать шедулер и запланировать загрузку

 **Шаг 02:**

Миссия 2

Вы отправили запрос к API, получили JSON с прогнозом. Что нужно сделать дальше перед сохранением в Oracle?

Подсказка: Нужно привести внешний формат к внутренней модели данных.

Ответ:

- **Преобразовать JSON-ответ в нужные поля и типы**
- Сохранить «как есть» в VARCHAR2 колонку
- Игнорировать ошибочные поля и продолжить

🔮 Шаг 03:

Миссия 3

При интеграции API погоды вы заметили, что API требует API-ключ в заголовке. Как лучше это реализовать в Oracle?

Подсказка: Ключ — чувствительный параметр, не стоит хардкодить его в коде.

Ответ:

- **Хранить API-ключ в защищённой таблице или wallet и читать динамически**
- Вписать ключ прямо в строку запроса
- Послать запрос без ключа и ожидать редиректа

🔮 Шаг 04:

Миссия 4

Перед тем как внедрять интеграцию в продакшен, вам предлагают сделать мок API для тестирования. Какой подход предпочтительней?

Подсказка: Мок не должен быть копией — он должен симулировать граничные случаи.

Ответ:

- **Создать локальный stub, который возвращает варианты: нормальный ответ, ошибку, таймаут**
- Моделировать только один сценарий с успешным ответом
- Запросы на реальный API, но с фильтрацией

🔮 Шаг 05:

Миссия 5

📄 Сопоставьте концепции интеграции API с их описаниями.

Понятия:

Ответ:

- REST
- SOAP
- JSON
- XML

Описание:

1. Лёгкий текстовый формат обмена данными, часто используется в REST API
2. Сложная спецификация обмена сообщениями, часто в корпоративной среде
3. Структурированная разметка с тегами, широко используемая в SOAP
4. Стиль архитектуры веб-сервисов, основанный на HTTP и ресурсах

Подсказка: REST строится поверх HTTP, SOAP — отдельный стандарт.

- **REST** — Стиль архитектуры веб-сервисов, основанный на HTTP и ресурсах
- **SOAP** — Сложная спецификация обмена сообщениями, часто в корпоративной среде
- **JSON** — Лёгкий текстовый формат обмена данными, часто используется в REST API
- **XML** — Структурированная разметка с тегами, широко используемая в SOAP

🔮 Шаг 06:

Миссия 6

 Сопоставьте типы HTTP-методов с характером операции.

Методы:

Ответ:

- GET
- POST
- PUT
- DELETE

Операции:

1. Создать новый ресурс
2. Удалить ресурс
3. Получить существующий ресурс
4. Обновить ресурс целиком

Подсказка: GET — чтение, POST — создание, PUT — замена, DELETE — удаление.

- **GET** — Получить существующий ресурс
- **POST** — Создать новый ресурс
- **PUT** — Обновить ресурс целиком
- **DELETE** — Удалить ресурс

Шаг 07:

Миссия 7

 Сопоставьте шаги обработки ответа API в PL/SQL с их задачами.

Шаги:

Ответ:

- PARSE_JSON
- VALIDATE_SCHEMA
- MAP_FIELDS
- INSERT_INTO_TABLE

Назначение:

1. Проверка, что все нужные поля присутствуют
2. Разбор ответа JSON в структуры Oracle
3. Сопоставление полей внешнего и внутреннего формата
4. Запись данных в целевую таблицу

Подсказка: Обработка запроса идёт в логическом порядке: распарсить, проверить, сопоставить, записать.

- **PARSE_JSON** — Разбор ответа JSON в структуры Oracle
- **VALIDATE_SCHEMA** — Проверка, что все нужные поля присутствуют
- **MAP_FIELDS** — Сопоставление полей внешнего и внутреннего формата
- **INSERT_INTO_TABLE** — Запись данных в целевую таблицу

Шаг 08:

Миссия 8

 Сопоставьте виды ошибок при интеграции API с мерами обработки.

Ошибка:

Ответ:

- HTTP 5xx
- HTTP 4xx
- Timeout
- Непарсируемый JSON

Обработка:

1. Ретрай через некоторое время
2. Логирование и уведомление разработчика
3. Игнорировать и продолжить (если поле необязательное)
4. Отбрасывать запрос с ошибкой клиентских данных

Подсказка: 4xx — ошибка клиента, 5xx — сервер, timeout — нестабильность связи, JSON — формат.

- **HTTP 5xx** — Ретрай через некоторое время
- **HTTP 4xx** — Отбрасывать запрос с ошибкой клиентских данных
- **Timeout** — Логирование и уведомление разработчика
- **Непарсируемый JSON** — Игнорировать и продолжить (если поле необязательное)

 Шаг 09:**Миссия 9**

Расположите шаги жизненного цикла запроса к API и обработки:

1. Отправка HTTP-запроса
2. Преобразование JSON в внутреннюю структуру
3. Проверка схемы и валидация полей
4. Запись данных в базу

Подсказка: Следуйте потоку: запрос → преобразование → проверка → сохранение.

Ответ:

- Отправка HTTP-запроса
- Преобразование JSON в внутреннюю структура
- Проверка схемы и валидация полей
- Запись данных в базу

 Шаг 10:**Миссия 10**

Расположите шаги обработки пагинации API:

1. Сделать первый запрос с offset или page=1
2. Проверить, есть ли следующий токен/страница
3. Отправить следующий запрос с токеном/номером страницы
4. Объединить результаты и продолжить до конца

Подсказка: Пагинация требует цикла запросов до последней страницы.

Ответ:

- Сделать первый запрос с offset или page=1
- Проверить, есть ли следующий токен/страница
- Отправить следующий запрос с токеном/номером страницы

- Объединить результаты и продолжить до конца

Шаг 11:

Миссия 11

Расположите шаги кеширования результатов API чтобы снизить нагрузку:

1. Проверить, есть ли кешированный ответ, не просрочен ли
2. **Если есть** — вернуть из кеша
3. **Если нет** — выполнить запрос к API
4. Сохранить результат в кеш с TTL

Подсказка: Логика кеширования — сначала проверить, потом использовать, иначе — запрос и кеш.

Ответ:

- Проверить, есть ли кешированный ответ, не просрочен ли
- **Если есть** — вернуть из кеша
- **Если нет** — выполнить запрос к API
- Сохранить результат в кеш с TTL

Шаг 12:

Миссия 12

Расположите шаги при интеграции webhook от стороннего сервиса в Oracle:

1. Получить POST запрос от webhook
2. Проверить подпись / авторизацию
3. Распарсить тело и валидировать структуру
4. Записать данные или обработать событие

Подсказка: Нужно убедиться в безопасности до обработки.

Ответ:

- Получить POST запрос от webhook
- Проверить подпись / авторизацию
- Распарсить тело и валидировать структуру
- Записать данные или обработать событие

Шаг 13:

Миссия 13

Расположите этапы тестирования интеграции с API:

1. Тестировать с моками различных сценариев
2. Тестировать на стенде с настоящим API (только чтение)
3. Тестировать полный сценарий с записью и обратным откатом
4. Нагрузочное тестирование и проверка устойчивости

Подсказка: Идём от простого к сложному: моки → стадиум → полный сценарий → нагрузка.

Ответ:

- Тестировать с моками различных сценариев
- Тестировать на стенде с настоящим API (только чтение)
- Тестировать полный сценарий с записью и обратным откатом

- Нагрузочное тестирование и проверка устойчивости

Шаг 14:

Миссия 14

Расположите шаги управления версионированием API в Oracle-слое:

1. Определить версии API и контракт
2. Добавить флаг версии к вызовам
3. **Управлять старой версии (deprecated)** — оставить поддержку временно
4. Постепенно отключать deprecated версии после миграции клиентов

Подсказка: Поддержка версий требует аккуратности при отключении.

Ответ:

- Определить версии API и контракт
- Добавить флаг версии к вызовам
- **Управлять старой версии (deprecated)** — оставить поддержку временно
- Постепенно отключать deprecated версии после миграции клиентов

Шаг 15:

Миссия 15

Расположите этапы масштабирования интеграции API под высокую нагрузку:

1. Добавить очереди (message queue) между Oracle и API
2. Пакетировать запросы в батчи
3. Кэшировать ответы и уменьшать частоту запросов
4. Балансировать нагрузку и использовать асинхронные вызовы

Подсказка: Масштабирование — сочетание асинхронности, кешей и балансировки.

Ответ:

- Добавить очереди (message queue) между Oracle и API
- Пакетировать запросы в батчи
- Кэшировать ответы и уменьшать частоту запросов
- Балансировать нагрузку и использовать асинхронные вызовы

Шаг 16:

Миссия 16

Расположите стратегические шаги при переходе к микросервисной архитектуре с API-шинами:

1. Определить домены и границы сервисов
2. Создать слой API Gateway / шину
3. Мигрировать вызовы внешних API через шину через фасады
4. Мониторинг, логирование и трассировка запросов через шину

Подсказка: Архитектура требует постепенного перехода, не сразу всё перевести.

Ответ:

- Определить домены и границы сервисов
- Создать слой API Gateway / шину
- Мигрировать вызовы внешних API через шину через фасады
- Мониторинг, логирование и трассировка запросов через шину

Урок 2 - Использование Oracle Data Guard для резервного копирования

Шаг 01:

Миссия 1

Вы являетесь DBA на крупном проекте, и команда спрашивает: что такое Oracle Data Guard?

Подсказка: Это компонент Oracle, связанный с резервированием и отказоустойчивостью.

Ответ:

- **Средство репликации и резервирования баз данных Oracle**
- Графический интерфейс для Oracle
- Язык программирования для PL/SQL
- Сторонний инструмент для бэкапов файловой системы

Шаг 02:

Миссия 2

У вас два сервера: primary и standby. Что делает Data Guard в режиме physical standby?

Подсказка: Подумайте о блок-уровне, не логическом.

Ответ:

- **Физическое копирование данных из redo логов на standby**
- Только экспорт/импорт таблиц между серверами
- Синхронизация через FTP
- Запрос-ответ через веб-сервис

Шаг 03:

Миссия 3

Когда вы переводите standby в режим Far Sync, какая основная цель?

Подсказка: Это промежуточный узел между primary и standby.

Ответ:

- **Уменьшить задержку передачи redo на удалённый standby**
- Запустить архивирование данных
- Замена primary
- Запуск логической репликации

Шаг 04:

Миссия 5

Какой параметр определяет режим работы Data Guard protection mode?

Подсказка: Это параметр в initialization. Обычно называется DataGuard_*

Ответ:

- **DG_BROKER_CONFIG_FILE1**
- REMOTE_ARCHIVE_DIR
- LOG_ARCHIVE_DEST_1
- FSARCHIVE_DEST

Шаг 05:

Миссия 6

Сопоставьте роли компонентов Data Guard с их описаниями.

Подсказка: Подумайте, кто читает и применяет redo, а кто служит промежуточным.

Роли:

Ответ:

- Primary
- Standby
- Far Sync
- Observer (FAL)

Описания:

1. Главная база, генерирующая redo и обслуживающая клиентов
 2. Принимает redo и применяется для failover
 3. Промежуточная нода, пересылающая redo далее
 4. Наблюдатель, осуществляющий автоматическое переключение
- **Primary** — Главная база, генерирующая redo и обслуживающая клиентов
 - **Standby** — Принимает redo и применяется для failover
 - **Far Sync** — Промежуточная нода, пересылающая redo далее
 - **Observer (FAL)** — Наблюдатель, осуществляющий автоматическое переключение

Шаг 06:

Миссия 7

Сравните режимы защиты Data Guard с их характеристиками.

Подсказка: Один режим жертвует производительностью ради надёжности.

Режимы защиты:

Ответ:

- Maximum Protection
- Maximum Availability
- Maximum Performance

Характеристики:

1. Синхронный, при котором primary не работает если standby не доступен
 2. Синхронный, допускающий работу primary при кратковременном сбое standby
 3. Асинхронный режим с минимальной задержкой для primary
- **Maximum Protection** — Синхронный, при котором primary не работает если standby не доступен
 - **Maximum Availability** — Синхронный, допускающий работу primary при кратковременном сбое standby
 - **Maximum Performance** — Асинхронный режим с минимальной задержкой для primary

Шаг 07:

Миссия 8

Свяжите команды SQL*Plus и их роли в настройке Data Guard.

Подсказка: Некоторые команды активируют, другие проверяют статус.

Команды:

Ответ:

- ALTER DATABASE COMMIT TO SWITCHOVER TO PHYSICAL STANDBY;
- SELECT SWITCHOVER_STATUS FROM V\$DATABASE;
- ALTER DATABASE RECOVER MANAGED STANDBY DATABASE;
- ALTER DATABASE ACTIVATE STANDBY DATABASE;

Назначения:

1. Запустить сдвиг ролей между primary и standby
 2. Показать, готова ли база к switchover
 3. Запустить standby в режим получения redo
 4. Преобразовать standby в primary после failover или активации
- **ALTER DATABASE COMMIT TO SWITCHOVER TO PHYSICAL STANDBY;** — Запустить сдвиг ролей между primary и standby
 - **SELECT SWITCHOVER_STATUS FROM V\$DATABASE;** — Показать, готова ли база к switchover
 - **ALTER DATABASE RECOVER MANAGED STANDBY DATABASE;** — Запустить standby в режим получения redo
 - **ALTER DATABASE ACTIVATE STANDBY DATABASE;** — Преобразовать standby в primary после failover или активации

 Шаг 08:

Миссия 9

Соотнесите типы переключения с их особенностями.

Подсказка: Один безопасный, другой — аварийный.

Типы:

Ответ:

- Switchover
- Failover
- Flashback Database
- Fast-Start Failover

Особенности:

1. Плановое переключение ролей между primary и standby
 2. Автоматическое переключение при отказе primary
 3. Откат базы к предыдущему моменту после ошибки
 4. Наблюдатель инициирует быстрый фейловер
- **Switchover** — Плановое переключение ролей между primary и standby
 - **Failover** — Автоматическое переключение при отказе primary
 - **Flashback Database** — Откат базы к предыдущему моменту после ошибки
 - **Fast-Start Failover** — Наблюдатель инициирует быстрый фейловер

 Шаг 09:

Миссия 10

Расположите этапы создания физического standby от начального до конечного:

Подсказка: Сначала резервная копия, затем настройка связи, потом старт Apply.

1. Настроить initialization и параметры DG

2. Создать резервную копию primary и восстановить на standby
3. Запустить Managed Recovery на standby
4. Установить сетевые связи и логические маршруты redo

Ответ:

- Настроить initialization и параметры DG
- Создать резервную копию primary и восстановить на standby
- Запустить Managed Recovery на standby
- Установить сетевые связи и логические маршруты redo

 Шаг 10:

Миссия 11

У инженера вышел сбой сети между primary и standby. Как действовать в порядке приоритета?
Подсказка: Сначала выявить проблему, затем переключиться, потом восстановить соединение.

1. Проверить статус сети и линки
2. Инициировать switchover или failover при необходимости
3. Восстановить связь redo и проверку лога
4. Возвратить роль сервера (если нужно) после восстановления сети

Ответ:

- Проверить статус сети и линки
- Инициировать switchover или failover при необходимости
- Восстановить связь redo и проверку лога
- Возвратить роль сервера (если нужно) после восстановления сети

 Шаг 11:

Миссия 12

При промахе резервной копии standby, восстановление через backup: упорядочите шаги:
Подсказка: Сначала восстановление, потом Apply, затем синхронизация.

1. Восстановить контрольные файлы и данные из бэкапа
2. Запустить Managed Recovery
3. Применить недостающие redo
4. Убедиться, что standby синхронизована с primary

Ответ:

- Восстановить контрольные файлы и данные из бэкапа
- Запустить Managed Recovery
- Применить недостающие redo
- Убедиться, что standby синхронизована с primary

 Шаг 12:

Миссия 13

Ваша задача: восстановить после failover. Расположите последовательность действий:
Подсказка: Сначала activate, потом открытие, потом вернуть конфигурацию DG.

1. На бывшем standby выполнить ALTER DATABASE ACTIVATE STANDBY DATABASE
2. Открыть базу в режиме read/write

3. Пересоздать конфигурацию Data Guard (if needed)
4. Проверить логи и применить необработанные redo

Ответ:

- На бывшем standby выполнить ALTER DATABASE ACTIVATE STANDBY DATABASE
- Открыть базу в режиме read/write
- Пересоздать конфигурацию Data Guard (if needed)
- Проверить логи и применить необработанные redo

 Шаг 13:

Миссия 14

У вас стоит архитектура с primary → Far Sync → remote standby. Упорядочите передачу redo:

Подсказка: Redo идёт от primary к Far Sync, оттуда к standby.

1. Primary отправляет redo в Far Sync
2. Far Sync пересылает redo дальше
3. Remote Standby получает redo
4. Standby применяет redo

Ответ:

- Primary отправляет redo в Far Sync
- Far Sync пересылает redo дальше
- Remote Standby получает redo
- Standby применяет redo

 Шаг 14:

Миссия 15

Теперь вы хотите протестировать failover в тестовом стенде. Упорядочите шаги:

Подсказка: Иницируйте failover, затем проверьте целостность, затем восстановите конфигурацию.

1. Инициировать failover вручную на standby
2. Проверить целостность данных и логи
3. Вернуть роли или пересоздать конфигурацию DG
4. Запустить мониторинг и alert'ы

Ответ:

- Инициировать failover вручную на standby
- Проверить целостность данных и логи
- Вернуть роли или пересоздать конфигурацию DG
- Запустить мониторинг и alert'ы

 Урок 3 - Работа с SOAP и REST API в Oracle

 Шаг 01:

Миссия 1

Ваша команда разработчиков интегрирует Oracle с внешним веб-сервисом через SOAP. Какой формат обычно используется в запросе SOAP?

Подсказка: Подумайте об XML-сообщении с обёрткой envelope.

Ответ:

- **XML (SOAP envelope)**
- JSON
- CSV
- YAML

Шаг 02:

Миссия 2

Вы пишете хранимую процедуру PL/SQL, которая вызывает REST-API через UTL_HTTP. Какую HTTP-метод вы выберете для получения данных?

Подсказка: Обычно используется метод, который не изменяет ресурс.

Ответ:

- **GET**
- POST
- PUT
- DELETE

Шаг 03:

Миссия 3

Коллега заявил, что REST лучше SOAP, потому что “легче читать”. Что вы ответите, если учитывать интеграцию с Oracle?

Подсказка: В Oracle есть встроенная поддержка XML DB и SOAP среды.

Ответ:

- **SOAP может быть удобнее в Oracle благодаря встроенной XML/WS поддержки**
- REST всегда медленнее SOAP
- REST не поддерживается в Oracle
- SOAP не поддерживает JSON

Шаг 04:

Миссия 4

 Сопоставьте заголовки HTTP с их смыслом при работе с REST-API через PL/SQL UTL_HTTP.

Подсказка: Один заголовок сообщает формат тела, другой — авторизацию.

Заголовки:

Ответ:

- Content-Type
- Accept
- Authorization
- Cache-Control

Роли:

1. Указывает, какой формат тела запроса
 2. Указывает, какой формат клиент ожидает в ответе
 3. Передаёт токен доступа или учетные данные
 4. Управляет кэшированием ответа
- **Content-Type** — Указывает, какой формат тела запроса

- **Accept** — Указывает, какой формат клиент ожидает в ответе
- **Authorization** — Передаёт токен доступа или учетные данные
- **Cache-Control** — Управляет кэшированием ответа

🔧 Шаг 05:

Миссия 5

📄 Сопоставьте код ответа HTTP с его значением при обращении к REST API.

Подсказка: 200 — ОК, 404 — не найдено.

Коды:

Ответ:

- 200
- 201
- 400
- 401

Значения:

1. Успех, ресурс получен
 2. Создан новый ресурс
 3. Плохой запрос (bad request)
 4. Не авторизован
- **200** — Успех, ресурс получен
 - **201** — Создан новый ресурс
 - **400** — Плохой запрос (bad request)
 - **401** — Не авторизован

🔧 Шаг 06:

Миссия 6

📄 Сопоставьте методы HTTP с их семантикой.

Подсказка: GET — читать, POST — создавать.

Методы:

Ответ:

- GET
- POST
- PUT
- DELETE

Семантики:

1. Получить существующий ресурс
 2. Создать новый ресурс
 3. Обновить или заменить ресурс
 4. Удалить ресурс
- **GET** — Получить существующий ресурс
 - **POST** — Создать новый ресурс
 - **PUT** — Обновить или заменить ресурс
 - **DELETE** — Удалить ресурс

Шаг 07:

Миссия 7

 В команде возник спор: REST API использует CRUD-операции, SOAP — методы. Сопоставьте операции и методы SOAP (пример):

Подсказка: Подумайте, что метод `getCustomer` соответствует чтению.

CRUD:

Ответ:

- Create
- Read
- Update
- Delete

SOAP методы:

1. `createOrder`
 2. `getOrder`
 3. `updateOrder`
 4. `deleteOrder`
- **Create** — `createOrder`
 - **Read** — `getOrder`
 - **Update** — `updateOrder`
 - **Delete** — `deleteOrder`

Шаг 08:

Миссия 8

У инженера Иванова задача: отправить POST-запрос через PL/SQL. Расположите шаги в правильном порядке:

Подсказка: Сначала создать `request`, затем установить заголовки, затем отправить, затем прочитать ответ.

1. `utl_http.begin_request`
2. `utl_http.set_header` (Content-Type и др.)
3. `utl_http.write_text` / `write_line` с телом запроса
4. `utl_http.get_response` и чтение тела ответа

Ответ:

- `utl_http.begin_request`
- `utl_http.set_header` (Content-Type и др.)
- `utl_http.write_text` / `write_line` с телом запроса
- `utl_http.get_response` и чтение тела ответа

Шаг 09:

Миссия 9

При ошибке 401 при вызове REST API из Oracle, как должен действовать алгоритм?

Подсказка: Попробуйте обновить токен, затем повторить запрос.

1. Получить новый токен (`refresh token` или `login`)
2. Повторить запрос с новым токеном

3. **Если снова 401** — поднять ошибку и логировать
4. Обработать успешный ответ или ошибку

Ответ:

- Получить новый токен (refresh token или login)
- Повторить запрос с новым токеном
- **Если снова 401** — поднять ошибку и логировать
- Обработать успешный ответ или ошибку

 Шаг 10:

Миссия 10

Ваша команда строит цепочку: PHP → Oracle → внешний SOAP-сервис → Oracle. Упорядочите flow обработки запроса:

Подсказка: PHP вызывает хранимую процедуру, она вызывает SOAP, получает ответ и отдаёт назад.

1. PHP вызывает PL/SQL процедуру через DB API
2. PL/SQL делает UTL_HTTP вызов к SOAP-сервису
3. PL/SQL получает ответ и парсит XML
4. PL/SQL возвращает результат в PHP

Ответ:

- PHP вызывает PL/SQL процедуру через DB API
- PL/SQL делает UTL_HTTP вызов к SOAP-сервису
- PL/SQL получает ответ и парсит XML
- PL/SQL возвращает результат в PHP

 Шаг 11:

Миссия 11

В проекте нужно последовательно отправить два REST вызова: fetch token, затем получить ресурс. Упорядочите шаги:

Подсказка: Сначала получить токен, затем использовать его.

1. utl_http.begin_request для token endpoint
2. utl_http.get_response и чтение токена
3. utl_http.begin_request к целевому REST API с токеном в заголовке
4. utl_http.get_response и чтение результата

Ответ:

- utl_http.begin_request для token endpoint
- utl_http.get_response и чтение токена
- utl_http.begin_request к целевому REST API с токеном в заголовке
- utl_http.get_response и чтение результата

 Шаг 12:

Миссия 12

После получения XML ответа от SOAP-сервиса нужно обработать result и faults. Упорядочите этапы:

Подсказка: Сначала проверить наличие fault, потом разобрать тело.

1. Проверить наличие тега в ответе
2. **Если есть fault** — вернуть ошибку клиенту
3. **Если нет fault** — распарсить тело ответа
4. Извлечь нужные данные и вернуть

Ответ:

- Проверить наличие тега в ответе
- **Если есть fault** — вернуть ошибку клиенту
- **Если нет fault** — распарсить тело ответа
- Извлечь нужные данные и вернуть

Шаг 13:

Миссия 13

В задаче интеграции REST вы хотите реализовать retry-механику при таймаутах. Упорядочите логику:

Подсказка: Попробовать, если timeout → ждать, повторить, если несколько попыток — ошибку.

1. Попытаться выполнить запрос
2. **Если получен timeout** — подождать backoff
3. Повторить запрос (до N попыток)
4. **Если все попытки неудачны** — вернуть ошибку

Ответ:

- Попытаться выполнить запрос
- **Если получен timeout** — подождать backoff
- Повторить запрос (до N попыток)
- **Если все попытки неудачны** — вернуть ошибку

Шаг 14:

Миссия 14

Ваша архитектура: PHP → Oracle → REST → SOAP → REST. Упорядочите полный поток запроса и ответа:

Подсказка: Подумайте о переходах Flask → PL/SQL → внешние сервисы → PL/SQL → PHP.

1. PHP отправляет входящий REST запрос к API вашего приложения
2. PHP вызывает Oracle PL/SQL процедуру с параметрами
3. PL/SQL вызывает внешний REST API через UTL_HTTP
4. Внутри внешнего REST вызывается SOAP веб-сервис (внешний компонент)
5. SOAP возвращает ответ к REST внешнему
6. PL/SQL принимает и обрабатывает ответ, возвращая структуру в PHP
7. PHP формирует ответ JSON и отдает клиенту

Ответ:

- PHP отправляет входящий REST запрос к API вашего приложения
- PHP вызывает Oracle PL/SQL процедуру с параметрами
- PL/SQL вызывает внешний REST API через UTL_HTTP
- Внутри внешнего REST вызывается SOAP веб-сервис (внешний компонент)
- SOAP возвращает ответ к REST внешнему

- PL/SQL принимает и обрабатывает ответ, возвращая структуру в PHP
- PHP формирует ответ JSON и отдает клиенту

Шаг 15:

Миссия 15

На продакшене вы получаете ошибку HTTP 429 (Too Many Requests) от REST-сервиса. Как встроить защиту? Упорядочите шаги:

Подсказка: Вводим лимит, ждём, повторяем.

1. Поймать код 429 в PL/SQL или PHP
2. Подождать указанный в заголовке Retry-After или backoff
3. Повторить запрос (если не превышен максимальный retry)
4. **Если лимит исчерпан** — вернуть ошибки клиенту и логировать

Ответ:

- Поймать код 429 в PL/SQL или PHP
- Подождать указанный в заголовке Retry-After или backoff
- Повторить запрос (если не превышен максимальный retry)
- **Если лимит исчерпан** — вернуть ошибки клиенту и логировать

Шаг 16:

Миссия 16

Вы хотите реализовать схему резервного вызова: если SOAP-сервис недоступен, пытаться REST-альтернативу. Упорядочите логику обращения:

Подсказка: Попробовать SOAP, если ошибка — переключиться на REST.

1. Попытаться вызвать SOAP-метод через PL/SQL
2. **Если SOAP возвращает ошибку (timeout или fault)** — логировать её
3. Инициировать вызов REST альтернативы с теми же параметрами
4. Вернуть первый успешный ответ клиенту или ошибку, если оба не сработали

Ответ:

- Попытаться вызвать SOAP-метод через PL/SQL
- **Если SOAP возвращает ошибку (timeout или fault)** — логировать её
- Инициировать вызов REST альтернативы с теми же параметрами
- Вернуть первый успешный ответ клиенту или ошибку, если оба не сработали

Урок 4 - Использование Oracle Streams для синхронизации данных

Шаг 01:

Миссия 1

Вы — разработчик крупной системы, и вас попросили внедрить Oracle Streams. Что такое Oracle Streams?

Подсказка: Это средство обмена данными и репликации в Oracle.

Ответ:

- **Механизм обмена данными, репликации и публикации/подписки внутри Oracle**
- Инструмент резервного копирования
- Веб-служба для интеграции PHP с Oracle
- Пакет мониторинга производительности Oracle

Шаг 02:

Миссия 2

Ваша команда хочет настроить публикацию изменений таблицы A из базы Oracle в другую базу. Какой объект Streams нужен для отправки изменений?

Подсказка: Публикация — «source» сторона.

Ответ:

- **Source-database capture / source queue / outbound server**
- Stream subscriber module
- LogMiner only
- Materialized view refresh job

Шаг 03:

Миссия 3

При настройке Streams вы слышите о Capture и Apply процессы. Что делает Capture?

Подсказка: Это читает изменения из redo/archivelog.

Ответ:

- **Извлекает изменения DML из redo/логов и помещает их в очередь Streams**
- Применяет изменения к подписчикам
- Удаляет старые журналы
- Синхронизирует Snapshot view

Шаг 04:

Миссия 4

Что делает Apply процесс в Oracle Streams?

Подсказка: Это на стороне подписчиков.

Ответ:

- **Берёт сообщения из очереди Streams и применяет DML к подписчикам**
- Копирует данные в внутренний буфер
- Удаляет таблицы на стороне подписчика
- Управляет сетевыми соединениями

Шаг 05:

Миссия 5

Вы хотите ограничить, какие столбцы публикуются из таблицы. Как называется механизм фильтрации в Streams?

Подсказка: Часто используется при частичной репликации.

Ответ:

- **Column filter / column transformation**
- Row suppression only
- Protocol filter
- Network filter

Шаг 06:

Миссия 6

 Сопоставьте ключевые компоненты Oracle Streams с их ролями.

Подсказка: Подумайте, что отправляет, что принимает, что хранит очередь.

Компоненты:

Ответ:

- Capture
- Propagation
- Apply
- Staging queue / Streams queue

Роли:

1. Читает изменения из логов и помещает их в очередь
 2. Пересылает сообщения из источника к подписчику через сеть
 3. Применяет изменения к целевой базе данных
 4. Хранит сообщения между этапами обработки
- **Capture** — Читает изменения из логов и помещает их в очередь
 - **Propagation** — Пересылает сообщения из источника к подписчику через сеть
 - **Apply** — Применяет изменения к целевой базе данных
 - **Staging queue / Streams queue** — Хранит сообщения между этапами обработки

Шаг 07:

Миссия 7

 Сопоставьте типы трансформаций данных в Streams с их описаниями.

Подсказка: Трансформации могут модифицировать, фильтровать или преобразовывать.

Тип трансформации:

Ответ:

- Column transformation
- Row filter
- Message transformation
- Expression filter

Описание:

1. Изменение значения отдельного столбца перед публикацией
 2. Отбрасывает целые строки по условию
 3. Модифицирует структуру сообщения или атрибуты Streams
 4. Фильтр, основанный на выражении SQL, применяемый к строкам
- **Column transformation** — Изменение значения отдельного столбца перед публикацией
 - **Row filter** — Отбрасывает целые строки по условию
 - **Message transformation** — Модифицирует структуру сообщения или атрибуты Streams
 - **Expression filter** — Фильтр, основанный на выражении SQL, применяемый к строкам

Шаг 08:

Миссия 8

 Соотнесите статус очереди Streams с её значением при синхронизации.

Подсказка: Некоторые очереди могут быть активны, другие приостановлены.

Статусы:

Ответ:

- ENABLED
- DISABLED
- NEEDS ATTENTION
- ERROR

Значения:

1. Очередь работает и принимает данные
 2. Очередь отключена, не принимает сообщения
 3. Очередь требует проверки администратором
 4. Очередь в состоянии ошибки, нужно вмешательство
- **ENABLED** — Очередь работает и принимает данные
 - **DISABLED** — Очередь отключена, не принимает сообщения
 - **NEEDS ATTENTION** — Очередь требует проверки администратором
 - **ERROR** — Очередь в состоянии ошибки, нужно вмешательство

 Шаг 09:

Миссия 9

 Соотнесите виды репликации Streams с их характеристиками.

Подсказка: Streams может работать как многопоточная или однонаправленная репликация.

Виды:

Ответ:

- Unidirectional replication
- Bidirectional replication
- Multi-source replication
- Peer-to-peer replication

Характеристики:

1. Из одного источника в одну цель
 2. Из двух или более источников в одну цель
 3. Двусторонний обмен между двумя базами
 4. Репликация между равноправными узлами (каждый может быть source и target)
- **Unidirectional replication** — Из одного источника в одну цель
 - **Multi-source replication** — Из двух или более источников в одну цель
 - **Bidirectional replication** — Двусторонний обмен между двумя базами
 - **Peer-to-peer replication** — Репликация между равноправными узлами (каждый может быть source и target)

 Шаг 10:

Миссия 10

Инженер Павел хочет настроить Streams от начальной установки до работы. Упорядочите шаги:

Подсказка: Сначала настроить очередь и capture, затем propagation и apply.

1. Создать очередь Streams (staging queue)
2. Запустить процесс Capture на источнике
3. Настроить Propagation между источником и подписчиком
4. Запустить Apply на целевой базе

Ответ:

- Создать очередь Streams (staging queue)
- Запустить процесс Capture на источнике
- Настроить Propagation между источником и подписчиком
- Запустить Apply на целевой базе

 Шаг 11:

Миссия 11

В продакшне одна из очередей переключилась в ERROR. Что делать в порядке шагов?

Подсказка: Сначала диагностировать, потом починить, затем возобновить.

1. Проверить логи и трассировку ошибок Streams
2. Исправить причину (например, сбой связи или ошибка данных)
3. Перевести очередь в состояние ENABLED или RESTART
4. Мониторить, что очередь вновь работает в нормальном режиме

Ответ:

- Проверить логи и трассировку ошибок Streams
- Исправить причину (например, сбой связи или ошибка данных)
- Перевести очередь в состояние ENABLED или RESTART
- Мониторить, что очередь вновь работает в нормальном режиме

 Шаг 12:

Миссия 12

Вы хотите выполнить graceful switchover реплики Streams. Какой порядок действий?

Подсказка: Переключайтесь, чтобы минимизировать потерю данных.

1. Остановить внесение новых изменений на текущем source
2. Дождаться, пока все сообщения будут propagated и applied
3. Переключить роли: source → target
4. Запустить новый capture / apply на новой стороне

Ответ:

- Остановить внесение новых изменений на текущем source
- Дождаться, пока все сообщения будут propagated и applied
- Переключить роли: source → target
- Запустить новый capture / apply на новой стороне

 Шаг 13:

Миссия 13

Вы хотите внедрить fallback: если Streams недоступен, использовать PL/SQL скрипт. Как построить логику?

Подсказка: Попробуйте Streams, если ошибка — запустить запасной путь.

1. Попыаться опубликовать через Streams API
2. Если публикация возвращает ошибку (очередь, сеть, статус), логировать её
3. Запустить PL/SQL процедуру альтернативной синхронизации (batch)
4. Вернуть результат клиенту или ошибку, если оба пути не сработали

Ответ:

- Попыаться опубликовать через Streams API
- Если публикация возвращает ошибку (очередь, сеть, статус), логировать её
- Запустить PL/SQL процедуру альтернативной синхронизации (batch)
- Вернуть результат клиенту или ошибку, если оба пути не сработали

 Шаг 14:

Миссия 14

В сценарии с многократным источником (multi-source replication) нужно объединить потоки. Упорядочите обработку:

Подсказка: Сначала propagate, затем transform, затем apply, затем детектировать конфликты.

1. Propagate сообщения от источников в целевую очередь
2. Transform / merge сообщений (объединение данных)
3. Apply данных в целевую базу
4. Обработать конфликты (conflict resolution)

Ответ:

- Propagate сообщения от источников в целевую очередь
- Transform / merge сообщений (объединение данных)
- Apply данных в целевую базу
- Обработать конфликты (conflict resolution)

 Шаг 15:

Миссия 15

Вы хотите обеспечить консистентность при запуске Streams после простоя. Как восстановить синхронизацию?

Подсказка: Нужно пропустить изменения, применить пропагейт, запустить apply.

1. Запустить Propagation для незатреканых сообщений (catchup)
2. Запустить Apply, чтобы применить накопленные изменения
3. Проверить, что очереди снова ENABLED и синхронизированы
4. Провести контрольные проверки целостности данных

Ответ:

- Запустить Propagation для незатреканых сообщений (catchup)
- Запустить Apply, чтобы применить накопленные изменения
- Проверить, что очереди снова ENABLED и синхронизированы
- Провести контрольные проверки целостности данных

 Шаг 16:

Миссия 16

В проекте решили переключаться между Streams и materialized views. Какой порядок действий при переключении?

Подсказка: Остановить Streams, синхронизировать MV, затем включить MV.

1. Отключить Capture / Apply на Streams
2. Синхронизировать данные в materialized view (fast refresh или полное обновление)
3. Переключить приложения на чтение из MV

4. Отключить подписки Streams / удалить очередь, если не нужна

Ответ:

- Отключить Capture / Apply на Streams
- Синхронизировать данные в materialized view (fast refresh или полное обновление)
- Переключить приложения на чтение из MV
- Отключить подписки Streams / удалить очередь, если не нужна

Урок 5 - Интеграция с другими СУБД с помощью Oracle Gateway

Шаг 01:

Миссия 1

Вы выступаете архитектором, и команда просит: что такое Oracle Gateway в контексте интеграции с другими СУБД?

Подсказка: Это компонент Oracle, который позволяет обращаться к удалённым базам через Oracle как к локальным таблицам.

Ответ:

- **Механизм, позволяющий Oracle обращаться к внешним СУБД как к удалённым схемам**
- Инструмент резервного копирования других СУБД
- Утилита миграции схем из MySQL в Oracle
- Графический интерфейс для управления внешними БД

Шаг 02:

Миссия 2

Ваша задача: подключить базу MySQL к Oracle через Gateway. Какой тип gateway наиболее вероятен?

Подсказка: Обычно для MySQL используют ODBC-провайдер.

Ответ:

- **Generic ODBC Gateway (DG4ODBC) / Heterogeneous Services**
- Oracle Transparent Gateway for SQL Server
- Oracle Gateway for DB2 LUW
- Oracle Gateway for Sybase

Шаг 03:

Миссия 4

Ваша команда протестировала соединение gateway, и при SELECT над внешней табличной именем вы получили ORA-02085 / PLS-306 ошибку. Что вы проверите первым?

Подсказка: Проверьте настройки сетевых служб и имя службы HS.

Ответ:

- **tnsnames.ora и listener для HS: SID / сервис имени**
- Версию MySQL клиента
- Размер буфера PGA в Oracle
- Параметр NLS_LANG для Oracle

Шаг 04:

Миссия 5

 Сопоставьте конфигурационные файлы/объекты Gateway с их назначениями.

Подсказка: Один задаёт DSN, другой – маршруты, третий – внешний доступ.

Компоненты:

Ответ:

- tnsnames.ora (entry for HS)
- listener.ora (для HS)
- init SID.ora (HS init файл)
- tns entry в Oracle schemas (database link)

Назначения:

1. Указывает клиенту/Oracle, как подключаться к HS listener
2. Настраивает сам listener HS для внешней СУБД
3. Определяет параметры и драйвер ODBC/Драйвера внешней СУБД
4. Указывает, как Oracle-клиент обращается к внешней СУБД через DB link

- **tnsnames.ora (entry for HS)** — Указывает клиенту/Oracle, как подключаться к HS listener
- **listener.ora (для HS)** — Настраивает сам listener HS для внешней СУБД
- **init SID.ora (HS init файл)** — Определяет параметры и драйвер ODBC/Драйвера внешней СУБД
- **tns entry в Oracle schemas (database link)** — Указывает, как Oracle-клиент обращается к внешней СУБД через DB link

Шаг 05:

Миссия 6

 Сопоставьте типы gateway с наиболее подходящей внешней СУБД.

Подсказка: Некоторые gateway специализированы.

Тип gateway:

Ответ:

- DG4ODBC (Generic ODBC Gateway)
- Oracle Transparent Gateway for SQL Server
- Oracle Gateway for DB2
- Oracle Gateway for Sybase

СУБД:

1. PostgreSQL, MySQL, любой ODBC-совместимый источник
2. Microsoft SQL Server
3. IBM DB2 LUW/UCDB2
4. Sybase ASE / OpenSybase

- **DG4ODBC (Generic ODBC Gateway)** — PostgreSQL, MySQL, любой ODBC-совместимый источник
- **Oracle Transparent Gateway for SQL Server** — Microsoft SQL Server
- **Oracle Gateway for DB2** — IBM DB2 LUW/UCDB2
- **Oracle Gateway for Sybase** — Sybase ASE / OpenSybase

Шаг 06:

Миссия 7

 Сопоставьте сообщения об ошибках gateway с возможными причинами.

Подсказка: ORA-02085 часто связан с неправильным алиасом.

Код ошибки:

Ответ:

- ORA-02085
- ORA-HS-ODBC error: “[unable to connect]”
- ORA-HS-ODBC error: “Data source name not found”
- ORA-HS-ODBC error: “Login timeout expired”

Причина:

1. Неверный TNS алиас или entry отсутствует
 2. listener для HS не запущен или неверен
 3. DSN (ODBC) не настроен или отсутствует в init файле
 4. Сетевые проблемы или СУБД не отвечает вовремя
- **ORA-02085** — Неверный TNS алиас или entry отсутствует
 - **ORA-HS-ODBC error: “[unable to connect]”** — listener для HS не запущен или неверен
 - **ORA-HS-ODBC error: “Data source name not found”** — DSN (ODBC) не настроен или отсутствует в init файле
 - **ORA-HS-ODBC error: “Login timeout expired”** — Сетевые проблемы или СУБД не отвечает вовремя

Шаг 07:

Миссия 8

 Сопоставьте уровни доступа (privileges) Oracle / Gateway, требуемые для внешних подключений.

Подсказка: Нужно разрешение на создание database link и execute on HS.*

Привилегия:

Ответ:

- CREATE DATABASE LINK
- EXECUTE ANY PROCEDURE on HS schema
- SELECT on public database link table
- GRANT CONNECT on remote via gateway

Назначение:

1. Позволяет создавать DB link к внешним источникам
 2. Позволяет выполнять процедуры внутри схемы HS
 3. Разрешает доступ к таблицам через link
 4. Обеспечивает право подключения через gateway к удалённой СУБД
- **CREATE DATABASE LINK** — Позволяет создавать DB link к внешним источникам
 - **EXECUTE ANY PROCEDURE on HS schema** — Позволяет выполнять процедуры внутри схемы HS
 - **SELECT on public database link table** — Разрешает доступ к таблицам через link
 - **GRANT CONNECT on remote via gateway** — Обеспечивает право подключения через gateway к удалённой СУБД

Шаг 08:

Миссия 9

Вы настраиваете flow запроса к внешней таблице через gateway. Упорядочите шаги:

Подсказка: Сначала TNS, затем listener HS, затем init и link.

1. Добавить запись в tnsnames.ora для HS alias
2. Настроить listener.ora и запустить listener HS
3. Создать init SID.ora с DSN/ODBC параметрами
4. Создать database link в Oracle к внешнему alias

Ответ:

- Добавить запись в tnsnames.ora для HS alias
- Настроить listener.ora и запустить listener HS
- Создать init SID.ora с DSN/ODBC параметрами
- Создать database link в Oracle к внешнему alias

Шаг 09:

Миссия 10

При выполнении INSERT INTO external_table@dblink возникла задержка. Как логически рассчитать порядок диагностики?

Подсказка: Проверьте сеть, настройки ODBC, статус listener, мониторинг.

1. Проверить пинг и сетевое соединение между хостами
2. Проверить, что listener HS слушает и доступен
3. Проверить конфигурацию ODBC / DSN в init файле HS
4. Проверить статистику wait events или trace HS логов

Ответ:

- Проверить пинг и сетевое соединение между хостами
- Проверить, что listener HS слушает и доступен
- Проверить конфигурацию ODBC / DSN в init файле HS
- Проверить статистику wait events или trace HS логов

Шаг 10:

Миссия 11

Ваша база MySQL изменила структуру: добавлен столбец. Как обновить gateway-конфигурацию?

Подсказка: Нужно обновить DSN / ODBC, возможно recompile внешние объекты и refresh dblink.

1. Остановить listener HS или временно отключить link
2. Обновить DSN / ODBC конфигурацию для новой структуры
3. Перекомпилировать внешние объекты / view в Oracle (if applicable)
4. Возобновить listener HS и протестировать SELECT/INSERT через link

Ответ:

- Остановить listener HS или временно отключить link
- Обновить DSN / ODBC конфигурацию для новой структуры
- Перекомпилировать внешние объекты / view в Oracle (if applicable)
- Возобновить listener HS и протестировать SELECT/INSERT через link

Шаг 11:

Миссия 12

Вы планируете failover-механизм: если gateway node недоступен, переключаться на резервный. Какова логика?

Подсказка: Попробовать основной link, если не отвечает — переключиться на резервный link.

1. Попытайтесь выполнить запрос через primary gateway link
2. **Если timeout или ошибка подключения** — задействовать резервный link (failover)
3. Логировать ошибку и переключение
4. Вернуть результат или ошибку клиенту

Ответ:

- Попытайтесь выполнить запрос через primary gateway link
- **Если timeout или ошибка подключения** — задействовать резервный link
- Логировать ошибку и переключение
- Вернуть результат или ошибку клиенту

Шаг 12:

Миссия 13

Когда вы мигрируете данные из PostgreSQL в Oracle через gateway, как выстраивать план миграции?

Подсказка: Сначала структура, потом данные, потом проверка и переключение.

1. Скрипт создания схемы / таблиц в Oracle, соответствующих внешним
2. Параллельная миграция данных через gateway (INSERT ... SELECT)
3. Проверка целостности и согласования данных
4. Переключение приложения на Oracle структуру, возможно деактивация внешнего link

Ответ:

- Скрипт создания схемы / таблиц в Oracle, соответствующих внешним
- Параллельная миграция данных через gateway (INSERT ... SELECT)
- Проверка целостности и согласования данных
- Переключение приложения на Oracle структуру, возможно деактивация внешнего link

Шаг 13:

Миссия 14

Вы хотите уменьшить нагрузку на внешний источник при частых селектах через gateway. Как оптимизировать?

Подсказка: Используйте кеширование, materialized views или частичные выборки.

1. Создать материализованные представления в Oracle, периодически обновляемые
2. Ограничивать выборку через WHERE / row filter в gateway
3. Кешировать результаты запроса в Oracle слое приложения / PL/SQL
4. Периодически очищать кеш / обновлять MV

Ответ:

- Создать материализованные представления в Oracle, периодически обновляемые
- Ограничивать выборку через WHERE / row filter в gateway
- Кешировать результаты запроса в Oracle слое приложения / PL/SQL

- Периодически очищать кеш / обновлять MV

Шаг 14:

Миссия 15

Вы хотите реализовать batch-синхронизацию с внешней СУБД и fallback, если gateway не отвечает. Какова последовательность?

Подсказка: В норме — online, при ошибке — batch.

1. Попробовать выполнить запрос через gateway (онлайн режим)
2. **Если ошибка (timeout, отказ)** — запустить batch-узел синхронизации (ETL через DB link)
3. Логировать инцидент и результаты batch
4. Вернуть ответ или сообщение об ошибке клиенту

Ответ:

- Попробовать выполнить запрос через gateway (онлайн режим)
- **Если ошибка (timeout, отказ)** — запустить batch-узел синхронизации (ETL через DB link)
- Логировать инцидент и результаты batch
- Вернуть ответ или сообщение об ошибке клиенту

Шаг 15:

Миссия 16

При нагрузочном тесте вы фиксируете падение производительности при многих соединениях через gateway. Как оптимально действовать?

Подсказка: Используйте пул соединений, разграничение и кеширование.

1. Внедрить пул соединений к gateway (persistent sessions)
2. Ограничить число одновременных соединений через HS listener
3. Кешировать часто используемые запросы / MV вместо частых обращений
4. Мониторить wait events и узкие места на HS / ODBC стороне

Ответ:

- Внедрить пул соединений к gateway (persistent sessions)
- Ограничить число одновременных соединений через HS listener
- Кешировать часто используемые запросы / MV вместо частых обращений
- Мониторить wait events и узкие места на HS / ODBC стороне

Шаг 16:

Миссия 17

Вы хотите плавно мигрировать нагрузку с gateway на собственные реплики Oracle. Как это сделать поэтапно?

Подсказка: Постепенно уменьшайте зависимость от gateway.

1. Выделить часть запросов, которые можно обслуживать через внутренние реплики Oracle
2. Настроить репликацию / Streams / DG между Oracle и внешним источником, если возможно
3. Постепенно перенести логику приложения на чтение из реплик Oracle
4. Снижать использование gateway, пока полностью не замените его на реплики

Ответ:

- Выделить часть запросов, которые можно обслуживать через внутренние реплики Oracle
- Настроить репликацию / Streams / DG между Oracle и внешним источником, если возможно

- Постепенно перенести логику приложения на чтение из реплик Oracle
- Снижать использование gateway, пока полностью не замените его на реплики

Модуль 6 - Безопасность и управление данными

Урок 1 - Безопасность данных в Oracle права доступа и роли

Шаг 01:

Миссия 1

Вы — DBA крупной компании, и разработчик просит выдать ему права SELECT на таблицу персональных данных сотрудников. Какое решение наиболее безопасно?

Подсказка: Давать максимальные права — не лучший вариант.

Ответ:

- **Выдать только привилегию SELECT на конкретные столбцы, не на всю таблицу**
- Выдать роль DBA
- Выдать права ALL PRIVILEGES

Шаг 02:

Миссия 2

Менеджер хочет, чтобы отчет SQL был выполнен от имени системной учётной записи, чтобы избежать ограничений. Как лучше поступить?

Подсказка: Подумайте о контексте выполнения, а не об обходе защиты.

Ответ:

- **Использовать проху-пользователя или контекст аутентификации с ограничениями**
- Передать менеджеру системный пароль
- Создать процедуру с авторским правом SYS и дать ему EXECUTE

Шаг 03:

Миссия 3

Разработчик просит роль с правом на INSERT/UPDATE/DELETE в схеме журналов. Как грамотно ограничить права?

Подсказка: Права нужно ограничивать по необходимости.

Ответ:

- **Выдать роль только с INSERT и UPDATE на нужные таблицы**
- Выдать роль DELETE по всем таблицам
- Выдать роль FULL DML
- Выдать роль ALL PRIVILEGES

Шаг 04:

Миссия 4

Системный администратор настаивает, что он должен иметь прямой доступ к схемам приложений. Как отреагировать?

Подсказка: Минимальный доступ — лучший путь к безопасности.

Ответ:

- **Выдать только те привилегии, что необходимы для его задач**

- Дать ему роль WITH ADMIN OPTION
- Разрешить доступ ко всему как DBA

🔮 Шаг 05:

Миссия 5

Вы создаёте роль для аналитиков, но хотите, чтобы они не видели конфиденциальные поля (пароли, данные карты). Что выбрать?

Подсказка: В Oracle можно ограничивать привилегии на уровне столбцов.

Ответ:

- **Выдать SELECT только на непроективные столбцы**
- Выдать полный SELECT на всю таблицу
- Спросить аналитиков не запрашивать конфиденциальные поля

🔮 Шаг 06:

Миссия 6

После аудита вы обнаружили, что роль ANALYST имеет избыточные права. Что делать первым шагом?

Подсказка: Принцип «least privilege» — минимальные права.

Ответ:

- **Проанализировать, какие права реально используются и отозвать лишние**
- Удалить роль и выдать права заново
- **Оставить как есть, аудит** — формальность

🔮 Шаг 07:

Миссия 7

Сопоставьте термин Oracle безопасности с его описанием.

Термины:

Ответ:

- ROLE
- PRIVILEGE
- OBJECT PRIVILEGE
- SYSTEM PRIVILEGE

Описание:

1. Привилегия, дающая право выполнять DDL/DCL (например, CREATE USER)
2. Привилегия, дающая право на объекты (таблицы, представления и др.)
3. Набор привилегий, который можно назначать пользователям
4. Общее право на систему (например, ALTER ANY TABLE)

Подсказка: ROLE — это не действие, а контейнер.

- **ROLE** — Набор привилегий, который можно назначать пользователям
- **PRIVILEGE** — Общее право на систему (например, ALTER ANY TABLE)
- **OBJECT PRIVILEGE** — Привилегия, дающая право на объекты (таблицы, представления и др.)
- **SYSTEM PRIVILEGE** — Привилегия, дающая право выполнять DDL/DCL (например, CREATE USER)

Шаг 08:

Миссия 8

Сопоставьте сценарий с механизмом ограничения доступа в Oracle.

Сценарии:

Ответ:

- Ограничение доступа к столбцам
- Скрытие строк (row-level security)
- Контроль привилегий DDL
- Ограничение вызова процедур

Механизмы:

1. Virtual Private Database (VPD) / Fine-Grained Access Control
2. GRANT EXECUTE ON procedure
3. Column-level privileges (GRANT on столбцы)
4. System privileges (например, ALTER ANY TABLE)

Подсказка: Подумайте, что изолирует строки, а что — столбцы.

- **Ограничение доступа к столбцам** — Column-level privileges (GRANT on столбцы)
- **Скрытие строк (row-level security)** — Virtual Private Database (VPD) / Fine-Grained Access Control
- **Контроль привилегий DDL** — System privileges (например, ALTER ANY TABLE)
- **Ограничение вызова процедур** — GRANT EXECUTE ON procedure

Шаг 09:

Миссия 9

Сопоставьте роль с задачей, для которой её стоит создать.

Роли:

Ответ:

- DATA_READER
- DATA_WRITER
- DBA_ASSISTANT
- REPORT_EXECUTOR

Задачи:

1. Роль, которая позволяет выполнять селекты по большинству таблиц
2. Роль, которая даёт INSERT/UPDATE/DELETE в журнальные схемы
3. Роль для поддержки, но без доступа к пользовательским данным
4. Роль, запускающая отчёты, но не меняющая данные

Подсказка: Названия роли подскажут назначение.

- **DATA_READER** — Роль, которая позволяет выполнять селекты по большинству таблиц
- **DATA_WRITER** — Роль, которая даёт INSERT/UPDATE/DELETE в журнальные схемы
- **DBA_ASSISTANT** — Роль для поддержки, но без доступа к пользовательским данным
- **REPORT_EXECUTOR** — Роль, запускающая отчёты, но не меняющая данные

Шаг 10:

Миссия 10

Сопоставьте тип гранта с контекстом его использования.

Типы гранта:

Ответ:

- GRANT ... WITH ADMIN OPTION
- GRANT ... WITH GRANT OPTION
- GRANT ... TO ROLE
- GRANT ... TO USER

Контексты:

1. Позволяет получателю далее делегировать права другим
2. Дает роль пользователю напрямую
3. Дает роль ролям (группам)
4. Позволяет управлять ролью (давать/отнимать роль другим)

Подсказка: WITH ADMIN — для управления ролью; WITH GRANT — для делегирования.

- **GRANT ... WITH ADMIN OPTION** — Позволяет управлять ролью (давать/отнимать роль другим)
- **GRANT ... WITH GRANT OPTION** — Позволяет получателю далее делегировать права другим
- **GRANT ... TO ROLE** — Дает роль ролям (группам)
- **GRANT ... TO USER** — Дает роль пользователю напрямую

Шаг 11:

Миссия 11

Сопоставьте аудит-контроль с его назначением.

Аудит:

Ответ:

- STANDARD AUDIT
- FINE-GRAINED AUDIT (FGA)
- DATABASE AUDIT TRIGGER
- UNIFIED AUDIT (в новых версиях)

Назначения:

1. Полный системный аудит событий (логин, DDL и др.)
2. Аудит по выражению (например, когда value > 1000)
3. Триггер на таблицу/схему, логирующий изменения
4. Совмещённая единая система аудита в Oracle 12c+

Подсказка: FGA — точечные проверки, STANDARD — общий лог.

- **STANDARD AUDIT** — Полный системный аудит событий (логин, DDL и др.)
- **FINE-GRAINED AUDIT (FGA)** — Аудит по выражению (например, когда value > 1000)
- **DATABASE AUDIT TRIGGER** — Триггер на таблицу/схему, логирующий изменения
- **UNIFIED AUDIT (в новых версиях)** — Совмещённая единая система аудита в Oracle 12c+

Шаг 12:

Миссия 12

Расположите шаги процесса выдачи роли безопасно.

1. Проверка бизнес-обоснования для роли
2. Назначение роли пользователю
3. Проверка, какие права уже есть
4. Тестирование, что роль не открывает лишнего доступа

Подсказка: Сначала проверяем, потом выдаём, затем тестируем.

Ответ:

- Проверка бизнес-обоснования для роли
- Проверка, какие права уже есть
- Назначение роли пользователю
- Тестирование, что роль не открывает лишнего доступа

Шаг 13:

Миссия 13

Расположите этапы аудита привилегий от обнаружения до устранения риска.

1. Сбор фактических привилегий (SELECT из словарей)
2. Сравнение с политикой безопасности
3. Идентификация избыточных прав
4. Отзыв лишних привилегий
5. Контроль после изменений (регрессия)

Подсказка: Путь: собрать → сравнить → определить → отозвать → проверить.

Ответ:

- Сбор фактических привилегий (SELECT из словарей)
- Сравнение с политикой безопасности
- Идентификация избыточных прав
- Отзыв лишних привилегий
- Контроль после изменений (регрессия)

Шаг 14:

Миссия 14

Расположите шаги при инциденте: обнаружен доступ к защищённой таблице.

1. Оповещение ответственных
2. Изоляция доступа (отзыв привилегии)
3. Сбор логов аудита за период
4. Анализ, кто и как получил доступ
5. Восстановление нормального состояния и контроль

Подсказка: При инциденте сначала ограничивайте ущерб, потом расследуйте.

Ответ:

- Оповещение ответственных
- Изоляция доступа (отзыв привилегии)
- Сбор логов аудита за период

- Анализ, кто и как получил доступ
- Восстановление нормального состояния и контроль

Шаг 15:

Миссия 15

Расположите прямые и косвенные способы ограничения доступа в порядке от наиболее детального к общему:

1. VPD / row-level policy
2. Column-level privileges
3. EXECUTE на процедуру
4. ROLE с SYSTEM PRIVILEGES

Подсказка: Детальнее — ближе к данным.

Ответ:

- VPD / row-level policy
- Column-level privileges
- EXECUTE на процедуру
- ROLE с SYSTEM PRIVILEGES

Шаг 16:

Миссия 16

Расположите лучшие практики назначения привилегий в правильном порядке:

1. Назначать права только на необходимые объекты
2. Использовать роли вместо прямых грантов
3. Периодически пересматривать роли и права
4. Документировать, кому и зачем даны роли

Подсказка: Это не просто техника, а культура безопасности.

Ответ:

- Назначать права только на необходимые объекты
- Использовать роли вместо прямых грантов
- Периодически пересматривать роли и права
- Документировать, кому и зачем даны роли

Урок 2 - Шифрование данных и защита паролей в Oracle

Шаг 01:

Миссия 1

Разработчик PHP просит вас сохранить пользовательский пароль в базе Oracle: что лучше?

Подсказка: Никогда не храните пароль в открытом виде.

Ответ:

- **Хэшировать пароль с солью и хранить хеш**
- Сохранять пароль в зашифрованном виде, но легко расшифровывать
- Хранить пароль как plain-text и полагаться на безопасность сервера

Шаг 02:

Миссия 2

Вы хотите шифровать колонку с кредитной картой в Oracle. Какой подход выбрать?

Подсказка: Подумайте, где хранить ключи.

Ответ:

- **Использовать Transparent Data Encryption (TDE) на уровне столбца**
- Шифровать в приложении, перед вставкой, без серверного шифрования
- Оставить данные открытыми, шифруя только резервные копии

Шаг 03:

Миссия 3

Ваша система требует, чтобы все данные в базе хранились зашифрованными «на диске». Что задействовать?

Подсказка: Это не на уровне столбца, а на уровне файлов данных.

Ответ:

- **Transparent Data Encryption (TDE) на уровне tablespaces**
- PL/SQL шифрование вручную каждой вставки
- Шифрование только резервных копий

Шаг 04:

Миссия 4

В проекте PHP надо сравнить введенный пользователем пароль с тем, что хранится в базе Oracle. Как сделать это безопасно?

Подсказка: Никогда не расшифровывайте хеш.

Ответ:

- **Хэшировать введенный пароль и сравнить с хешем в базе**
- Дешифровать хеш из базы и сравнить строку
- Отправить пароль на сервер и сравнить там в открытом виде

Шаг 05:

Миссия 5

Вы собираетесь создать резервную копию базы, включая зашифрованные данные. Как обеспечить безопасность ключей шифрования?

Подсказка: Ключи не должны храниться вместе с резервной копией.

Ответ:

- **Хранить ключи на отдельном HSM или ключевом хранилище**
- Копировать ключи вместе с резервной базой
- Не шифровать резервную копию

Шаг 06:

Миссия 6

После аудита безопасности вас просят зашифровать отдельные столбцы логов, но оставить другие открытыми для аналитики. Как это реализовать?

Подсказка: В Oracle можно гибко комбинировать уровни шифрования.

Ответ:

- **Использовать column encryption на нужных полях, остальные оставить открытыми**
- Применить TDE ко всей таблице целиком
- Шифровать вручную через PL/SQL в каждой вставке

Шаг 07:

Миссия 7

Сопоставьте тип шифрования с его назначением в Oracle.

Типы шифрования:

Ответ:

- TDE tablespace encryption
- TDE column encryption
- PL/SQL encryption (DBMS_CRYPTO)
- Encrypt in application (до вставки)

Назначения:

1. Шифрование данных «на диске» на уровне файлов tablespace
2. Шифрование отдельных колонок при вставке/обновлении
3. Шифрование внутри PL/SQL пакетов, логика шифрования на стороне БД
4. Шифрование на стороне PHP до передачи к БД

Подсказка: Подумайте, на каком уровне применяется механизм.

- **TDE tablespace encryption** — Шифрование данных «на диске» на уровне файлов tablespace
- **TDE column encryption** — Шифрование отдельных колонок при вставке/обновлении
- **PL/SQL encryption (DBMS_CRYPTO)** — Шифрование внутри PL/SQL пакетов, логика шифрования на стороне БД
- **Encrypt in application (до вставки)** — Шифрование на стороне PHP до передачи к БД

Шаг 08:

Миссия 8

Сопоставьте алгоритм шифрования/хэширования с его особенностью или применением.

Алгоритмы:

Ответ:

- AES
- RSA
- SHA-256
- PBKDF2 / bcrypt / scrypt

Особенности / применение:

1. **Симметричное блочное шифрование** — быстрое для данных
2. Асимметричное шифрование для обмена ключами
3. Криптографический хэш для целостности и проверки паролей
4. Хэширование с солью и итерациями для паролей пользователей

Подсказка: Помните различие между шифрованием и хэшированием.

- **AES** — Симметричное блочное шифрование — быстрое для данных

- **RSA** — Асимметричное шифрование для обмена ключами
- **SHA-256** — Криптографический хэш для целостности и проверки паролей
- **PBKDF2 / bcrypt / scrypt** — Хэширование с солью и итерациями для паролей пользователей

🔮 Шаг 09:

Миссия 9

Сопоставьте угрозу с подходом защиты в контексте шифрования данных в Oracle.

Угрозы:

Ответ:

- Утечка файлов данных
- Компрометация ключей
- SQL-инъекция, выдающая здоровые данные
- Подмена пароля администратора

Механизмы защиты:

1. TDE шифрование файлов данных
2. Хранение ключей в HSM / Wallet
3. Хэширование и проверка паролей, минимизация раскрытия данных
4. Аудит привилегий, разделение обязанностей

Подсказка: У каждой угрозы — своя стена защиты.

- **Утечка файлов данных** — TDE шифрование файлов данных
- **Компрометация ключей** — Хранение ключей в HSM / Wallet
- **SQL-инъекция, выдающая здоровые данные** — Хэширование и проверка паролей, минимизация раскрытия данных
- **Подмена пароля администратора** — Аудит привилегий, разделение обязанностей

🔮 Шаг 10:

Миссия 10

Сопоставьте уровень шифрования/контекста с его применением.

Контексты:

Ответ:

- Tablespace
- Column
- Application
- PL/SQL внутри БД

Преимущества / особенности:

1. Защита всего физического файла БД
2. Гибкое шифрование отдельных полей
3. Полный контроль и шифрование до передачи к БД
4. Шифрование внутри базы, сокращая сетевой трафик

Подсказка: Уровень ближе к приложению — выше контроль, но больше сложности.

- **Tablespace** — Защита всего физического файла БД
- **Column** — Гибкое шифрование отдельных полей

- **Application** — Полный контроль и шифрование до передачи к БД
- **PL/SQL внутри БД** — Шифрование внутри базы, сокращая сетевой трафик

Шаг 11:

Миссия 11

Расположите шаги процесса безопасной смены ключа шифрования данных.

1. Создание нового ключа (генерация)
2. Обновление метаданных БД, указывающих на новый ключ
3. Реэнкрипция данных (data rekeying)
4. Переключение на новый ключ в работе системы
5. Архивация и уничтожение старого ключа

Подсказка: Начинаем с генерации, затем меняем метаданные, затем перешифруем, переключаем и утилизируем старый ключ.

Ответ:

- Создание нового ключа (генерация)
- Обновление метаданных БД, указывающих на новый ключ
- Реэнкрипция данных (data rekeying)
- Переключение на новый ключ в работе системы
- Архивация и уничтожение старого ключа

Шаг 12:

Миссия 12

Расположите этапы аудита шифрования и защиты паролей в вашей системе.

1. Проверка, какие поля не зашифрованы
2. Сравнение со стандартом безопасности (политикой компании)
3. Проверка схем ключей и их хранения
4. Тестирование восстановления из резервных копий с ключами
5. Документирование всех решений

Подсказка: Сначала обнаруживаем недочёты, затем проверяем ключи, тестируем восстановление, фиксируем всё.

Ответ:

- Проверка, какие поля не зашифрованы
- Сравнение со стандартом безопасности (политикой компании)
- Проверка схем ключей и их хранения
- Тестирование восстановления из резервных копий с ключами
- Документирование всех решений

Шаг 13:

Миссия 13

Расположите порядок реакций на утечку хешей паролей.

1. Оповестить пользователей о необходимости смены пароля
2. Приостановить доступ к системе для замены хешей
3. Анализ логов и выяснение, как хеши утекли
4. Пересоздать новые соли и хеши для всех пользователей

5. Внести изменения в политику паролей и внедрить 2FA

Подсказка: Сначала защитить систему, потом разобраться, потом улучшать политику.

Ответ:

- Оповестить пользователей о необходимости смены пароля
- Приостановить доступ к системе для замены хешей
- Анализ логов и выяснение, как хеши утекли
- Пересоздать новые соли и хеши для всех пользователей
- Внести изменения в политику паролей и внедрить 2FA

Шаг 14:

Миссия 14

Расположите пути совершенствования системы шифрования и паролей в порядке от базового к продвинутому.

1. Хэширование паролей с солью
2. Применение защищённого алгоритма (bcrypt, PBKDF2)
3. Внедрение TDE на tablespace / column
4. Управление ключами через HSM / Wallet
5. Ротация ключей и аудит криптоопераций

Подсказка: Начинайте с паролей, потом шифруйте данные, управлять ключами, и лишь затем развивать процессы.

Ответ:

- Хэширование паролей с солью
- Применение защищённого алгоритма (bcrypt, PBKDF2)
- Внедрение TDE на tablespace / column
- Управление ключами через HSM / Wallet
- Ротация ключей и аудит криптоопераций

Шаг 15:

Миссия 15

Расположите слои защиты данных от пользователя до физического хранения:

1. Шифрование на уровне приложения до отправки в БД
2. Хэширование паролей
3. Шифрование колонок или TDE столбцов
4. Шифрование файлов данных (TDE tablespace)

Подсказка: Чем ближе к пользователю — тем выше контроль, чем ближе к диску — защита данных «на диске».

Ответ:

- Шифрование на уровне приложения до отправки в БД
- Хэширование паролей
- Шифрование колонок или TDE столбцов
- Шифрование файлов данных (TDE tablespace)

Шаг 16:

Миссия 16

Расположите этапы внедрения комплексной криптографической защиты в крупном проекте.

1. Анализ данных и требований безопасности
2. Выбор архитектуры шифрования и хранения ключей
3. Реализация на уровне приложения и БД
4. Тестирование, аудит и проверка восстановления
5. Мониторинг, ротация ключей и обновление политики

Подсказка: Начинайте с анализа, затем проектируйте, внедряйте, проверяйте и поддерживайте.

Ответ:

- Анализ данных и требований безопасности
- Выбор архитектуры шифрования и хранения ключей
- Реализация на уровне приложения и БД
- Тестирование, аудит и проверка восстановления
- Мониторинг, ротация ключей и обновление политики

Урок 3 - Управление сессиями и мониторинг доступа

Шаг 01:

Миссия 1

Вы — разработчик крупной финансовой системы на PHP + Oracle. Пользователь входит в систему, и вы хотите защитить сессию от подмены. Что вы сделаете первым?

Подсказка: Сессия — это идентификатор, передаваемый клиенту.

Ответ:

- Хранить ID сессии в URL
- **Использовать защиту по cookie с атрибутами Secure и HttpOnly**
- Передавать сессионные данные напрямую через POST каждый раз

Шаг 02:

Миссия 2

Вы заметили, что сессии пользователей живут слишком долго без активности. Что вы настроите?

Подсказка: Есть параметр “timeout” или “lifetime” сессии.

Ответ:

- **Установить ограничение времени бездействия (idle timeout)**
- Отключить автоматический выход пользователей
- Сделать бесконечные сессии

Шаг 03:

Миссия 3

При интеграции PHP с Oracle вы решили хранить сессионные данные в базе Oracle (таблица sessions). Какой из вариантов предпочтителен?

Подсказка: Доступ к сессии должен быть быстрым и атомарным.

Ответ:

- **Использовать CLOB/BLOB для сериализации сессии**
- Хранить каждое поле сессии как отдельный столбец
- Хранить всё как JSON-строку и не индексировать

Шаг 04:

Миссия 4

Вы хотите предотвратить «session fixation» атаку. Какой подход выберете при логине?

Подсказка: Сессия, выданная до аутентификации, может быть захвачена.

Ответ:

- **Сгенерировать новую сессию после успешного логина**
- Продолжать старую сессию
- Игнорировать сессионный идентификатор

Шаг 05:

Миссия 5

Коллега говорит: «Я могу использовать session_start() с опцией name, и этим защититься».

Насколько это безопасно?

Подсказка: Переименование параметра не защищает от атак.

Ответ:

- **Это слабая защита, не достаточно сама по себе**
- Это полная защита
- Это делает систему уязвимой

Шаг 06:

Миссия 6

Сопоставьте принцип защиты сессий с описанием:

Ответ:

- Регистрация сессии по IP
- Хеширование данных сессии
- Ограничение по user agent
- Ротация сессионного ID

1. При каждом запросе сравнивать текущий User Agent с тем, что при старте сессии
2. Сравнивать IP-адрес пользователя с сохранённым в сессии
3. Обновлять идентификатор сессии через равные промежутки или при смене прав
4. Сохранение хэша (не открытого содержимого) сессионных данных для проверки целостности

Подсказка: Подумайте, какая защита проверяет условия, а какая — изменяет идентификатор.

- **Регистрация сессии по IP** — Сравнивать IP-адрес пользователя с сохранённым в сессии
- **Хеширование данных сессии** — Сохранение хэша (не открытого содержимого) сессионных данных для проверки целостности
- **Ограничение по user agent** — При каждом запросе сравнивать текущий User Agent с тем, что при старте сессии
- **Ротация сессионного ID** — Обновлять идентификатор сессии через равные промежутки или при смене прав

Шаг 07:

Миссия 7

Сопоставьте уязвимость с техникой атаки:

Ответ:

- Session fixation
- Session hijacking
- Cross-site scripting
- Cross-site request forgery (CSRF)

1. Атака, при которой злоумышленник подставляет заранее известный session ID
2. Кража сессии через перехват сессионного токена
3. Внедрение скриптов на страницу жертвы
4. Принуждение браузера выполнить нежелательный запрос сессией жертвы

Подсказка: Подумайте, какая атака насильно подставляет ID, а какая — реализует действия от лица сессии.

- **Session fixation** — Атака, при которой злоумышленник подставляет заранее известный session ID
- **Session hijacking** — Кража сессии через перехват сессионного токена
- **Cross-site scripting** — Внедрение скриптов на страницу жертвы
- **Cross-site request forgery (CSRF)** — Принуждение браузера выполнить нежелательный запрос сессией жертвы

 Шаг 08:

Миссия 8

Сопоставьте элемент мониторинга с примерами данных, которые вы захотите хранить:

Ответ:

- Лог входа/выхода
 - Неудачные попытки доступа
 - Изменение прав доступа
 - Истечение сессии
1. Запись timestamp + ID пользователя при логине/логауте
 2. Запись IP и время, когда пользователь ввёл неверный пароль
 3. Фиксация, что пользователь получил или лишился роли админа
 4. Фиксация таймаута сессии из-за бездействия

Подсказка: Лог должен покрывать как успешные, так и отказанные события.

- **Лог входа/выхода** — Запись timestamp + ID пользователя при логине/логауте
- **Неудачные попытки доступа** — Запись IP и время, когда пользователь ввёл неверный пароль
- **Изменение прав доступа** — Фиксация, что пользователь получил или лишился роли админа
- **Истечение сессии** — Фиксация таймаута сессии из-за бездействия

 Шаг 09:

Миссия 9

Сопоставьте стадию с мониторинговым действием:

Ответ:

- Аутентификация
- Авторизация

- Использование сессии
- Завершение сессии

1. Лог успеха/неудачи логина
2. Лог доступа к ресурсу с проверкой ролей
3. Лог активности внутри сессии (запросы, время, IP)
4. Запись события logout или таймаут

Подсказка: Подумайте, на каком этапе что логируется.

- **Аутентификация** — Лог успеха/неудачи логина
- **Авторизация** — Лог доступа к ресурсу с проверкой ролей
- **Использование сессии** — Лог активности внутри сессии (запросы, время, IP)
- **Завершение сессии** — Запись события logout или таймаут

Шаг 10:

Миссия 10

Расположите шаги обработки сессии при запросе пользователя:

1. Проверка валидности session ID
2. Загрузка данных сессии из Oracle
3. Проверка прав доступа к ресурсу
4. Проверка времени бездействия (idle timeout)
5. Обновление времени последней активности

Подсказка: Сначала validate, затем load, затем проверки, затем обновить timestamp.

Ответ:

- Проверка валидности session ID
- Загрузка данных сессии из Oracle
- Проверка времени бездействия (idle timeout)
- Проверка прав доступа к ресурсу
- Обновление времени последней активности

Шаг 11:

Миссия 11

Расположите этапы ответа при попытке неавторизованного доступа:

1. Запись события в лог мониторинга
2. Вернуть HTTP 403 или редирект к логину
3. Проверка прав текущей сессии
4. Завершение выполнения запроса защищённого ресурса

Подсказка: Сначала проверка, потом лог, потом ответ.

Ответ:

- Проверка прав текущей сессии
- Запись события в лог мониторинга
- Вернуть HTTP 403 или редирект к логину
- Завершение выполнения запроса защищённого ресурса

Шаг 12:

Миссия 12

Расположите задачи администратора по монитору безопасности за сервером приложений:

1. Анализ логов на необычные сессии
2. Настройка предупреждений при множественных провалах логина
3. Обработка инцидентов (блокировка IP, уведомление)
4. Регулярный аудит прав доступа

Подсказка: Мониторинг → предупреждение → реакция → аудит.

Ответ:

- Анализ логов на необычные сессии
- Настройка предупреждений при множественных провалах логина
- Обработка инцидентов (блокировка IP, уведомление)
- Регулярный аудит прав доступа

Шаг 13:

Миссия 13

Расположите в хронологическом порядке действия злоумышленника при атаке сессии:

1. Получение или угадывание session ID
2. Отправка запроса от имени захваченной сессии
3. Доступ к защищённому ресурсу
4. Перехват трафика или фиксация ID

Подсказка: Вначале идёт захват, потом использование.

Ответ:

- Получение или угадывание session ID
- Перехват трафика или фиксация ID
- Отправка запроса от имени захваченной сессии
- Доступ к защищённому ресурсу

Шаг 14:

Миссия 14

Расположите уровни логов по важности (от наименее подробного к самому подробному):

1. ERROR
2. DEBUG
3. WARN
4. INFO

Подсказка: DEBUG — самый подробный; ERROR — минимальный.

Ответ:

- DEBUG
- INFO
- WARN
- ERROR

Шаг 15:

Миссия 15

Расположите этапы восстановления после инцидента с компрометацией сессий:

1. Изменить ключи шифрования / секреты сессий
2. Уведомить пользователей о перезапуске сессий
3. Принудительно завершить все активные сессии
4. Провести анализ, как была взломана система

Подсказка: Сначала остановить утечку, потом уведомить, потом проанализировать.

Ответ:

- Принудительно завершить все активные сессии
- Изменить ключи шифрования / секреты сессий
- Уведомить пользователей о перезапуске сессий
- Провести анализ, как была взломана система

Шаг 16:

Миссия 16

Расположите приоритеты безопасности сессий в архитектуре крупного проекта:

1. Защита от XSS / CSRF
2. Мониторинг и логирование аномалий
3. Контроль таймаутов и ротация ID
4. Шифрование содержимого сессии

Подсказка: Защита, детекция, контроль и защита данных внутри.

Ответ:

- Защита от XSS / CSRF
- Контроль таймаутов и ротация ID
- Мониторинг и логирование аномалий
- Шифрование содержимого сессии

Шаг 17:

Миссия 17

Расположите уровни реагирования на системные тревоги (от простого к максимально строгому):

1. Уведомление администраторов
2. Автоматическая блокировка подозрительной сессии
3. Перевод системы в режим повышенной безопасности
4. Принудительное завершение всех сессий

Подсказка: Сначала предупреждение, потом изоляция, затем ограничение, затем полная реакция.

Ответ:

- Уведомление администраторов
- Автоматическая блокировка подозрительной сессии
- Перевод системы в режим повышенной безопасности
- Принудительное завершение всех сессий

Урок 4 - Работа с аудиторскими журналами Oracle

Шаг 01:

Миссия 1

Вы — разработчик, и вам нужно включить аудит в Oracle чтобы отслеживать DML-операции. Какой параметр аудита вы активируете сначала?

Подсказка: Начните с базового уровня для всего схемы.

Ответ:

- **AUDIT ALL BY SCHEMA**
- AUDIT SELECT ANY TABLE
- AUDIT ROLLBACK

Шаг 02:

Миссия 2

Аудит в Oracle уже включён, но вам нужно ограничить его по типу операций для таблицы USERS — только UPDATE и DELETE. Какой AUDIT-директивой вы воспользуетесь?

Подсказка: Используйте ключевые слова операций после имени таблицы.

Ответ:

- **AUDIT UPDATE, DELETE ON users**
- AUDIT ALL ON users
- AUDIT INSERT ON users

Шаг 03:

Миссия 3

Вы хотите, чтобы Oracle записывал не только успешные операции, но и неудачные попытки доступа. Какой вариант аудита выбрать?

Подсказка: Ищите директиву, включающую FAILED или BY ACCESS.

Ответ:

- **AUDIT ALL BY SCHEMA WHENEVER NOT SUCCESSFUL**
- AUDIT ALL WHENEVER SUCCESSFUL
- NOAUDIT ALL

Шаг 04:

Миссия 4

Вы ведёте аудит на уровне СУБД и замечаете, что журнал быстро растёт. Как лучше управлять объёмом записей?

Подсказка: Поищите способ архивировать старые записи.

Ответ:

- **Создать политику purge старых записей в DBA_AUDIT_TRAIL**
- Отключить аудит полностью
- Увеличить размер журнала

Шаг 05:

Миссия 5

Сопоставьте виды аудита Oracle с их назначением:

Виды аудита:

Ответ:

- Fine-grained audit (FGA)
- Standard audit
- Unified audit trail
- Операционный аудит (режим SYSLOG)

Назначения:

1. Аудит на основе условий (WHERE) в запросах
2. Классический аудит в старых версиях Oracle
3. Новый унифицированный журнал аудита
4. Отправка записей аудита в системный журнал ОС

Подсказка: FGA реагирует на фильтры, Unified объединяет старые журналы.

- **Fine-grained audit (FGA)** — Аудит на основе условий (WHERE) в запросах
- **Standard audit** — Классический аудит в старых версии Oracle
- **Unified audit trail** — Новый унифицированный журнал аудита
- **Операционный аудит (режим SYSLOG)** — Отправка записей аудита в системный журнал ОС

Шаг 06:

Миссия 6

Вы хотите, чтобы запись аудита хранилась в XML-формате в таблице. Какой вариант спецификации используете?

Подсказка: Ищите формат XML в настройках AUDIT_TRAIL.

Ответ:

- **AUDIT_TRAIL = XML**
- AUDIT_TRAIL = DB
- AUDIT_TRAIL = NONE

Шаг 07:

Миссия 7

Вы активировали унифицированный аудит. Какая таблица содержит записи аудита?

Подсказка: Оно часто начинается с "UNIFIED_..."

Ответ:

- **UNIFIED_AUDIT_TRAIL**
- DBA_AUDIT_TRAIL
- V\$AUDIT_RECORDS

Шаг 08:

Миссия 8

Сопоставьте столбцы в UNIFIED_AUDIT_TRAIL и их смысл:

Столбцы:

Ответ:

- EVENT_TIMESTAMP
- DBUSERNAME
- OBJ_NAME
- SQL_TEXT

Описание:

1. Имя схемы, от имени которой выполнена операция
2. Название объекта, к которому обращаются
3. Текст выполняемой SQL-команды
4. Временная метка события

Подсказка: По названию легко угадать смысл столбца.

- **EVENT_TIMESTAMP** — Временная метка события
- **DBUSERNAME** — Имя схемы, от имени которой выполнена операция
- **OBJ_NAME** — Название объекта, к которому обращаются
- **SQL_TEXT** — Текст выполняемой SQL-команды

 Шаг 09:

Миссия 9

Чтобы отфильтровать записи аудита по времени, вы пишете WHERE EVENT_TIMESTAMP > :last_run. Что важно учесть?

Подсказка: Формат даты при сравнении с TIMESTAMP.

Ответ:

- **Преобразовать :last_run в TIMESTAMP**
- Использовать VARCHAR формат
- Сравнить как целое число

 Шаг 10:

Миссия 10

Сопоставьте проблемы с аудитом с решениями:

Проблемы:

Ответ:

- Журнал перегружается
- Высокая нагрузка записи
- Записи не видны приложению
- Нарушение конфиденциальности

Решения:

1. Архивация и purge
2. Буферизация или пакетная запись
3. Выдача прав SELECT на UNIFIED_AUDIT_TRAIL
4. Фильтрация или маскирование колонок аудита

Подсказка: Каждая проблема имеет своё специфическое решение.

- **Журнал перегружается** — Архивация и purge
- **Высокая нагрузка записи** — Буферизация или пакетная запись

- **Записи не видны приложению** — Выдача прав SELECT на UNIFIED_AUDIT_TRAIL
- **Нарушение конфиденциальности** — Фильтрация или маскирование колонок аудита

Шаг 11:

Миссия 11

Вы пишете PHP-код, который читает журнал аудита из Oracle. Какой тип соединения предпочтительнее?

Подсказка: Ищите драйвер, поддерживающий чтение CLOB и TIMESTAMP.

Ответ:

- **OCI8**
- mysqlnd
- pgsql

Шаг 12:

Миссия 12

В вашем PHP-скрипте получены поля аудита, сопоставьте формат и способ обработки:

Поля аудита:

Ответ:

- **EVENT_TIMESTAMP (TIMESTAMP)**
- **SQL_TEXT (CLOB)**
- **DBUSERNAME (VARCHAR2)**
- **OBJ_NAME (VARCHAR2)**

Обработка в PHP:

1. (string) приведение к строке
2. Использование DateTime
3. Без изменений, как строка из БД
4. Проверка на NULL и экранирование

Подсказка: TIMESTAMP требует работы с датой, CLOB — чтения длинного текста.

- **EVENT_TIMESTAMP (TIMESTAMP)** — Использование DateTime
- **SQL_TEXT (CLOB)** — (string) приведение к строке
- **DBUSERNAME (VARCHAR2)** — Без изменений, как строка из БД
- **OBJ_NAME (VARCHAR2)** — Проверка на NULL и экранирование

Шаг 13:

Миссия 13

Вы решили сохранять результаты аудита в отдельной таблице PHP-приложения. Какой подход безопаснее?

Подсказка: Старайтесь не копировать всё из журнала без фильтрации.

Ответ:

- **Копировать только нужные поля через безопасный SELECT**
- Копировать весь UNIFIED_AUDIT_TRAIL *
- Копировать через прямой файл экспорта

Шаг 14:

Миссия 14

Вы хотите настроить ротацию аудита с сохранением копий по дням. Какой механизм используете на уровне БД?

Подсказка: Думайте о планировщике задач Oracle.

Ответ:

- **Использовать DBMS_SCHEDULER для ежедневного PURGE**
- Установить триггер на UNIFIED_AUDIT_TRAIL
- Запуск внешнего cron-скрипта из PHP

Шаг 15:

Миссия 15

При аудите вы обнаруживаете, что столбец PASSWORD_HASH попадает в записи аудита. Что разумно сделать?

Подсказка: Не разглашайте чувствительные данные.

Ответ:

- **Исключить этот столбец из аудита (NOAUDIT COLUMN)**
- Оставить как есть
- Шифровать его в приложении после чтения

Шаг 16:

Миссия 16

Расположите этапы настройки унифицированного аудита с нуля в правильном порядке:

1. Включить механизм Unified Audit
2. Настроить правила аудита (CREATE AUDIT POLICY ...)
3. Применить политику (AUDIT POLICY ...)
4. Проверить записи в UNIFIED_AUDIT_TRAIL
5. Очищать старые записи (PURGE или Scheduler)

Подсказка: Сначала активность, затем политика, затем применение и проверка.

Ответ:

- Включить механизм Unified Audit
- Настроить правила аудита (CREATE AUDIT POLICY ...)
- Применить политику (AUDIT POLICY ...)
- Проверить записи в UNIFIED_AUDIT_TRAIL
- Очищать старые записи (PURGE или Scheduler)

Шаг 17:

Миссия 17

Упорядочите шаги реагирования на инцидент (подозрение на SQL-инъекцию), используя данные аудита:

1. Идентифицировать подозрительные записи SQL_TEXT из журнала
2. Проверить EVENT_TIMESTAMP и DBUSERNAME
3. Изолировать подключение или пользователя
4. Собрать журналы и экспортировать их на расследование

5. Обновить политику аудита или исключения

Подсказка: Сначала надо обнаружить, затем ограничить ущерб, затем документировать и усилить защиту.

Ответ:

- Идентифицировать подозрительные записи SQL_TEXT из журнала
- Проверить EVENT_TIMESTAMP и DBUSERNAME
- Изолировать подключение или пользователя
- Собрать журналы и экспортировать их на расследование
- Обновить политику аудита или исключения

Урок 5 - Реализация резервного копирования и восстановления данных

Шаг 01:

Миссия 1

Вы разворачиваете систему бэкапов для PHP + Oracle, и хотите начать с простого уровня. Что из ниже перечисленного является ****первым действием**** при планировании резервного копирования?

Подсказка: Начинать надо с оценки, прежде чем писать скрипты.

Ответ:

- **Оценка объёмов данных и критичности восстановления**
- Сразу писать PL/SQL скрипты резервного копирования
- Выполнить полное резервное копирование всего диска
- Запустить full export Oracle без планирования

Шаг 02:

Миссия 2

Сопоставьте тип восстановления с его описанием в контексте Oracle-бэкапов:

Типы:

Ответ:

- Point-in-time recovery
- Complete recovery
- Tablespace restore
- Incremental recovery

Описание:

1. Восстановление до заданного времени (внутри window)
2. Восстановление всей базы до последнего целого состояния
3. Восстановление конкретного пространства данных
4. Применение инкрементальных бэкапов после полного восстановления

Подсказка: Подумайте, какие операции влияют на всю базу, а какие — только часть.

- **Point-in-time recovery** — Восстановление до заданного времени (внутри window)
- **Complete recovery** — Восстановление всей базы до последнего целого состояния
- **Tablespace restore** — Восстановление конкретного пространства данных
- **Incremental recovery** — Применение инкрементальных бэкапов после полного восстановления

🔗 Шаг 03:

Миссия 3

Сопоставьте ситуацию и стратегию восстановления:

Ситуация:

Ответ:

- Потеря всего сервера в необычное время
- Сбой отдельной таблицы с данными заказов
- Случайное удаление данных вчера вечером
- Загрязнение данных (ошибочный INSERT) в 2 часа назад

Стратегия восстановления:

1. Point-in-time recovery до времени перед ошибкой
2. Restore full + apply incremental + archived redo
3. Tablespace recovery + экспорт/импорт части схемы
4. Flashback table или восстановление из архивов логов

Подсказка: Подумайте, насколько широк масштаб сбоя и какие инструменты доступны.

- **Потеря всего сервера в необычное время** — Restore full + apply incremental + archived redo
- **Сбой отдельной таблицы с данными заказов** — Tablespace recovery + экспорт/импорт части схемы
- **Случайное удаление данных вчера вечером** — Point-in-time recovery до времени перед ошибкой
- **Загрязнение данных (ошибочный INSERT) в 2 часа назад** — Flashback table или восстановление из архивов логов

🔗 Шаг 04:

Миссия 4

Сопоставьте термин и его значение при резервировании Oracle:

Термины:

Ответ:

- SCN (System Change Number)
- Archive log
- Channel
- Backup set

Значения:

1. Номер точки согласованности для восстановления
2. Сохранённая копия файлов бэкапа, возможно объединённая
3. Устройство или поток, через который выполняется бэкап/восстановление
4. Журнал redo, который архивируется после переключения

Подсказка: Вспомните архитектуру RMAN и архивации Oracle.

- **SCN (System Change Number)** — Номер точки согласованности для восстановления
- **Archive log** — Журнал redo, который архивируется после переключения
- **Channel** — Устройство или поток, через который выполняется бэкап/восстановление
- **Backup set** — Сохранённая копия файлов бэкапа, возможно объединённая

Шаг 05:

Миссия 5

Упорядочите шаги восстановления полной базы с архивами до рабочего состояния:

1. RESTORE DATABASE
2. RECOVER DATABASE
3. BACKUP ARCHIVELOG APPLY
4. OPEN RESETLOGS / OPEN
5. APPLY ARCHIVELOGS (redo logs)

Подсказка: Восстановление идёт от полного → применяем логи → открываем базу.

Ответ:

- RESTORE DATABASE
- BACKUP ARCHIVELOG APPLY
- APPLY ARCHIVELOGS (redo logs)
- RECOVER DATABASE
- OPEN RESETLOGS / OPEN

Шаг 06:

Миссия 6

Упорядочите следующие элементы политики резервирования от наиболее частого до наименее частого:

1. Полное резервное копирование
2. Инкрементальное резервное копирование
3. Архивация redo-журналов
4. Тестовое восстановление

Подсказка: Как часто вы реально применяете, а как — разово или редко?

Ответ:

- Архивация redo-журналов
- Инкрементальное резервное копирование
- Полное резервное копирование
- Тестовое восстановление

Шаг 07:

Миссия 7

Расположите шаги восстановления только одной таблицы через экспорты и импорты в Oracle:

1. Импорт таблицы в рабочую схему
2. Экспорт нужной таблицы из резервной копии или схемы backup
3. Отключение FK/индексов на целевой схеме
4. Включение FK/индексов и проверка связей
5. Очистка или переименование старой таблицы

Подсказка: Перед импортом надо подготовить место, затем вернуть ограничения.

Ответ:

- Экспорт нужной таблицы из резервной копии или схемы backup
- Очистка или переименование старой таблицы

- Отключение FK/индексов на целевой схеме
- Импорт таблицы в рабочую схему
- Включение FK/индексов и проверка связей

🔮 Шаг 08:

Миссия 8

Упорядочите этапы автоматизации рутинного восстановления (playbook) для PHP-сервиса на Oracle:

1. Проверка успешности восстановления и целостности данных
2. Запуск скрипта восстановления (backend + базы)
3. Отключение сервисов приложения перед восстановлением
4. Оповещение команды о старте восстановления
5. Возобновление работы сервиса после проверки

Подсказка: Автоматизация — это план, уведомление, действия и проверка в конце.

Ответ:

- Оповещение команды о старте восстановления
- Отключение сервисов приложения перед восстановлением
- Запуск скрипта восстановления (backend + базы)
- Проверка успешности восстановления и целостности данных
- Возобновление работы сервиса после проверки

🔮 Шаг 09:

Миссия 9

Упорядочите шаги при восстановлении с зашифрованных резервных копий в Oracle RMAN:

1. RESTORE with DECRYPT option или предоставить ключ
2. RECOVER после расшифровки и применения логов
3. OPEN RESETLOGS
4. VALIDATE backupset перед использованием
5. CONFIGURE ENCRYPTION параметров при бэкапе

Подсказка: Шифрование надо учесть заранее, затем расшифровать в нужный момент.

Ответ:

- CONFIGURE ENCRYPTION параметров при бэкапе
- VALIDATE backupset перед использованием
- RESTORE with DECRYPT option или предоставить ключ
- RECOVER после расшифровки и применения логов
- OPEN RESETLOGS

🔮 Шаг 10:

Миссия 10

Упорядочите шаги ****реального сценария**** восстановления при катастрофе (крах целого сервера) для PHP-приложения + Oracle:

1. Разворачивание ОС и базового ПО на новом сервере
2. Настройка сетевых прав и доступа к хранилищу бэкапов
3. RESTORE DATABASE + APPLY ARCHIVELOGS + RECOVER

4. Восстановление приложения PHP и его конфигураций
5. Тестирование интеграции и бизнес-логики

Подсказка: Сначала инфраструктура, потом данные, потом приложение, затем тесты.

Ответ:

- Разворачивание ОС и базового ПО на новом сервере
- Настройка сетевых прав и доступа к хранилищу бэкапов
- RESTORE DATABASE + APPLY ARCHIVELOGS + RECOVER
- Восстановление приложения PHP и его конфигураций
- Тестирование интеграции и бизнес-логики

Шаг 11:

Миссия 11

Вы работаете в команде, которая отвечает за безопасность данных. Какой метод резервного копирования данных Oracle будет самым быстрым для восстановления после сбоя?

Подсказка: Важно выбрать способ, минимизирующий время простоя системы.

Ответ:

- **Incremental backup (инкрементальное резервное копирование)**
- Full backup (полное резервное копирование)
- Cold backup (холодное резервное копирование)
- Export dump (экспорт дампа)

Шаг 12:

Миссия 12

Ваш сервер Oracle перестал отвечать, и нужно срочно восстановить данные. Какой тип восстановления будет самым безопасным для минимизации потерь?

Подсказка: Рассмотрите вариант, который позволяет восстановить данные до последнего доступного момента.

Ответ:

- **Point-in-time recovery (восстановление на определённый момент времени)**
- Restore from cold backup (восстановление из холодного бэкапа)
- Restore from export dump (восстановление из дампа)
- Physical restore only (только физическое восстановление)

Шаг 13:

Миссия 13

Для обеспечения отказоустойчивости в системе Oracle вы хотите реализовать резервное копирование с минимальным влиянием на работу базы. Какой метод наиболее подходит?

Подсказка: Метод должен работать без остановки сервиса.

Ответ:

- **Online backup (горячее резервное копирование)**
- Offline backup (холодное резервное копирование)
- Logical backup (логическое резервное копирование)
- Snapshot backup (резервное копирование с помощью снимков)

Шаг 14:

Миссия 14

Какой инструмент Oracle предназначен для автоматизации резервного копирования и восстановления?

Подсказка: Этот инструмент часто используется для управления бэкапами и восстановлениями.

Ответ:

- **RMAN (Recovery Manager)**
- Data Guard
- SQL*Loader
- Oracle Streams

Шаг 15:

Миссия 15

При восстановлении базы из резервной копии, какой файл необходимо восстановить в первую очередь для восстановления работы базы данных?

Подсказка: Без него восстановление не начнется.

Ответ:

- **Control file (контрольный файл)**
- Redo log file (журнал повторного выполнения)
- Datafile (файл данных)
- Parameter file (файл параметров)

Шаг 16:

Миссия 16

Сопоставьте тип резервного копирования с его описанием.

Подсказка: Обратите внимание на особенности каждого метода.

Типы резервного копирования:

Ответ:

- Полное резервное копирование
- Инкрементальное резервное копирование
- Дифференциальное резервное копирование
- Холодное резервное копирование

Описание:

1. Резервирование всех данных базы на момент копирования.
 2. Копирование только изменений с момента последнего полного бэкапа.
 3. Копирование изменений с момента последнего инкрементального бэкапа.
 4. Резервирование базы данных при её остановке.
- **Полное резервное копирование** — Резервирование всех данных базы на момент копирования.
 - **Инкрементальное резервное копирование** — Копирование только изменений с момента последнего полного бэкапа.
 - **Дифференциальное резервное копирование** — Копирование изменений с момента последнего инкрементального бэкапа.
 - **Холодное резервное копирование** — Резервирование базы данных при её остановке.

Шаг 17:

Миссия 17

Сопоставьте компоненты Oracle с их ролью в резервном копировании и восстановлении.

Подсказка: Понимание функций поможет выбрать правильный инструмент.

Компоненты:

Ответ:

- RMAN
- Control file
- Redo log
- Archive log

Роли:

1. Инструмент автоматизации резервного копирования и восстановления.
2. Хранит структуру базы данных и важную метаинформацию.
3. Записывает текущие изменения в базе для быстрой откатки.
4. Сохраняет копии redo log для восстановления после сбоев.

- **RMAN** — Инструмент автоматизации резервного копирования и восстановления.
- **Control file** — Хранит структуру базы данных и важную метаинформацию.
- **Redo log** — Записывает текущие изменения в базе для быстрой откатки.
- **Archive log** — Сохраняет копии redo log для восстановления после сбоев.

Шаг 18:

Миссия 18

Сопоставьте понятия резервного копирования с соответствующими им преимуществами.

Подсказка: Подумайте, какие цели стоят за каждым типом резервного копирования.

Понятия:

Ответ:

- Full backup
- Incremental backup
- Cold backup
- Recovery catalog

Преимущества:

1. Минимальное время простоя базы при резервировании.
2. Возможность хранения истории резервных копий.
3. Восстановление всей базы данных из единого источника.
4. Экономия дискового пространства при частом копировании.

- **Full backup** — Восстановление всей базы данных из единого источника.
- **Incremental backup** — Экономия дискового пространства при частом копировании.
- **Cold backup** — Минимальное время простоя базы при резервировании.
- **Recovery catalog** — Возможность хранения истории резервных копий.

Шаг 19:

Миссия 19

Сопоставьте этапы восстановления базы данных с их описаниями.

Подсказка: Восстановление — это четкая последовательность действий.

Этапы восстановления:

Ответ:

- Restore datafiles
- Recover database
- Open database
- Backup control file

Описание:

1. Восстановление файлов данных из резервной копии.
 2. Применение журналов для восстановления изменений.
 3. Разрешение доступа к базе для пользователей.
 4. Использование копии контрольного файла для инициализации.
- **Restore datafiles** — Восстановление файлов данных из резервной копии.
 - **Recover database** — Применение журналов для восстановления изменений.
 - **Open database** — Разрешение доступа к базе для пользователей.
 - **Backup control file** — Использование копии контрольного файла для инициализации.

Модуль 7 - Итоговый проект

Урок 1 - Разработка ТЗ для интеграции PHP и Oracle

Шаг 01:

Миссия 1

Сопоставьте разделы ТЗ с их назначением при интеграции PHP и Oracle:

Разделы:

Ответ:

- Описание API и интерфейсов
- Схемы данных и преобразования
- Ограничения и SLA
- Режим ошибок и восстановление

Назначения:

1. Как система реагирует на сбои или ошибки
2. Скорости отклика, доступность, требования
3. Какие данные, в каком формате и как преобразовывать
4. Какие методы, параметры, форматы запросов

Подсказка: Подумайте, какой раздел должен описывать поведение при ошибках, а какой — структуру данных.

- **Описание API и интерфейсов** — Какие методы, параметры, форматы запросов
- **Схемы данных и преобразования** — Какие данные, в каком формате и как преобразовывать
- **Ограничения и SLA** — Скорости отклика, доступность, требования
- **Режим ошибок и восстановление** — Как система реагирует на сбои или ошибки

Шаг 02:

Миссия 2

Сопоставьте ключевые технические аспекты ТЗ с примерами:

Ключевые аспекты:

Ответ:

- Транзакции
- Кэширование
- Пакетные операции
- Режим асинхронности

Примеры:

1. Объединение нескольких запросов в одну большую партию
2. Использование Redis между PHP и Oracle для часто читаемых данных
3. Откат всех изменений, если одна операция упала
4. Послать запрос на обновление и через очередь обработать позднее

Подсказка: Подумайте, в каком случае нужен откат, а в каком — очередь.

- **Транзакции** — Откат всех изменений, если одна операция упала
- **Кэширование** — Использование Redis между PHP и Oracle для часто читаемых данных
- **Пакетные операции** — Объединение нескольких запросов в одну большую партию
- **Режим асинхронности** — Послать запрос на обновление и через очередь обработать позднее

Шаг 03:

Миссия 3

Сопоставьте сущность и её роль в интеграции PHP + Oracle:

Сущности:

Ответ:

- DAO (Data Access Object)
- DTO (Data Transfer Object)
- Слой сервисов
- Модуль логирования

Роли:

1. Трансформация данных между слоями
2. Инкапсуляция логики доступа к базе Oracle
3. Запись ошибок, трассировка запросов
4. Координация вызовов между PHP и логикой бизнес-правил

Подсказка: DAO касается работы с БД, DTO — передачи данных.

- **DAO (Data Access Object)** — Инкапсуляция логики доступа к базе Oracle
- **DTO (Data Transfer Object)** — Трансформация данных между слоями
- **Слой сервисов** — Координация вызовов между PHP и логикой бизнес-правил
- **Модуль логирования** — Запись ошибок, трассировка запросов

Шаг 04:

Миссия 4

Сопоставьте риск и его потенциальное последствие при неверно составленном ТЗ:

Риски:

Ответ:

- Неучтённые типы данных
- Неправильные права доступа
- Отсутствие обработчика ошибок сети
- Непроработанный сценарий сбоев

Последствия:

1. Ошибки при записи или чтении данных
2. Нарушение безопасности, утечка данных
3. Сервис замрёт без уведомлений при обрыве связи
4. Сложность восстановления после падения, неконсистентность

Подсказка: Свяжите, какой риск ведёт к какому сценарию отказа.

- **Неучтённые типы данных** — Ошибки при записи или чтении данных
- **Неправильные права доступа** — Нарушение безопасности, утечка данных
- **Отсутствие обработчика ошибок сети** — Сервис замрёт без уведомлений при обрыве связи
- **Непроработанный сценарий сбоев** — Сложность восстановления после падения, неконсистентность

Шаг 05:

Миссия 5

Расположите шаги подготовки ТЗ перед тем, как передать заказчику:

1. Сбор требований и обсуждение с заказчиком
2. Описание API, схем, ошибок и SLA
3. Внутренний review и согласование команды
4. Корректировки по замечаниям заказчика
5. Финальная версия и подписание

Подсказка: ТЗ проходит путь: сбор → черновик → review → правки → итог.

Ответ:

- Сбор требований и обсуждение с заказчиком
- Описание API, схем, ошибок и SLA
- Внутренний review и согласование команды
- Корректировки по замечаниям заказчика
- Финальная версия и подписание

Шаг 06:

Миссия 6

Упорядочите этапы разработки прототипа интеграции после утверждения ТЗ:

1. Подготовка схем БД и тестовой базы Oracle
2. Написание PHP-кода для минимального рабочего случая (CRUD)

3. Тестирование на граничные случаи и ошибки
4. Добавление механизма обработки ошибок и логирования
5. Демонстрация прототипа заказчику

Подсказка: Сначала база, затем базовый код, потом устойчивость и демонстрация.

Ответ:

- Подготовка схем БД и тестовой базы Oracle
- Написание PHP-кода для минимального рабочего случая (CRUD)
- Добавление механизма обработки ошибок и логирования
- Тестирование на граничные случаи и ошибки
- Демонстрация прототипа заказчику

Шаг 07:

Миссия 7

Расположите фазы согласования изменений в ТЗ после первой версии прототипа:

1. Сбор замечаний заказчика
2. Оценка изменений командой разработчиков
3. Обновление ТЗ и протокола изменений
4. Согласование новой версии с заказчиком
5. Утверждение и вливание в основной документ

Подсказка: Процесс согласования — это итерации “заказчик → команда → документ → заказчик”.

Ответ:

- Сбор замечаний заказчика
- Оценка изменений командой разработчиков
- Обновление ТЗ и протокола изменений
- Согласование новой версии с заказчиком
- Утверждение и вливание в основной документ

Шаг 08:

Миссия 8

Расположите шаги оценки стоимости проекта интеграции:

1. Оценка трудозатрат по каждому разделу ТЗ
2. Учет рисков и резервов
3. Суммирование оценки задач
4. Оценка инфраструктуры и лицензионных затрат
5. Формирование окончательной сметы и буфера

Подсказка: Общая смета — это задачи + инфраструктура + риски.

Ответ:

- Оценка трудозатрат по каждому разделу ТЗ
- Учет рисков и резервов
- Суммирование оценки задач
- Оценка инфраструктуры и лицензий
- Формирование окончательной сметы и буфера

Шаг 09:

Миссия 9

Упорядочите шаги при приёме интеграционного решения по ТЗ:

1. Проверка соответствия функций ТЗ
2. Нагрузочное и стресс-тестирование
3. Тесты на граничные случаи и отказоустойчивость
4. Презентация результатов заказчику
5. Сбор обратной связи и мелкие корректировки

Подсказка: Сначала проверяем, потом стресс, потом презентация и доработка.

Ответ:

- Проверка соответствия функций ТЗ
- Тесты на граничные случаи и отказоустойчивость
- Нагрузочное и стресс-тестирование
- Презентация результатов заказчику
- Сбор обратной связи и мелкие корректировки

Шаг 10:

Миссия 10

Расположите шаги итеративного улучшения интеграции после запуска:

1. Сбор метрик и мониторинг ошибок
2. Анализ узких мест и предложения улучшений
3. Изменения и расширения по ТЗ (новые требования)
4. Тестирование после изменений
5. Релиз и выпуск версий

Подсказка: Жизненный цикл после запуска — мониторинг, анализ, развитие, тест, выпуск.

Ответ:

- Сбор метрик и мониторинг ошибок
- Анализ узких мест и предложения улучшений
- Изменения и расширения по ТЗ (новые требования)
- Тестирование после изменений
- Релиз и выпуск версий

Шаг 11:

Миссия 11

Расположите фазы разработки полноценного интеграционного проекта PHP + Oracle с момента идеи до эксплуатации:

1. Идея, концепция и сбор требований
2. Разработка ТЗ и согласование
3. Прототип и proof of concept
4. Полная реализация с обработкой ошибок и безопасностью
5. Тестирование, приёмка и запуск
6. Поддержка, мониторинг и итерации

Подсказка: Большие проекты проходят стадию прототипа перед полномасштабной реализацией.

Ответ:

- Идея, концепция и сбор требований
- Разработка ТЗ и согласование
- Прототип и proof of concept
- Полная реализация с обработкой ошибок и безопасностью
- Тестирование, приёмка и запуск
- Поддержка, мониторинг и итерации

Урок 2 - Оптимизация запросов и архитектуры для работы с большими данными

Шаг 01:

Миссия 1

Вы запускаете новый проект: интеграция PHP-сервиса с Oracle для аналитики больших данных. Какой приём оптимизации стоит заложить уже в ТЗ?

Подсказка: Подумайте о передаче минимального объёма данных.

Ответ:

- **Фильтрация и проекция полей на стороне Oracle**
- Всегда передавать все столбцы таблицы
- Делать join-ы в PHP
- Передавать данные в XML без сжатия

Шаг 02:

Миссия 2

Вы обсуждаете с аналитиками интерфейсы запросов к большим таблицам. Что обязательно указать в ТЗ?

Подсказка: Нужно минимизировать ненужную нагрузку.

Ответ:

- **Возможность пагинации / курсоров / offset-limit**
- Передавать всю страницу данных сразу
- Сортировка только на стороне PHP
- Ограничение только по дате без фильтров

Шаг 03:

Миссия 3

В ТЗ один из разработчиков предлагает: «Всегда использовать курсоры PL/SQL для выборки по частям». Какой риск вы отметите в ТЗ?

Подсказка: Курсоры не всегда оптимальны при больших объёмах.

Ответ:

- **Высокая нагрузка на сессию и накладные расходы на переключения курсора**
- Курсоры плохо работают
- Нельзя использовать курсоры вовсе
- Курсоры всегда медленные во всех сценариях

🔗 Шаг 04:

Миссия 4

При разговорах с DBA вы упоминаете: «Нам нужны материализованные представления». В каком сценарии это стоит указать в ТЗ?

Подсказка: Подумайте про готовые агрегаты и повторные запросы.

Ответ:

- **Когда одни и те же агрегаты часто запрашиваются**
- Когда данные никогда не агрегируются
- Когда материализованные представления всегда свежие
- Когда Oracle не поддерживает материализованные представления

🔗 Шаг 05:

Миссия 5

Сопоставьте приём оптимизации с его эффектом:

Приёмы:

Ответ:

- Индексирование по колонкам фильтрации
- Разделение таблиц (partitioning)
- Материализованные представления (materialized views)
- Вертикальное шардирование (vertical sharding)

Эффекты:

1. Ускорение фильтрации по дате или ключу
2. Уменьшение размера сканируемых сегментов по диапазону
3. Хранение предагрегированных данных для частых запросов
4. Разделить колонки высокого объёма в отдельные таблицы

Подсказка: Подумайте, какой метод даёт быстрые выборки, а какой — уменьшает нагрузку.

- **Индексирование по колонкам фильтрации** — Ускорение фильтрации по дате или ключу
- **Разделение таблиц (partitioning)** — Уменьшение размера сканируемых сегментов по диапазону
- **Материализованные представления (materialized views)** — Хранение предагрегированных данных для частых запросов
- **Вертикальное шардирование (vertical sharding)** — Разделить колонки высокого объёма в отдельные таблицы

🔗 Шаг 06:

Миссия 6

Сопоставьте архитектурные паттерны с ситуациями:

Паттерны:

Ответ:

- MapReduce / Batch processing
- Stream / real-time processing
- Lambda Architecture (гибридный подход)
- Micro-batches

Ситуации:

1. Обработка исторических данных раз в сутки
2. Обработка событий в режиме реального времени
3. Комбинированная обработка: быстрые ответы + исторические слои
4. Обработка мини-пакетов каждые минуты

Подсказка: Подумайте, какой подход нужен для истории, а какой — для стрима.

- **MapReduce / Batch processing** — Обработка исторических данных раз в сутки
- **Stream / real-time processing** — Обработка событий в режиме реального времени
- **Lambda Architecture (гибридный подход)** — Комбинированная обработка: быстрые ответы + исторические слои
- **Micro-batches** — Обработка мини-пакетов каждые минуты

Шаг 07:

Миссия 7

Сопоставьте метрику и её важность в большом нагрузочном сценарии:

Метрики:

Ответ:

- Время ответа (latency)
- Throughput (пропускная способность)
- IOPS / диск
- CPU / память

Важно для:

1. Непрерывной работы при росте нагрузки
2. Ограничения дискового ввода-вывода при чтении больших таблиц
3. Меньших задержек при мелких запросах
4. Стабильной обработки большого числа запросов

Подсказка: Подумайте: latency — это задержка, throughput — сколько операций за время.

- **Время ответа (latency)** — Меньших задержек при мелких запросах
- **Throughput (пропускная способность)** — Стабильной обработки большого числа запросов
- **IOPS / диск** — Ограничения дискового ввода-вывода при чтении больших таблиц
- **CPU / память** — Непрерывной работы при росте нагрузки

Шаг 08:

Миссия 8

Сопоставьте способ масштабирования с его характеристикой:

Способы:

Ответ:

- Вертикальное масштабирование (scale-up)
- Горизонтальное масштабирование (scale-out)
- Многомастер или шардирование
- Read replicas / репликация

Характеристики:

1. Добавление ресурсов к одному узлу
2. Добавление новых узлов к кластеру
3. Разделение данных между узлами с маршрутизацией запросов
4. Использование копий для чтения без записи

Подсказка: Подумайте, что значит «horizontal» и «vertical».

- **Вертикальное масштабирование (scale-up)** — Добавление ресурсов к одному узлу
- **Горизонтальное масштабирование (scale-out)** — Добавление новых узлов к кластеру
- **Многомастер или шардирование** — Разделение данных между узлами с маршрутизацией запросов
- **Read replicas / репликация** — Использование копий для чтения без записи

Шаг 09:

Миссия 9

Упорядочите шаги построения шардинга для большого набора данных:

1. Выбор ключа шардинга
2. Разделение таблиц по диапазонам или хеш-функции
3. Настройка маршрутизации запросов
4. Распределение существующих данных между шардами
5. Обновление клиентской логики и точек доступа

Подсказка: Сначала ключ, затем сегментация, затем маршруты и миграция.

Ответ:

- Выбор ключа шардинга
- Разделение таблиц по диапазонам или хеш-функции
- Настройка маршрутизации запросов
- Распределение существующих данных между шардами
- Обновление клиентской логики и точек доступа

Шаг 10:

Миссия 10

Упорядочите шаги применения разрежённых (sparse) индексов на больших таблицах:

1. Анализ распределения данных (NULL, редкие значения)
2. Выбор колонок для разреженного индекса
3. Создание индекса с параметром sparse / bitmap
4. Тестирование производительности с и без индекса
5. Мониторинг и решение о включении/отключении

Подсказка: Не спешите — сначала анализ, потом реализация и мониторинг.

Ответ:

- Анализ распределения данных (NULL, редкие значения)
- Выбор колонок для разреженного индекса
- Создание индекса с параметром sparse / bitmap
- Тестирование производительности с и без индекса
- Мониторинг и решение о включении/отключении

Шаг 11:

Миссия 11

Упорядочите этапы реализации гибридной (lambda) архитектуры:

1. Построение batch-слоя для исторических расчётов
2. Настройка real-time стрима (очередь, поток)
3. Объединение данных из batch и real-time слоёв
4. Трансформация и нормализация данных в общий формат
5. Обеспечение согласованности между слоями

Подсказка: Lambda — это сочетание исторического и текущего слоёв.

Ответ:

- Построение batch-слоя для исторических расчётов
- Настройка real-time стрима (очередь, поток)
- Объединение данных из batch и real-time слоёв
- Трансформация и нормализация данных в общий формат
- Обеспечение согласованности между слоями

Шаг 12:

Миссия 12

Упорядочите шаги анализа узкого места (bottleneck) в архитектуре:

1. Сбор метрик (CPU, I/O, время ответа)
2. Идентификация узкого компонента (дисковые операции, сеть и т.д.)
3. Проведение тестов при нагрузке именно на этот компонент
4. Применение оптимизации именно к узкому месту
5. Ретест и проверка улучшения

Подсказка: Сначала метрики, потом локализация, потом оптимизация и тест.

Ответ:

- Сбор метрик (CPU, I/O, время ответа)
- Идентификация узкого компонента (дисковые операции, сеть и т.д.)
- Проведение тестов при нагрузке именно на этот компонент
- Применение оптимизации именно к узкому месту
- Ретест и проверка улучшения

Шаг 13:

Миссия 13

Упорядочите шаги внедрения read-replica (реплика только для чтения):

1. Настройка репликации данных Oracle (master → replica)
2. Тест согласованности между мастером и репликой
3. Конфигурация PHP-слоя на чтение с реплики
4. Мониторинг задержки репликации и времени отклика
5. Откат на мастер, если реплика недоступна

Подсказка: Реплика нужна для чтения, но надо контролировать отставание и сбои.

Ответ:

- Настройка репликации данных Oracle (master → replica)

- Тест согласованности между мастером и репликой
- Конфигурация RHP-слоя на чтение с реплики
- Мониторинг задержки репликации и времени отклика
- Откат на мастер, если реплика недоступна

🔮 Шаг 14:

Миссия 14

Упорядочите шаги реализации barbell-архитектуры (двухуровневая оптимизация):

1. Разделение данных на горячий и холодный сегменты
2. Оптимизация горячего сегмента для быстрых запросов
3. Перенос старых данных в холодное хранилище
4. Интеграция горячих и холодных данных в единый интерфейс
5. Мониторинг производительности обоих сегментов

Подсказка: Горячее + холодное — это как два крыла архитектуры.

Ответ:

- Разделение данных на горячий и холодный сегменты
- Оптимизация горячего сегмента для быстрых запросов
- Перенос старых данных в холодное хранилище
- Интеграция горячих и холодных данных в единый интерфейс
- Мониторинг производительности обоих сегментов

📖 Урок 3 - Интеграция внешнего API в систему с Oracle

🔮 Шаг 01:

Миссия 1

Сопоставьте компонент интеграции API с его ролью:

Компоненты:

Ответ:

- Клиент API (HTTP-модуль)
- Слой преобразования / маппинг
- Сервис интеграции
- Логирование и трассировка

Роли:

1. Выполняет запросы к внешнему API и получает ответы
2. Конвертирует данные внешнего формата в модель Oracle / PHP
3. Координирует фазы вызова API, обработку ошибок и логику
4. Записывает трассы, ошибки, время ожидания

Подсказка: Клиент делает HTTP, маппер трансформирует, сервис управляет, логгинг фиксирует.

- **Клиент API (HTTP-модуль)** — Выполняет запросы к внешнему API и получает ответы
- **Слой преобразования / маппинг** — Конвертирует данные внешнего формата в модель Oracle / PHP
- **Сервис интеграции** — Координирует фазы вызова API, обработку ошибок и логику
- **Логирование и трассировка** — Записывает трассы, ошибки, время ожидания

🔮 Шаг 02:

Миссия 2

Сопоставьте стратегию обработки ошибок API с её результатом:

Стратегии:

Ответ:

- Idempotent retries
- Fail fast + откат транзакций
- Отложенная повторная отправка (dead-letter / очередь)
- Fallback на кеш или запасные данные

Результаты:

1. Запрос можно попробовать снова безопасно без дублирования действий
2. При ошибке сразу прерываем и фиксируем проблему
3. Сохраняем задачу в очередь на повторный обработчик позже
4. Вернуть устаревшее значение из кеша, если API упал

Подсказка: Идем: повтор, откат, очередь или fallback.

- **Idempotent retries** — Запрос можно попробовать снова безопасно без дублирования действий
- **Fail fast + откат транзакций** — При ошибке сразу прерываем и фиксируем проблему
- **Отложенная повторная отправка (dead-letter / очередь)** — Сохраняем задачу в очередь на повторный обработчик позже
- **Fallback на кеш или запасные данные** — Вернуть устаревшее значение из кеша, если API упал

🔮 Шаг 03:

Миссия 3

Сопоставьте формат обмена данных API с его преимуществом:

Форматы:

Ответ:

- JSON
- XML
- Protocol Buffers / binary
- CSV / текстовый

Преимущества:

1. Читаемость, легко дебажить
2. Стандарты SOAP, схемы, сложные структуры
3. Компактность, производительность парсинга
4. Простота, совместимость со старыми системами

Подсказка: JSON — читаемо, XML — формально, binary — быстро, CSV — просто.

- **JSON** — Читаемость, легко дебажить
- **XML** — Стандарты SOAP, схемы, сложные структуры
- **Protocol Buffers / binary** — Компактность, производительность парсинга
- **CSV / текстовый** — Простота, совместимость со старыми системами

Шаг 04:

Миссия 4

Сопоставьте параметры API-запроса с их функциями:

Параметры:

Ответ:

- limit / offset (pagination)
- filter (условие)
- sort (порядок)
- fields (проекция полей)

Функции:

1. Ограничивает число записей и смещение
2. Фильтрует записи по условиям
3. Определяет сортировку результата
4. Указывает, какие поля вернуть

Подсказка: API часто поддерживает pagination, фильтры, сортировку и проекцию.

- **limit / offset (pagination)** — Ограничивает число записей и смещение
- **filter (условие)** — Фильтрует записи по условиям
- **sort (порядок)** — Определяет сортировку результата
- **fields (проекция полей)** — Указывает, какие поля вернуть

Шаг 05:

Миссия 5

Расположите шаги интеграции API → Oracle:

1. Получение ответа от API
2. Проверка статуса ответа и ошибок
3. Преобразование / валидация данных
4. Запись данных в Oracle / обновление
5. Логирование результата / отправка ответа клиенту

Подсказка: Сначала API, потом проверка, трансформация, запись, лог.

Ответ:

- Получение ответа от API
- Проверка статуса ответа и ошибок
- Преобразование / валидация данных
- Запись данных в Oracle / обновление
- Логирование результата / отправка ответа клиенту

Шаг 06:

Миссия 6

Упорядочите шаги обеспечения безопасности при интеграции внешнего API:

1. Шифрование трафика (HTTPS / TLS)
2. Аутентификация и авторизация (OAuth, ключи)
3. Валидация и фильтрация входных данных
4. Ограничение скорости (rate limiting / throttling)

5. Логирование доступа и мониторинг аномалий

Подсказка: Безопасность начинается с трафика и удостоверений, затем — фильтры, лимиты, лог.

Ответ:

- Шифрование трафика (HTTPS / TLS)
- Аутентификация и авторизация (OAuth, ключи)
- Валидация и фильтрация входных данных
- Ограничение скорости (rate limiting / throttling)
- Логирование доступа и мониторинг аномалий

Шаг 07:

Миссия 7

Расположите шаги обработки батчевого импорта из API в Oracle:

1. Разбивка полученных данных на порции / чанки
2. Параллельная отправка запросов API (batch calls)
3. Обработка ответов и валидация
4. Запись в Oracle (bulk insert / merge)
5. Логирование ошибок и резюме

Подсказка: При больших данных — чанки, параллелизм, проверка, запись, лог.

Ответ:

- Разбивка полученных данных на порции / чанки
- Параллельная отправка запросов API (batch calls)
- Обработка ответов и валидация
- Запись в Oracle (bulk insert / merge)
- Логирование ошибок и резюме

Шаг 08:

Миссия 8

Упорядочите шаги обработки несоответствия версии API (версии несовпадают):

1. Проверка версии API в ответе / заголовках
2. Сравнение с поддерживаемой версией в системе
3. **Если несовпадение** — fallback или предупреждение
4. Маршрут к корректному эндпоинту или адаптация маппера
5. Логирование события несовместимости

Подсказка: Версия влияет на контракт, поэтому сперва проверь, затем адаптируй или уподи.

Ответ:

- Проверка версии API в ответе / заголовках
- Сравнение с поддерживаемой версией в системе
- **Если несовпадение** — fallback или предупреждение
- Маршрут к корректному эндпоинту или адаптация маппера
- Логирование события несовместимости

Шаг 09:

Миссия 9

Расположите фазы тестирования интеграции API с Oracle:

1. Unit-тесты HTTP-модуля и мапперов
2. Интеграционные тесты с живым API или моком
3. Тесты отказов, таймаутов и retry
4. Тесты нагрузочной выдержки
5. Приёмочные тесты и проверка соответствия ТЗ

Подсказка: Тестируем от мелкого к глобальному: модуль → интеграция → отказ → нагрузка → приемка.

Ответ:

- Unit-тесты HTTP-модуля и мапперов
- Интеграционные тесты с живым API или моком
- Тесты отказов, таймаутов и retry
- Тесты нагрузочной выдержки
- Приёмочные тесты и проверка соответствия ТЗ

Шаг 10:

Миссия 10

Упорядочите этапы миграции от тестового API к продакшн:

1. Переход конфигурации URL / ключей до продакшн
2. Повторная проверка схемы ответов и адаптация к продакшн-версии
3. Наблюдение логов и метрик после запуска
4. Фоллбэк на предыдущую версию при ошибках
5. Оповещение команды о переключении

Подсказка: Переход на прод — это конфигурация, проверка, мониторинг, откат, уведомления.

Ответ:

- Переход конфигурации URL / ключей до продакшн
- Повторная проверка схемы ответов и адаптация к продакшн-версии
- Наблюдение логов и метрик после запуска
- Фоллбэк на предыдущую версию при ошибках
- Оповещение команды о переключении

Шаг 11:

Миссия 11

Расположите шаги запуска гибридной схемы (polling + webhook) интеграции:

1. Настройка webhook на стороне внешнего API
2. Создание fallback механизма polling (опрос API по расписанию)
3. Обработка уведомлений вебхуков в PHP
4. Синхронизация данных через polling, если webhook не доставлен
5. Ведение лога доставки уведомлений и повторов

Подсказка: webhook + polling — это надёжность в сочетании.

Ответ:

- Настройка webhook на стороне внешнего API
- Создание fallback механизма polling (опрос API по расписанию)
- Обработка уведомлений вебхуков в PHP

- Синхронизация данных через polling, если webhook не доставлен
- Ведение лога доставки уведомлений и повторов

🔧 Шаг 12:

Миссия 12

Упорядочите этапы полномасштабной интеграции API + Oracle:

1. Сбор требований и исследование API
2. Разработка ТЗ интеграции (контракты, логика ошибок, трансформация)
3. Реализация клиента, мапперов и сервисного слоя
4. Тестирование (unit, интеграция, нагрузка, отказ)
5. Развёртывание и мониторинг в продуктиве
6. Итеративное улучшение и адаптация к изменениям API

Подсказка: Полная интеграция — от идеи до поддержки и эволюции.

Ответ:

- Сбор требований и исследование API
- Разработка ТЗ интеграции (контракты, логика ошибок, трансформация)
- Реализация клиента, мапперов и сервисного слоя
- Тестирование (unit, интеграция, нагрузка, отказ)
- Развёртывание и мониторинг в продуктиве
- Итеративное улучшение и адаптация к изменениям API

📖 Урок 4 - Профилирование и улучшение производительности базы данных

🔧 Шаг 01:

Миссия 6

Сопоставьте тип wait event или задержки с причиной:

Типы:

Ответ:

- db file sequential read
- db file scattered read
- buffer busy waits
- CPU usage

Причины:

1. Индексное чтение одного блока за раз
2. Чтение множества блоков (full scan) в нескольких блоках
3. Соревнование за буферный кэш
4. Высокая загрузка процессора операций вычислений

Подсказка: «scattered» — множество блоков, «sequential» — линейное чтение.

- **db file sequential read** — Индексное чтение одного блока за раз
- **db file scattered read** — Чтение множества блоков (full scan) в нескольких блоках
- **buffer busy waits** — Соревнование за буферный кэш
- **CPU usage** — Высокая загрузка процессора операций вычислений

🔗 Шаг 02:

Миссия 7

Сопоставьте индексационный метод с его применением:

Методы:

Ответ:

- B-tree индекс
- Bitmap индекс
- Function-based индекс
- Объединённый составной индекс (composite)

Применения:

1. Для точных сравнений и равенств
2. Для столбцов с низкой кардинальностью
3. Для выражений / преобразований столбцов
4. Когда фильтруют по нескольким колонкам одновременно

Подсказка: Composite объединяет колонки, function-based — выражения.

- **B-tree индекс** — Для точных сравнений и равенств
- **Bitmap индекс** — Для столбцов с низкой кардинальностью
- **Function-based индекс** — Для выражений / преобразований столбцов
- **Объединённый составной индекс (composite)** — Когда фильтруют по нескольким колонкам одновременно

🔗 Шаг 03:

Миссия 8

Сопоставьте стратегию оптимизации с её эффектом:

Стратегии:

Ответ:

- Партирование (partitioning)
- Материализованные представления (materialized views)
Кэширование результатов запросов
- Обновление статистик / сбор статистики (gather stats)

Эффекты:

1. Уменьшение объёма сканируемых данных
2. Преагрегаты для частых запросов
3. Избежать повторного выполнения тяжёлых запросов
4. Повысить точность планировщика запросов

Подсказка: Partition уменьшает сканы, MVs — преагрегаты, кэш — переиспользование, статистика — планы.

- **Партирование (partitioning)** — Уменьшение объёма сканируемых данных
- **Материализованные представления (materialized views)** — Преагрегаты для частых запросов
- **Кэширование результатов запросов** — Избежать повторного выполнения тяжёлых запросов

- **Обновление статистик / сбор статистики (gather stats)** — Повысить точность планировщика запросов

🔮 Шаг 04:

Миссия 9

Сопоставьте причину деградации производительности с её симптомом:

Причины:

Ответ:

- Неверные / устаревшие статистики
- Сильные конкуренции (lock / waits)
- Неподходящие планы запросов (bad plan)
- Неоптимальные конструкции SQL (подзапросы, корреляции)

Симптомы:

1. Длительное повышение времени выполнения после изменений
2. Высокие ожидания блокировок / wait events
3. Резкий спад производительности на одних версиях, но не на других
4. Запросы выполняются медленно с вложенными SELECT'ами

Подсказка: План может стать плохим без статистики, SQL конструкции влияют на логику.

- **Неверные / устаревшие статистики** — Резкий спад производительности на одних версии, но не на других
- **Сильные конкуренции (lock / waits)** — Высокие ожидания блокировок / wait events
- **Неподходящие планы запросов (bad plan)** — Длительное повышение времени выполнения после изменений
- **Неоптимальные конструкции SQL (подзапросы, корреляции)** — Запросы выполняются медленно с вложенными SELECT'ами

🔮 Шаг 05:

Миссия 10

Упорядочите шаги для анализа и оптимизации медленного SQL:

1. Сбор execution plan и статистик
2. Идентификация узких мест в плане (full scan, nested loops и т.д.)
3. Корректировка SQL / добавление индексов / переписывание запросов
4. Сбор новых статистик и пересчёт плана
5. Сравнение времени до и после и мониторинг

Подсказка: Процесс: понимание → проблема → изменение → проверка → контроль.

Ответ:

- Сбор execution plan и статистик
- Идентификация узких мест в плане (full scan, nested loops и т.д.)
- Корректировка SQL / добавление индексов / переписывание запросов
- Сбор новых статистик и пересчёт плана
- Сравнение времени до и после и мониторинг

🔮 Шаг 06:

Миссия 11

Расположите шаги профилирования на уровне PHP + Oracle:

1. Сбор медленных запросов из кода (log slow SQL)
2. Сопоставление запросов к планам в Oracle
3. Оптимизация SQL / переписывание ORM-запросов
4. Повторное тестирование и измерение скорости
5. Мониторинг и оповещение о проблемах

Подсказка: Приложение → база → смена SQL → проверка → контроль.

Ответ:

- Сбор медленных запросов из кода (log slow SQL)
- Сопоставление запросов к планам в Oracle
- Оптимизация SQL / переписывание ORM-запросов
- Повторное тестирование и измерение скорости
- Мониторинг и оповещение о проблемах

Шаг 07:

Миссия 12

Расположите шаги применения параллелизма или параллельных запросов:

1. Определение тяжёлых SQL-запросов, пригодных к параллелизации
2. Настройка параметров параллельности Oracle (parallel_degree, etc.)
3. Переписывание запросов с hint'ами PARALLEL
4. Тестирование под нагрузкой с разным числом потоков
5. Мониторинг улучшения и эффектов конкуренции

Подсказка: Только подходящие запросы выиграют от параллелизма.

Ответ:

- Определение тяжёлых SQL-запросов, пригодных к параллелизации
- Настройка параметров параллельности Oracle (parallel_degree, etc.)
- Переписывание запросов с hint'ами PARALLEL
- Тестирование под нагрузкой с разным числом потоков
- Мониторинг улучшения и эффектов конкуренции

Шаг 08:

Миссия 13

Упорядочите шаги использования partitioning для масштабируемости:

1. Анализ ключей разбивки (range, list, hash)
2. Создание партиций / изменение таблицы на partitioned
3. Роутинг запросов по партициям (при добавлении WHERE фильтров)
4. Реорганизация данных в партиции
5. Мониторинг выгоды: уменьшение latency и IO

Подсказка: Партиционирование должно сопровождаться маршрутом и мониторингом.

Ответ:

- Анализ ключей разбивки (range, list, hash)
- Создание партиций / изменение таблицы на partitioned
- Роутинг запросов по партициям (при добавлении WHERE фильтров)

- Реорганизация данных в партиции
- Мониторинг выгоды: уменьшение latency и IO

Шаг 09:

Миссия 14

Расположите шаги применения adaptive cursor sharing / bind peeking:

1. Обнаружение проблем из-за bind variable peeking
2. Активировать adaptive cursor sharing или использование bind_statistics
3. Тестирование планов под разными bind-значениями
4. Отключение или настройка параметров при ухудшении
5. Мониторинг стабильности планов

Подсказка: Неправильное поведение bind может портить планы запросов.

Ответ:

- Обнаружение проблем из-за bind variable peeking
- Активировать adaptive cursor sharing или использование bind_statistics
- Тестирование планов под разными bind-значениями
- Отключение или настройка параметров при ухудшении
- Мониторинг стабильности планов

Шаг 10:

Миссия 15

Расположите фазы внедрения индекс-организованных таблиц (IOT) для оптимизации:

1. Выбор таблиц с частыми поисками по ключу
2. Преобразование таблиц в IOT или создание IOT
3. Адаптация SQL-запросов к IOT (убрать лишние joins)
4. Тестирование производительности и сравнение
5. Мониторинг использования и откат, если ухудшение

Подсказка: IOT хороши при частом доступе по ключу.

Ответ:

- Выбор таблиц с частыми поисками по ключу
- Преобразование таблиц в IOT или создание IOT
- Адаптация SQL-запросов к IOT (убрать лишние joins)
- Тестирование производительности и сравнение
- Мониторинг использования и откат, если ухудшение

Шаг 11:

Миссия 16

Расположите шаги при настройке Resource Manager / resource groups:

1. Определение групп ресурсов (например, reporting, OLTP)
2. Настройка лимитов CPU / I/O для групп
3. Назначение сессий к группам ресурсов
4. Тестирование под нагрузкой, чтобы проверить изоляцию
5. Мониторинг и корректировка границ

Подсказка: Resource Manager помогает ограничивать влияние тяжёлых клиентов.

Ответ:

- Определение групп ресурсов (например, reporting, OLTP)
- Настройка лимитов CPU / I/O для групп
- Назначение сессий к группам ресурсов
- Тестирование под нагрузкой, чтобы проверить изоляцию
- Мониторинг и корректировка границ

 Шаг 12:

Миссия 17

Упорядочите этапы полного процесса профилирования и улучшения производительности:

1. Сбор базовых метрик (latency, waits, I/O, CPU)
2. Идентификация «горячих» запросов и узких мест
3. Сбор планов, статистики, трассировок
4. Внедрение оптимизаций (индексы, переписывание SQL, partitioning и др.)
5. Ретестирование и сравнение метрик
6. Мониторинг в эксплуатации и итеративное улучшение

Подсказка: Оптимизация — не однократное действие, а цикл.

Ответ:

- Сбор базовых метрик (latency, waits, I/O, CPU)
- Идентификация «горячих» запросов и узких мест
- Сбор планов, статистики, трассировок
- Внедрение оптимизаций (индексы, переписывание SQL, partitioning и др.)
- Ретестирование и сравнение метрик
- Мониторинг в эксплуатации и итеративное улучшение

 Урок 5 - Подготовка и защита финального проекта

 Шаг 01:

Миссия 1

Вы работаете с проектом, где PHP должен подключаться к Oracle через PDO. Какой параметр важен для правильного подключения?

Подсказка: DSN — это строка, которая содержит данные для подключения к базе.

Ответ:

- **Использование правильного DSN (Data Source Name) для Oracle**
- Использование mysql_connect вместо PDO
- Открытие соединения через ftp

 Шаг 02:

Миссия 2

Какой метод PDO используется для безопасного выполнения SQL-запроса с параметрами?

Подсказка: Ищите метод, который позволяет избежать SQL-инъекций.

Ответ:

- **prepare()**
- exec()
- query()

- connect()

Шаг 03:

Миссия 3

При выполнении большого проекта команда обнаружила утечки памяти при работе с Oracle через PHP. Что наиболее вероятно вызвало эту проблему?

Подсказка: Не забывайте про правильное закрытие соединений и освобождение ресурсов.

Ответ:

- **Отсутствие закрытия курсоров и соединений**
- Использование prepared statements
- Неверный DSN

Шаг 04:

Миссия 4

 Сопоставьте термины и их описания, связанные с интеграцией PHP и Oracle.

Термины:

Ответ:

- PDO
- OCI8
- PL/SQL
- DSN

Определения:

1. Расширение PHP для работы с Oracle через объектно-ориентированный интерфейс.
2. Нативный драйвер Oracle для PHP, обеспечивающий прямое взаимодействие с базой.
3. Язык процедурного программирования для Oracle, используется для написания хранимых процедур.
4. Строка подключения, содержащая параметры для подключения к базе данных.

Подсказка:

Подумайте, какие технологии отвечают за разные части взаимодействия PHP с Oracle.

- **PDO** — Расширение PHP для работы с Oracle через объектно-ориентированный интерфейс.
- **OCI8** — Нативный драйвер Oracle для PHP, обеспечивающий прямое взаимодействие с базой.
- **PL/SQL** — Язык процедурного программирования для Oracle, используется для написания хранимых процедур.
- **DSN** — Строка подключения, содержащая параметры для подключения к базе данных.

Шаг 05:

Миссия 5

 Соотнесите виды ошибок и возможные причины при интеграции PHP с Oracle.

Виды ошибок:

Ответ:

- ORA-12154
- ORA-00942
- ORA-01017

- ORA-03113

Причины:

1. Неверное имя пользователя или пароль
2. Проблема с подключением к базе (TNS не найден)
3. Попытка доступа к несуществующей таблице
4. Неожиданное закрытие соединения

 Подсказка:

Обратите внимание на стандартные Oracle коды ошибок и их описание.

- **ORA-12154** — Проблема с подключением к базе (TNS не найден)
- **ORA-00942** — Попытка доступа к несуществующей таблице
- **ORA-01017** — Неверное имя пользователя или пароль
- **ORA-03113** — Неожиданное закрытие соединения

 Шаг 06:

Миссия 6

 Сопоставьте типы запросов с их описанием в контексте PHP и Oracle.

Типы запросов:

Ответ:

- SELECT
- INSERT
- UPDATE
- DELETE

Описание:

1. Извлекает данные из базы данных.
2. Добавляет новые записи в таблицу.
3. Изменяет существующие записи.
4. Удаляет записи из таблицы.

 Подсказка:

Помните, что каждый тип SQL-запроса отвечает за определённую операцию с данными.

- **SELECT** — Извлекает данные из базы данных.
- **INSERT** — Добавляет новые записи в таблицу.
- **UPDATE** — Изменяет существующие записи.
- **DELETE** — Удаляет записи из таблицы.

 Шаг 07:

Миссия 7

Расположите последовательность действий при настройке подключения PHP к Oracle через PDO:

1. Создать объект PDO с правильным DSN
2. Обработать исключения при попытке подключения
3. Подключиться к базе данных
4. Настроить параметры подключения (логин, пароль)

 Подсказка:

Вспомните, что первым создаётся объект, но без правильных параметров подключения это не сработает.

Ответ:

- Настроить параметры подключения (логин, пароль)
- Создать объект PDO с правильным DSN
- Подключиться к базе данных
- Обработать исключения при попытке подключения

Шаг 08:

Миссия 8

Упорядочите этапы работы с подготовленным запросом в PHP и Oracle:

1. Выполнить запрос
2. Подготовить SQL с помощью prepare()
3. Связать параметры
4. Обработать результаты

 **Подсказка:**

Сначала запрос нужно подготовить, потом связать параметры, выполнить и обработать.

Ответ:

- Подготовить SQL с помощью prepare()
- Связать параметры
- Выполнить запрос
- Обработать результаты

Шаг 09:

Миссия 9

Отсортируйте этапы отладки интеграции PHP и Oracle при ошибках подключения:

1. Проверить настройки TNS и DSN
2. Проверить правильность логина и пароля
3. Проверить наличие сети и доступность сервера
4. Посмотреть логи ошибок и исключения

 **Подсказка:**

Сначала проверяем параметры подключения, потом сеть, затем логин, и в конце - логи для деталей.

Ответ:

- Проверить настройки TNS и DSN
- Проверить наличие сети и доступность сервера
- Проверить правильность логина и пароля
- Посмотреть логи ошибок и исключения

Шаг 10:

Миссия 10

Какой из вариантов лучше всего поможет оптимизировать выполнение большого количества запросов к Oracle из PHP?

Подсказка: Подумайте про уменьшение количества соединений и повторное использование ресурсов.

Ответ:

- **Использование постоянных соединений (persistent connections) через PDO**
- Открытие нового соединения для каждого запроса
- Использование файлового кэша вместо базы

Шаг 11:

Миссия 11

Вы разрабатываете масштабируемое приложение. Какой способ обработки ошибок в PDO Oracle предпочтителен для отладки и продакшена?

Подсказка: Важна информативность ошибок для разработчика и безопасность для пользователя.

Ответ:

- **Использовать режим выбрасывания исключений (PDO::ERRMODE_EXCEPTION) с логированием**
- Игнорировать ошибки и продолжать выполнение
- Выводить ошибки напрямую пользователю

Шаг 12:

Миссия 12

 Сопоставьте виды SQL-инъекций и способы их предотвращения при работе с PHP и Oracle.

Виды инъекций:

Ответ:

- In-band SQL Injection
- Inferential SQL Injection
- Out-of-band SQL Injection
- Prepared Statements

Способы предотвращения:

1. Использование параметризованных запросов с bind-параметрами
2. Анализ и фильтрация пользовательских данных
3. Мониторинг сетевого трафика и логов
4. Извлечение данных через прямой ответ сервера

 **Подсказка:**

Помните, что подготовленные выражения — это лучший способ защититься от большинства видов инъекций.

- **In-band SQL Injection** — Извлечение данных через прямой ответ сервера
- **Inferential SQL Injection** — Анализ и фильтрация пользовательских данных
- **Out-of-band SQL Injection** — Мониторинг сетевого трафика и логов
- **Prepared Statements** — Использование параметризованных запросов с bind-параметрами

Шаг 13:

Миссия 13

 Сопоставьте методы PDO с их назначением при работе с Oracle:

Методы:

Ответ:

- `fetch()`
- `execute()`
- `bindParam()`
- `prepare()`

Назначение:

1. Подготовка SQL-запроса к выполнению
2. Связывание параметра с переменной
3. Выполнение подготовленного запроса
4. Получение строки результата

 **Подсказка:**

Помните стандартный жизненный цикл выполнения запроса с PDO.

- **`prepare()`** — Подготовка SQL-запроса к выполнению
- **`bindParam()`** — Связывание параметра с переменной
- **`execute()`** — Выполнение подготовленного запроса
- **`fetch()`** — Получение строки результата

 **Шаг 14:**

Миссия 14

Какой формат данных в Oracle следует использовать для хранения больших текстовых блоков, чтобы эффективно работать с ними через PHP?

Подсказка: Рассмотрите возможности работы с CLOB.

Ответ:

- **CLOB (Character Large Object)**
- BLOB (Binary Large Object)
- VARCHAR2(4000)

 **Шаг 15:**

Миссия 15

Что из перечисленного поможет эффективно управлять транзакциями в PHP при работе с Oracle?

Подсказка: Обратите внимание на методы PDO, которые управляют транзакциями.

Ответ:

- **Методы `beginTransaction()`, `commit()` и `rollBack()`**
- Метод `exec()` для всех запросов
- Отключение PDO от базы при каждом запросе
- Использование только SELECT-запросов

 **Шаг 16:**

Миссия 16

Какой из следующих способов лучше всего обеспечивает безопасность подключения PHP к Oracle?

Подсказка: Подумайте о хранении учетных данных и шифровании трафика.

Ответ:

- **Использование зашифрованных соединений (например, Oracle Advanced Security) и безопасное хранение паролей**
- Хранение паролей в открытом виде в коде
- Передача учетных данных через GET-параметры URL

Шаг 17:

Миссия 17

Расположите шаги для настройки окружения PHP для работы с Oracle OCI8 расширением:

1. Установить Oracle Instant Client
2. Настроить переменные окружения (LD_LIBRARY_PATH или PATH)
3. Установить и активировать расширение OCI8 в PHP
4. Перезапустить веб-сервер

Подсказка:

Правильная последовательность начинается с установки клиента, затем настройка окружения, установка расширения и перезапуск сервера.

Ответ:

- Установить Oracle Instant Client
- Настроить переменные окружения (LD_LIBRARY_PATH или PATH)
- Установить и активировать расширение OCI8 в PHP
- Перезапустить веб-сервер

Шаг 18:

Миссия 18

Какой из форматов FETCH лучше всего подходит для получения результатов запроса в виде ассоциативного массива в PDO при работе с Oracle?

Подсказка: Выберите константу PDO, которая задаёт тип возвращаемого массива.

Ответ:

- **PDO::FETCH_ASSOC**
- PDO::FETCH_NUM
- PDO::FETCH_OBJ

Шаг 19:

Миссия 19

При работе с большими объемами данных из Oracle в PHP, что поможет улучшить производительность и снизить нагрузку на память?

Подсказка: Подумайте о способах поэтапной загрузки данных.

Ответ:

- **Использование курсоров и fetch по частям (fetch с лимитом)**
- Загрузка всех данных в память одним запросом
- Повторное выполнение одного и того же запроса без изменений